

MACHINE LEARNING IN FINANCE

MODULE 4 | LESSON 1

ENSEMBLE LEARNING

Reading Time 18 minutes

Prior Knowledge Naive Bayes, Support Vector Machines, Decision Tree, Train and Test set.

Keywords Blending, Bagging, Stacking, Validation.

*Thus far, we have covered a variety of machine learning techniques specifically classifiers. Oftentimes, we look for various opinions and then use the majority to guide our decision in the end. Think of ratings/feedback on a product advertised online. Ensemble learning works in a similar way. Decisions from various classifiers are combined and a majority vote or weighted average is taken as the final decision. This has some benefits such as improving the accuracy, reducing the generalization error of the individual classifiers (Opitz and Maclin 169) and reducing the variance error (Ali and Pazzani 173) without compromising bias. We've touched on ensemble learning in Module 3 when we mentioned boosting, the idea of combining weak learners to make a single strong one. We look at an ensemble method, specifically **Bagging** in the next section.*

```
In [ ]: import pandas as pd
```

1. Bagging

Bagging or bootstrap aggregation is a way to reduce variance within a noisy dataset. In this process, weak classifiers are trained independently on a sample instead of the complete training set. These samples are not necessarily smaller than the training set; however, the sample size is often the same size as the training set to ensure each classifier has sufficient data to train. The sample is drawn with replacement; hence, observations can be exposed to more than one classifier and samples would have observations more than once. The weak classifiers do not necessarily have the same predictors or features. Each weak classifier will likely have different ways of predicting target classes not only on different features but also different splits on the same feature since the samples are different. This diversity among the weak learners is caused by an *inducer*. A stable inducer results in similar weak learners, which produce similar predictions that are unlikely to improve the performance of the model. It is therefore advisable to use a relatively unstable inducer that causes small perturbations to the training set. The weak learners are also homogeneous, i.e., they are of the same model type, such as all being trees for example. There is also an added benefit of parallelizing

the process since classifiers are trained on independent samples. In the end, an average for regression, or majority prediction for classification, is used as the output. This majority voting for classification is known as hard voting or majority voting.

1.1. Bagging vs Boosting

We've touched on boosting in Module 3, and we will go through it in more detail in Lessons 3 and 4 of this module. At a high level, it is worth pausing and reflecting on the differences between these two methods. Figure 1 illustrates a summary of the methodology of Bagging and Boosting. Bagging is a parallel process whereby each base learner is not dependent on another, and the dataset for each is also independently sampled. Boosting on the other hand is done sequentially such that each base learner is developed based on the previous learner. With boosting, the training dataset, or samples fed into the base learner, is derived from the predictions of the previous base learner.

Figure 1: Bagging vs Boosting

Base learner icon has been designed using resources from Flaticon.com

The downfall of an ensemble learner is the lack of interpretation of the model. It is not clear as to the decision flow through features as in a decision tree classifier since it is various classifiers that resulted in the final decision. There are methods such as **SHAP** and **LIME** that have overcome this problem and removed the black-box element of ensemble models, however accurate methods like SHAP are computationally expensive and can take a long time especially as observations that require interpreting increases. We look at another ensemble learning method called *stacking* in the next section.

2. Stacking

Figure 2: Datasets for Building a Model

To understand stacking, we first need to understand the validation data and model build set. Figure 2 shows a full dataset commonly split between a test and training set shown by A and B respectively. This is often done in Python with the `train_test_split` function from `sklearn`. We will assume we are working with a binary target with class 0 and 1. To provide some context, let's assume our full dataset has 10,000 observations. We split our data into 75/25 Train/Test split, so A and B have 2,500 and 7,500 observations respectively. The training set, B, is then further split into a 20/80 split, comprising the validation (C) and build (D) set, such that C has 1,500 observations and D has 6,000 observations. Dataset D is used to train a model, and the model's performance is assessed on C. Keep in mind that dataset D is also commonly referred to as the training set since we train our classifiers on this set; however, we use *Build* to distinguish this from

B. We can use C and D to build the model since it allows for fine-tuning a model and reduces the chances of overfitting.

With these datasets in mind, let's suppose we train 3 classifiers such as Naive Bayes (Clf1), SVM (Clf2), and a decision tree (Clf3) on D. We then apply these classifiers on the validation set, C, and obtain the following:

```
In [ ]: df = pd.read_csv("meta_dat2train.csv", index_col=False)
df
```

The last column "Y" is the actual class value. Looking at the dataset above, we can use columns Clf1 to Clf3 as inputs into another model with target variable Y. This model that we develop is called a *Meta* or *Level-1* model and the approach described is called *blending*. Datasets C and D can be randomly shuffled into different 20/80 arrangements say \$k\$. A term to describe this is called *k-fold cross-validation*. This can be shown in Figure 3. We can develop the Meta model on the k^{th} bucket and use the remaining $k-1$ buckets to train the 3 classifiers, Clf1-3. This approach over *k-folds* is known as *stacking* and is the main difference between blending. The classifiers Clf1-3 are known as base models, and this does not have to be restricted to 3, as in our example. You'll notice that 3 different families of classifiers were used, which is different from bagging. This implies that base or individual learners for stacking can be heterogeneous.

Figure 3: K-Fold Cross-Validation

Source: Gufosowa, Wikimedia Commons, September 2019, https://commons.wikimedia.org/wiki/File:K-fold_cross_validation_EN.svg.

3. Conclusion

Ensemble learning is based on the principle that many weak learners combined provide a better estimate than a single learner. This method often improves the bias-variance tradeoff problem. We've mentioned two popular ensemble learning methods, namely Bagging and Stacking. Bagging is restricted to homogeneous learners whereas stacking is not. In the next lesson, we will implement an example of each method in Python.

References

- Ali, Kamal M., and Michael J. Pazzani. "Error Reduction through Learning Multiple Descriptions." *Machine Learning*, vol. 24, 1996, pp. 173–202.
- Opitz, David W., and Richard Maclin. "Popular Ensemble Methods: An Empirical Study." *Journal of Artificial Intelligence Research*, vol. 11, 1999, pp. 169–172.

MACHINE LEARNING IN FINANCE

MODULE 4 | LESSON 2

ENSEMBLE LEARNING IN PRACTICE

Reading Time 15 minutes

Prior Knowledge Bagging, Stacking, SVM, Decision Trees, Naïve Bayes, Logistic regression, ROC curve, AUC.

Keywords Base Learners, Meta Model.

In the previous lesson, we introduced two ensemble learning methods, namely bagging and stacking. This lesson will put these two methods into practice using Python.

1. Introduction

In this predictive problem, we apply ensemble methods to predict whether the Luxembourg index (LUXXX) exceeds a return of 0.25% in any direction. The predictors are a combination of country indices and technical indicators. Let's begin by importing the data.

```
In [ ]: import warnings

import numpy as np
import pandas as pd

warnings.filterwarnings("ignore")

data_df = pd.read_csv("MScFE 650 MLF GWP Data.csv")
# Convert string to datetime
data_df["Date"] = pd.to_datetime(data_df["Date"])
```

We create the target variable as before.

```
In [ ]: # Set Target Index for predicting
target ETF = "LUXXX"

# Use returns instead of prices for other Indices
# Other Indices used as Index_features
ETF_features = data_df.loc[:, ~data_df.columns.isin(["Date", target ETF])]
data_df[ETF_features] = data_df[ETF_features].pct_change()

data_df[target ETF + "_returns"] = data_df[target ETF].pct_change()

# Create Target Column.
```

```

# Shift period for target column
data_df[target_ETF + "_returns" + "_Shift"] = data_df[target_ETF + "_returns"].s

# Strategy to take long position for anticipated returns of 0.5%
data_df["Target"] = np.where(
    (data_df[target_ETF + "_returns_Shift"].abs() > 0.025), 1, 0
)

```

Create the predictors below, namely four country indices and three technical indicators: slow to fast moving average ratio (SMA_ratio), relative strength index (RSI), and rate of change (RC).

```

In [ ]: # Four country indices used.
feats = ["MSCI KOREA", "MSCI DENMARK", "MSCI FRANCE", "MSCI NORWAY"]

# creating the technical indicators
data_df["SMA_5"] = data_df[target_ETF].rolling(5).mean()
data_df["SMA_15"] = data_df[target_ETF].rolling(15).mean()
data_df["SMA_ratio"] = data_df["SMA_15"] / data_df["SMA_5"]

# Can drop SMA columns since not needed anymore.
data_df.drop(["SMA_5", "SMA_15"], axis=1, inplace=True)

# shift the price of the target by 1 unit previous in time
data_df["Diff"] = data_df[target_ETF] - data_df[target_ETF].shift(1)
data_df["Up"] = data_df["Diff"]
data_df.loc[(data_df["Up"] < 0), "Up"] = 0

data_df["Down"] = data_df["Diff"]
data_df.loc[(data_df["Down"] > 0), "Down"] = 0
data_df["Down"] = abs(data_df["Down"])

data_df["avg_5up"] = data_df["Up"].rolling(5).mean()
data_df["avg_5down"] = data_df["Down"].rolling(5).mean()

data_df["avg_15up"] = data_df["Up"].rolling(15).mean()
data_df["avg_15down"] = data_df["Down"].rolling(15).mean()

data_df["RS_5"] = data_df["avg_5up"] / data_df["avg_5down"]
data_df["RS_15"] = data_df["avg_15up"] / data_df["avg_15down"]

data_df["RSI_5"] = 100 - (100 / (1 + data_df["RS_5"]))
data_df["RSI_15"] = 100 - (100 / (1 + data_df["RS_15"]))

data_df["RSI_ratio"] = data_df["RSI_5"] / data_df["RSI_15"]

# Can drop RS Calc columns
data_df.drop(
    ["Diff", "Up", "Down", "avg_5up", "avg_5down", "avg_15up", "avg_15down"],
    axis=1,
    inplace=True,
)

data_df["RC"] = data_df[target_ETF].pct_change(15)

# all_feats
feats.append("SMA_ratio")

```

```
feats.append("RSI_ratio")
feats.append("RC")
```

Now that we have our data, we can apply the ensemble methods beginning with bagging.

2. Bagging with Random Forest

An example of a classifier that uses the bagging algorithm is the random forest classifier. Random forest is a tree-based algorithm; therefore, the weak learners are decision trees. The "random" applies more to the random features allocated to each decision tree; hence, not all features are used by the weak learners. The different subset of features and different samples used by the decision trees also result in uncorrelated trees, which improves the performance of the algorithm. The final outcome is simply a majority vote among the weak learners. For instance, if there are five decision trees in the random forest and three trees predict class 1 while two trees predict class 0, the random forest classification would be class 1 due to the majority prediction. We begin by importing the necessary libraries.

```
In [ ]: from sklearn import model_selection
        from sklearn.ensemble import RandomForestClassifier
```

Perform the train/test split of 80/20.

```
In [ ]: # Train/Test split
        # Train/Test split. No NaNs in the data.
        NoNaN_df = data_df.dropna()
        X = NoNaN_df[feats]

        X = X.iloc[:, :] # .values
        y = NoNaN_df.loc[:, "Target"] # .values

        del NoNaN_df

        # from sklearn.cross_validation import train_test_split
        X_train, X_test, y_train, y_test = model_selection.train_test_split(
            X, y, test_size=0.2, random_state=0
        )
```

We now create the random forest classifier. We call it our bag model since it is our bagging classifier. Among the hyperparameters specified is `n_estimators`, which specifies the number of trees. To reduce the time taken to train the classifier, we use a small number of 10 trees. A large number of trees will not lead to overfitting according to James (341).

```
In [ ]: bagmodel = RandomForestClassifier(n_estimators=10, random_state=4)
```

We train the model and look at its accuracy on the train and test set.

```
In [ ]: bagmodel.fit(X_train, y_train)
```

```
print("Accuracy on train set: %.4f" % (bagmodel.score(X_train, y_train)))
print("Accuracy on test set: %.4f" % (bagmodel.score(X_test, y_test)))
```

The big difference between the accuracies on the train and test set indicate over-fitting. We will look at optimal hyperparameter values to improve performance such as *min_samples_split*, which is the minimum number of samples required to split an internal node. We've mentioned *max_depth* in Lesson 3 of Module 3 for decision trees, which is the same for random forests. With limited computational power and quick turnaround times, we will explore just a few samples for these parameters. We use `GridSearchCV` as in Lesson 2 of Module 5 to find the optimal hyperparameters.

```
In [ ]: from sklearn.model_selection import GridSearchCV
```

```
In [ ]: # defining parameter range
param_grid = {
    "n_estimators": [10],
    "max_depth": [2, 3, 4],
    "min_samples_split": [2, 4, 8],
}

grid = GridSearchCV(
    RandomForestClassifier(random_state=8), param_grid, refit=True, verbose=3, cv=5
)

# fitting the model for grid search
grid.fit(X_train, y_train)
```

```
In [ ]: # print best parameter after tuning
print(grid.best_params_)

# print how our model looks after hyper-parameter tuning
print(grid.best_estimator_)
```

Apply optimal parameters and check if test score improves.

```
In [ ]: # Train with Tuned RF
# Create a tuned RF Classifier
bagmodel_tuned = RandomForestClassifier(
    n_estimators=10,
    max_depth=grid.best_params_["max_depth"],
    min_samples_split=grid.best_params_["min_samples_split"],
    random_state=8,
)

bagmodel_tuned.fit(X_train, y_train)
print("Accuracy on test set: %.4f" % (bagmodel_tuned.score(X_test, y_test)))
```

There is an uplift in accuracy for the test set by just exploring a small hyperparameter space. The small hyperparameter space is considered to keep computational time within reasonable limits. Students are encouraged to explore more hyperparameter options. Another measure of model performance to explore is the out-of-bag error, which is essentially the test error of a bagged model (James 342). Keep in mind that accuracy is based on the predicted classes, whereas the ROC AUC is based on predicted scores. We therefore plot the ROC Curve as in the previous modules to compare to the random

guess or no-skill model. From the output below, there is a clear benefit to the Random forest model over the no-skill model.

```
In [ ]: import matplotlib.pyplot as plt

# Performance
from sklearn.metrics import roc_auc_score, roc_curve

# predicted probabilities generated by tuned classifier
y_pred_proba = bagmodel_tuned.predict_proba(X_test)

# RF ROC dependencies
fpr, tpr, _ = roc_curve(y_test, y_pred_proba[:, 1])
auc = round(roc_auc_score(y_test, y_pred_proba[:, 1]), 4)

# RF Model
plt.plot(fpr, tpr, label="RF, auc=" + str(auc))

# Random guess model
plt.plot(fpr, fpr, "-", label="Random")
plt.title("ROC")
plt.ylabel("TPR")
plt.xlabel("FPR")

plt.legend(loc=4)
plt.show()
```

Another useful feature from bagging is the *Out of Bag Score*. Refer to the video below for an explanation on this feature.

```
In [ ]: from IPython.display import VimeoVideo

VimeoVideo("785136013", h="2e436e22bd", width=600)
```

[Access Video Transcript here](#)

3. Stacking Model

The stacking model depends on the base models and meta model. For this illustration, we use the Gaussian Naïve Bayes, decision tree, and SVM classifiers as the base models. For the meta model, we use the simplistic logistic regression model. This is to show how we can leverage the logistic regression model using stacking. Let's import the necessary libraries for each of them. The names are well described and should be easy to match to the respective model.

```
In [ ]: from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
```

We define three base classifiers without any hyperparameter specifications, i.e., we use the default settings other than the kernel for SVC. This is to show the predictive power of

stacking without optimizing.

```
In [ ]: clf1 = DecisionTreeClassifier(random_state=2) # Decision Tree

clf2 = SVC(kernel="rbf", random_state=2) # Support Vector Classifier

clf3 = GaussianNB() # Gaussian Naive Bayes

est_rs = [("DTree", clf1), ("SVM", clf2), ("NB", clf3)]
```

Define the meta model.

```
In [ ]: mylr = LogisticRegression(random_state=2)

In [ ]: # creating a stacking classifier
stackingCLF = StackingClassifier(
    estimators=est_rs, final_estimator=mylr, stack_method="auto", cv=3
)
```

The stacking model is ready to go, and now, we need to train it on the training data. We also show the training score since we want to compare it to each of its base learners below as well.

```
In [ ]: stackingCLF.fit(X_train, y_train)
acc_score = stackingCLF.score(X_train, y_train)
round(acc_score, 4)

In [ ]: # Let's run the individual base learners and stacking `clf` to compare
for iterclf, iterlabel in zip([clf1, clf2, clf3], ["DTree", "SVM", "NB"]):
    scores = model_selection.cross_val_score(
        iterclf, X_train, y_train, cv=3, scoring="accuracy"
    )
    print("accuracy: %0.3f %s " % (scores.mean(), iterlabel))
```

It is quite evident that the stacking model outperforms all of its base learners and shows the benefit of ensemble learning in this case. It is worth comparing the performance of the two ensemble learning models used in this lesson; therefore, we look at the ROC curve and AUC of the stacking model versus the random forest model.

```
In [ ]: # predicted probabilities generated by tuned classifier
y_pred_probaStack = stackingCLF.predict_proba(X_test)

# Stacking Model ROC dependencies
fpr, tpr, _ = roc_curve(y_test, y_pred_probaStack[:, 1])
auc = round(roc_auc_score(y_test, y_pred_probaStack[:, 1]), 4)

# RF ROC dependencies
fpr_RF, tpr_RF, _ = roc_curve(y_test, y_pred_proba[:, 1])
auc_RF = round(roc_auc_score(y_test, y_pred_proba[:, 1]), 4)

# RF Model
plt.plot(fpr_RF, tpr_RF, label="RF, auc=" + str(auc_RF))
# Stacking Model
plt.plot(fpr, tpr, label="StackM, auc=" + str(auc))
```

```
# Random guess model
plt.plot(fpr, fpr, "-", label="Random")
plt.title("ROC")
plt.ylabel("TPR")
plt.xlabel("FPR")

plt.legend(loc=4)
plt.show()
```

From the plot above, it is clear that the stacking model slightly outperforms the random forest model.

4. Conclusion

This lesson looked at bagging and stacking in practice to predict a change of the LUXXX index by a certain threshold, i.e., a classification problem. Both these methods show added value over a no-skill model. In the next lesson, we will cover another ensemble learning method namely boosting and adaptive boosting.

References

James, Gareth et al. *An Introduction to Statistical Learning: With Applications in R*. 2nd ed., Springer, 2021.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

MACHINE LEARNING IN FINANCE

MODULE 4 | LESSON 3

BOOSTING METHODS

Reading Time	25 minutes
Prior Knowledge	Classification, Decision Trees
Keywords	Sampling with replacement, Performance, Amount of Say

The previous lesson covered ensemble learning methods such as bagging and stacking and also applied these methods to a problem in finance. This lesson will cover another ensemble learning method called adaptive boosting in detail.

1. Introduction

Boosting was briefly discussed in Module 3, Lesson 3, and we will add to that in this lesson. Boosting, like a random forest, uses many weak learners to improve the end result. Remember that a weak learner is a learner that performs slightly better than a no-skill model. The difference with boosting and a random forest model is that the weak learners are **not** independent. These learners are built sequentially, i.e., a weak learner "improves" upon the learner before it. This improvement happens with the next learner focusing on the previous weak learner's errors. This is the fundamental concept of boosting. This lesson will introduce the reader to *adaptive* boosting.

2. Adaptive Boosting

Adaptive boosting or AdaBoost is a popular algorithm and was among the earliest boosting models introduced. The sequential development of this model occurs by adjusting the weights or importance of the previous learner's predictions. Higher weights are given to predictions that deviate from the actual value and lower weights to the predictions that are close to the actual value. In other words, more weighting is assigned to incorrect predictions, or errors are corrected iteratively. AdaBoost can work on both classification and regression problems. One should keep in mind that AdaBoost is not restricted to tree-based algorithms; however, the weak learners or base learners are homogeneous in that they must all be of the same family or class of models. As with random forests, the majority vote is the final decision. However, with AdaBoost, not all base learners have equal weight or "say" on the vote. We will use the Taiwan Credit Card default dataset to explain how AdaBoost works.

```
In [ ]: # import any libraries needed to create dataset
import random
import warnings

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn import tree

# from sklearn.metrics import confusion_matrix
from sklearn.tree import DecisionTreeClassifier # , `DecisionTreeRegressor`

warnings.filterwarnings("ignore")
```

The dataset is used to identify default payments for credit cards in Taiwan. The Target class is binary such that 1 refers to a client that defaults on their credit repayments and 0 otherwise. There are 22 predictors in this dataset. More information on this dataset can be found at <https://archive.ics.uci.edu/ml/datasets/default+of+credit+card+clients> . We import the dataset below.

```
In [ ]: data4lesson = pd.read_excel("Copy of default of credit card clients.xlsx")
```

```
In [ ]: # Make ID column the index
data4lesson.set_index("ID", inplace=True, drop=True)
data4lesson.head()
```

```
In [ ]: # Rename the target column to 'Class'
data4lesson.rename(columns={"default payment next month": "Class"}, inplace=True)

# keep predictors in a separate list
feats = list(data4lesson.columns[0 : data4lesson.shape[1] - 1])
```

Check the dimension of the data and the frequency of each target class.

```
In [ ]: print("Rows {} columns {}".format(data4lesson.shape[0], data4lesson.shape[1]))
data4lesson["Class"].value_counts()
```

Let's begin the steps for AdaBoost.

2.1 Step 1: Sample Weights

The first step for AdaBoost is to perform random sampling with replacement on our dataset. To do this, we initially assign each observation an equal weight, i.e., $1/N$ for N being the number of observations. To explain the process of random sampling, we will insert a cumulative column of the weights called `RangeSelect` to illustrate the idea.

```
In [ ]: # Assign equal weights to all samples
N = len(data4lesson)
data4lesson["probR1"] = 1 / (1.0 * N)
data4lesson["RangeSelect"] = data4lesson["probR1"].cumsum()
# Show last 5 columns
data4lesson[["PAY_AMT5", "PAY_AMT6", "Class", "probR1", "RangeSelect"]].head()
```

Now that we have the cumulative column, we are able to explain the method of sampling.

2.2 Step 2: Sampling with Replacement

Think of each value in the `RangeSelect` column as an upper limit with the lower limit being the value in the observation, or row, before it. For example, the output above ID 2 would have a `RangeSelect` of $[\text{\$}(0.000033, 0.000067)\text{\$}]$. Now draw a random number in the range $[\text{\$}(0, 1)\text{\$}]$. Suppose this number is $\text{\$}(0.000043)\text{\$}$. We therefore add ID 2 to our sample since the random number of $\text{\$}(0.000043)\text{\$}$ is in the range $[\text{\$}(0.000033, 0.000067)\text{\$}]$, which corresponds to ID 2. If our random number drawn was $\text{\$}(0.00014)\text{\$}$, then ID 5 would be added to the sample since $\text{\$}(0.00014)\text{\$}$ is in the range $[\text{\$}(0.000133, 0.000167)\text{\$}]$. One may notice that observations with large ranges would have a higher chance of being selected. Hence, large weights or `probR1` values increase the chances of being sampled, which is crucial for the AdaBoost algorithm. Fortunately, Python has a more elegant and efficient way of performing random sampling with replacement, so we won't need the `RangeSelect` column anymore.

```
In [ ]: # Drop `RangeSelect` column
data4lesson.drop("RangeSelect", axis=1, inplace=True)
```

We'll use the pandas `sample` function to sample with replacement based on the weights column `probR1`. The amount of samples is the same as N, that is, the same number of observations as our original dataset. There will be duplicate observations in this new dataset since the sampling is done with replacement.

```
In [ ]: # Random selection based on weights
# specify seed in case we wish to replicate
# the Random Sampling with replacement
random.seed(10)

data4lesson1 = data4lesson.sample(
    len(data4lesson), replace=True, weights=data4lesson["probR1"]
)
```

2.3 Step 3: Fit a Base Learner

The next step in AdaBoost is to fit a weak classifier or base learner to the samples. For this example, we will use decision trees as our base learners. We'll call this learner `weak_l1` and fit it to the samples.

```
In [ ]: # Since it is a stump the max depth is 1
weak_l1 = DecisionTreeClassifier(random_state=10, max_depth=1)

X = data4lesson1.loc[:, feats]
y = data4lesson1["Class"]

weak_l1.fit(X, y)
```

We should be able to view the flow diagram of the stump.

```
In [ ]: # visualize the stump
# features we used

fn = feats

# labels of the target class

cn = ["0", "1"]

fig = plt.figure(figsize=(25, 20))
_ = tree.plot_tree(weak_l1, feature_names=fn, class_names=cn, filled=True)

# save the figure to file

# fig.savefig('imagename.png')
fig.savefig("decision_tree.png")
```

Figure 1: The Decision Stump of the Base Learner at the First Iteration

Notice the base learner found *Pay_0* to be the strongest predictor to split on. Let us compare the predictions to the actual values.

```
In [ ]: data4lesson1["pred1"] = weak_l1.predict(X)
```

```
In [ ]: data4lesson1[["Class", "pred1", "probR1"]].head()
```

2.4 Step 4: Calculate Performance

The snapshot of just the first five rows shows one misclassification for ID 16973. The number of misclassifications will determine the *amount of say* or *performance* of this classifier. The formula for the amount of say is

$$\text{Amount of say} = \frac{1}{2} \ln \left(\frac{1 - \text{Total Error}}{\text{Total Error}} \right) \quad (1.1)$$

where the **Total Error** is simply the sum of weights of misclassified observations. The Total Error for the base classifier at our first iteration, i.e., *weak_l1* is

```
In [ ]: # Total error of weak_l1
E_1 = round(
    np.sum(
        np.where(
            data4lesson1["Class"] != data4lesson1["pred1"], data4lesson1["probR1"]
        )
    ),
    5,
)

E_1
```

The Amount of say in (1.1) against the total error in general is shown below.

```
In [ ]: n_pts = 100
TE = 0.01 + np.arange(n_pts) * (1 - 0.01) / n_pts
AoS = 0.5 * np.log((1 - TE) / TE)
plt.plot(TE, AoS)
plt.ylabel("Amount of Say")
plt.xlabel("Total Error")
```

Figure 2: Performance or Amount of Say against Total Error for a Classifier

From figure 2, we see that for close to perfect classifications, the base classifier will have a performance that tends to infinity. For almost all incorrect classifications, the learner will have performance that tends to negative infinity. For a Total Error of 0.5, this results in a performance of zero. If all samples had equal weights, this would be equivalent to a no-skill classifier. In practice, a small error term is added to the Total Error to avoid the case when the Total Error is zero. Our first base learner has a Total Error of 0.1796, which results in a performance of 0.7596. Remember that AdaBoost focuses on the misclassifications by adjusting the weights in an iterative manner. Let us now look into this process of adjusting the weights.

For the observations that were misclassified, the formula to adjust the weights is given by,

$$\text{Adjusted weight} = \text{Current Weight} \times e^{\text{performance}} \tag{1.2}$$

similarly the formula for adjusting the weights of the correct classification is,

$$\text{Adjusted weight} = \text{Current Weight} \times e^{-\text{performance}} \tag{1.3}$$

Notice the difference between (1.2) and (1.3) is the sign of the exponent. This implies that correct classifications are downweighted whereas misclassifications are adjusted to larger weights. Applying (1.2) and (1.3) to the weights creates the new weights shown below.

```
In [ ]: # performance of weak_l1
p1 = 0.5 * np.log((1 - E_1) / E_1)
# Call the new weights `probR2` since it is the second iteration
data4lesson1["probR2"] = np.where(
    data4lesson1["Class"] != data4lesson1["pred1"],
    data4lesson1["probR1"] * np.exp(p1), # misclassifications (1.2)
    data4lesson1["probR1"] * np.exp(-p1),
)
```

```
In [ ]: data4lesson1[["Class", "pred1", "probR1", "probR2"]].head()
```

We can see the correct classifications have new weights that are lower than the original weights of 3.3×10^{-5} whereas the misclassifications have weights that have increased. Remember that an increase in weight would increase the chances of choosing that observation when sampling with replacement. One thing to take note of is that the new weights are normalized such that they sum to 1 as were the original weights in column `probR1`.

```
In [ ]: # normalize the weights
data4lesson1["probR2"] = data4lesson1["probR2"] / data4lesson1["probR2"].sum()
```

This brings us back to Step 1 with a new set of weights to repeat steps 2 to 4. The number of times we would repeat these cycles would depend on the number of weak classifiers we specify. This is the iterative process that AdaBoost follows to develop the final model. Ideally, we would want to continue this process until all observations are classified correctly, but this can allow for overfitting. Hyperparameter tuning should be taken into consideration. To illustrate the final prediction of the model, let's assume we had chosen 5 base learners and they each had the following performance or Amount of Say and prediction.

```
In [ ]: dat = [[0.7596, 1], [0.7821, 1], [0.79019, 1], [0.8241, 0], [0.8410, 0]]
model_summary = pd.DataFrame(data=dat, columns=["performance", "prediction"])
model_summary.index.name = "iteration"
model_summary
```

The hypothetical result above over 5 iterations shows 3 base learners predicting class 1 and 2 base learners predicting class 0. The final vote is based on the sum of performance for each class voted, i.e., instead of the counts being used, the sum of the performance is used. The calculation below shows that class 1 would be favored since the sum of the performance for base learners in favor of that class is 2.332, which is greater than the sum of the performance for base learners in favor of class 0.

```
In [ ]: class_0 = np.sum(
    np.where(model_summary["prediction"] == 0, model_summary["performance"], 0)
)
class_1 = np.sum(
    np.where(model_summary["prediction"] == 1, model_summary["performance"], 0)
)

print("Weight of Votes ")
print("Class 0: {} Class 1: {}".format(class_0, class_1))
```

Something to keep in mind is that the example above showed the updating procedure on the entire dataset whereas in practice we would be using the training set to update and develop AdaBoost. We've also shown the process using weak decision stumps, but there is evidence to support more advanced learners being effective (Chuan et al. 3).

Boosting has found success in the data science field especially in Kaggle competitions. There is one more boosting technique we will cover in the next lesson called gradient boosting.

3. Conclusion

This lesson focused on adaptive boosting and showed the iterative updating procedure to derive the final model and prediction. The procedure was shown for a classification problem, namely predictive credit card payment defaults. In the next lesson, we'll look at

another boosting algorithm called gradient boosting and compare all ensemble learning methods covered in this module to a common problem.

References

1. Chuan, Yijian, et al. "The Success of AdaBoost and Its Application in Portfolio Management." *arXiv*, 2021, pp. 1–6.
2. University of California, Irvine Machine Learning Repository. "Default of Credit Card Clients." <https://archive.ics.uci.edu/ml/datasets/default+of+credit+card+clients>

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

MACHINE LEARNING IN FINANCE

MODULE 4 | LESSON 4

ENSEMBLE LEARNING COMPARISONS

Reading Time	25 minutes
Prior Knowledge	Boosting methodology, Adaptive Boosting, Ensemble learning, Derivatives
Keywords	Pseudo residuals, loss function, learning rate, gradient boosting

The previous lesson introduced the reader to the details of boosting, specifically adaptive boosting. This lesson will explore Gradient Boosting and compare all ensemble learning models covered in this module to a common predictive problem.

1. Gradient Boosting

In the previous lesson we've looked at AdaBoost as it was among the first designed boosting algorithm with a particular loss function that can be sensitive to outliers. Gradient Boosting, however, is a generic algorithm that allows the optimization of an arbitrary loss function, thus making Gradient Boosting more flexible and robust to outliers than AdaBoost. The requirement of the loss function is that it is differentiable. We've seen that AdaBoost places more emphasis on predictions that were incorrect from the previous base learner although there is still some weight assigned to the correct predictions. With Gradient Boosting, however, there is only an emphasis on building the subsequent base learners based on the misclassifications from the previous learner. To illustrate this, we will go through an example to explain the process. Unlike the previous section, we will look at a regression example, i.e., a continuous target variable to illustrate boosting applied to a different type of problem. The dataset we use is the house price dataset from the UCI Machine Learning Repository. We will begin by importing the necessary libraries for this lesson along with the data.

```
In [1]: # import all necessary libraries

import warnings

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.ensemble import (
    AdaBoostClassifier,
    GradientBoostingClassifier,
    RandomForestClassifier,
```

```

        StackingClassifier,
    )
    from sklearn.linear_model import LogisticRegression
    from sklearn.metrics import mean_squared_error
    from sklearn.model_selection import train_test_split
    from sklearn.naive_bayes import GaussianNB

    # for stacking model later
    from sklearn.svm import SVC
    from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor

    warnings.filterwarnings("ignore")

```

```

In [2]: data4gradboost = pd.read_excel("Real estate valuation data set.xlsx")

data4gradboost.set_index("No", drop=True, inplace=True)
data4gradboost.drop("X1 transaction date", axis=1, inplace=True)
data4gradboost.head()

```

```

Out[2]:

```

	X2 house age	X3 distance to the nearest MRT station	X4 number of convenience stores	X5 latitude	X6 longitude	Y house price of unit area
No						
1	32.0	84.87882	10	24.98298	121.54024	37.9
2	19.5	306.59470	9	24.98034	121.53951	42.2
3	13.3	561.98450	5	24.98746	121.54391	47.3
4	13.3	561.98450	5	24.98746	121.54391	54.8
5	5.0	390.56840	5	24.97937	121.54245	43.1

Above shows a snapshot of the dataset, which has 5 predictors and a target variable in the last column that represents the price per unit area. Refer to <https://archive.ics.uci.edu/ml/datasets/Real+estate+valuation+data+set> for more details on this dataset. We start by separating the dataset into the features and target column for a train/test split. The size of the test set is not important for this example since we are only interested in understanding the gradient boosting algorithm methodology.

```

In [3]: # Separate into X and Y
X = data4gradboost.iloc[:, :-1]
y = data4gradboost.iloc[:, -1]

```

```

In [4]: X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=2)

```

1.1 Step 1: Fit Weak Learner

Now that we have the data to develop our gradient boosting model, we can begin the first step. Similarly, for the AdaBoost in the previous lesson, we use weak decision tree classifiers as the base learners. We choose a max_depth of 2 and then fit the base learner to the training dataset.

```
In [5]: tree_1 = DecisionTreeRegressor(max_depth=2, random_state=2)
tree_1.fit(X_train, y_train)
```

```
Out[5]: ▾ DecisionTreeRegressor
DecisionTreeRegressor(max_depth=2, random_state=2)
```

1.2 Step 2: Calculate Residuals

Once we fit our first weak learner to the data, we then calculate the residuals. The residuals are simply given by

$$\text{residuals} = y_{\{i\}} - \hat{y}_{\{i\}}$$

where $y_{\{i\}}$ is the actual outcome and $\hat{y}_{\{i\}}$ is the predicted value for observation i . The residuals now become the new target data to train the next base learner. Gradient boost builds trees based on the residuals or errors of the previous tree.

1.3 Step 3: Train Next Base Learner on Residuals

```
In [6]: # predictions
y_pred1 = tree_1.predict(X_train)
# residuals become the next target data to train
y2_train = y_train - y_pred1
```

Once we have the new target values to train the next base learner, we continue this cycle, i.e., steps 1 to 3. In this example, we will do this for 3 base learners, i.e., 3 iterations.

```
In [7]: # initialize new tree or 2nd base Learner
tree_2 = DecisionTreeRegressor(max_depth=2, random_state=2)
tree_2.fit(X_train, y2_train)

# predictions
y_pred2 = tree_2.predict(X_train)
# new target values
y3_train = y2_train - y_pred2

# initialize new tree or 3rd base Learner
tree_3 = DecisionTreeRegressor(max_depth=2, random_state=2)
tree_3.fit(X_train, y3_train)

# last set of predictions. Stop at 3rd base Learner
y_pred3 = tree_3.predict(X_train)
```

1.4 Final Step: Calculate Final Predictions

Finally, for predictions of unseen observations, we would add the predictions of each base learner as below. We assign the final predictions to `y_pred` and then calculate a performance metric, namely Root Mean Squared Error (RMSE). We obtain an RMSE of 9.9625.

```
In [8]: # Make predictions on test set for all 3 base Learners
y1_pred = tree_1.predict(X_test)
```

```

y2_pred = tree_2.predict(X_test)
y3_pred = tree_3.predict(X_test)

y_pred = y1_pred + y2_pred + y3_pred

# check MSE
round(mean_squared_error(y_test, y_pred) ** 0.5, 4)

```

Out[8]: 9.9625

Building the gradient boosting algorithm from scratch is quite tedious, and it is easy to make an error coding it. Fortunately, Python has the built-in `GradientBoostingRegressor` from sklearn to make things much easier. Below is the more elegant and efficient way to code this. Notice the hyperparameter `learning_rate` = 1, which we'll address later.

```
In [9]: from sklearn.ensemble import GradientBoostingRegressor
```

```
In [10]: gbr = GradientBoostingRegressor(
          max_depth=2, n_estimators=3, random_state=10, learning_rate=1
        )
```

```
In [11]: gbr.fit(X_train, y_train)
```

```
Out[11]: ▼ GradientBoostingRegressor
          GradientBoostingRegressor(learning_rate=1, max_depth=2, n_estimators=3,
                                     random_state=10)
```

The RMSE results are identical to our model built from scratch.

```
In [12]: gbr_pred = gbr.predict(X_test)
          round(mean_squared_error(y_test, gbr_pred) ** 0.5, 4)
```

Out[12]: 9.9625

The example above illustrates the procedure followed by a gradient boosting algorithm. Below will show how it works with a bit more attention to the mathematics behind it.

1.5 Gradient Boosting Reasoning

The algorithm begins with a naive guess of using the average, say $\langle y \rangle$, for all house prices or target values. The reasoning behind this is gradient boosting aims to minimize the loss function $L(y_i, \hat{y}_i)$. A common L used is of the form

$$L(y_i, \hat{y}_i) = \frac{1}{2} \sum_{i=1}^N \left(y_i - \hat{y}_i \right)^2 \tag{1.1}$$

over N observations. This type of loss function is an L2 Loss function.

At the start, we assume $\hat{y}_i$ is a constant function $F_0(x)$ over predictors x . From calculus, minimizing implies finding the derivative of (1.1) with respect to $F_0(x)$,

equating to zero and solving for $F_0(x)$. If we do this, the solution $F_0^*(x)$ is,
$$F_0^*(x) = \frac{1}{N} \sum_{i=1}^N y_i = \bar{y} \tag{1.2}$$

which is how we get the average as the first guess. Note, had we chosen a different L this may not have been the case. For illustrative purposes, we will use the loss function in (1.1) to explain the concept.

Remember from our example in section 1.1 that the next base learner would aim to predict the residuals of the previous base learner instead of the actual values. Starting off with predictions as the average of the actuals is a very simplistic estimation; however, the next iteration 1 will improve on the previous base learner predictions such that new predictions are
$$F_1(x) = F_0(x) + h_1(x) \tag{1.3}$$

where $h_1(x)$ is an added estimator such that together with the previous estimate F_0 , will improve the estimates. We hope that our new estimates would predict the actuals i.e.,
$$F_1(x) = F_0(x) + h_1(x) = y \tag{1.4}$$

but rearranging (1.4) gives us $h_1(x) = y - F_0(x)$. This is just the residuals from the previous base learner's predictions and is what we were doing in section 1.1. So to get this estimator $h_1(x)$ we would provide as a target column $(y - F_0(x))$ and fit a base learner on the dataset $\{(x_i, y_i - F_0(x))\}_{i=1}^N$ or $\{(x_i, r_{i1})\}_{i=1}^N$ where r_{i1} are the pseudo residuals at iteration 1. In general at iteration j , we have from (1.4),

$$F_j(x) = F_{j-1}(x) + h_j(x) \tag{1.5}$$

and we develop our estimator $h_j(x)$ in the same way. There is a parameter ν , that is not shown in (1.5) where,

$$F_j(x) = F_{j-1}(x) + \nu h_j(x) \tag{1.6}$$

which we've set as $\nu = 1$ but can actually take on values in the range $[0,1]$. This parameter is the *learning rate*, which improves accuracy in the long run. Think of it as a step size, and often many learners with small step sizes provide better results. From recursive substitution over all iterations, the last iteration T will provide a final model such that for a test or unseen data point x^* the prediction would be

$$F_T(x^*) = F_0 + \sum_{j=1}^{T-1} \nu h_j(x^*), \tag{1.6}$$

which ties in to the last step for our gradient boosting model built from scratch.

Classification is very similar to the regression process but differs mainly in the scoring. The reader is referred to Murphy (607) for further reading on classification. There is also a modified gradient boosting algorithm, which improves on what we've covered above called Extreme Gradient Boosting or XGBoost. One of the modifications is that XGBoost uses a 2nd order approximation of the loss function and adds a regularizer on the tree complexity. Refer to Murphy (615) for details on this.

2. Ensemble Learning Comparison

This section will apply Bagging, Stacking, AdaBoost, and Gradient boosting to a common classification problem. We predict whether the Luxembourg index (LUXXX) will exceed a return of 0.25% in any direction. The predictors are a combination of country indices and technical indicators. We start by importing the data.

```
In [13]: import warnings

warnings.filterwarnings("ignore")

# Loc = "ENTER YOUR FULL PATH TO LOCATION OF DATA FILE HERE"
# data_df = pd.read_csv(Loc+"/MScFE 650 MLF GWP Data.csv")

data_df = pd.read_csv("MScFE 650 MLF GWP Data.csv")
# Convert string to datetime
data_df["Date"] = pd.to_datetime(data_df["Date"])
```

Create the target variable.

```
In [14]: # Set Target Index for predicting
target ETF = "LUXXX"

# Use returns instead of prices for other Indices
# Other Indices used as Index_features
ETF_features = data_df.loc[:, ~data_df.columns.isin(["Date", target ETF])]
data_df[ETF_features] = data_df[ETF_features].pct_change()

data_df[target ETF + "_returns"] = data_df[target ETF].pct_change()

# Create Target Column.
# Shift period for target column
data_df[target ETF + "_returns" + "_Shift"] = data_df[target ETF + "_returns"].s

# Strategy to take Long position for anticipated returns of 0.5%
data_df["Target"] = np.where(
    (data_df[target ETF + "_returns_Shift"].abs() > 0.025), 1, 0
)
```

Technical indicators included as predictors are slow to fast moving average ratio (SMA_ratio), Relative Strength Index (RSI), and Rate of Change (RC).

```
In [15]: # Four country indices used.
feats = ["MSCI KOREA", "MSCI DENMARK", "MSCI FRANCE", "MSCI NORWAY"]

# creating the technical indicators
data_df["SMA_5"] = data_df[target ETF].rolling(5).mean()
data_df["SMA_15"] = data_df[target ETF].rolling(15).mean()
data_df["SMA_ratio"] = data_df["SMA_15"] / data_df["SMA_5"]

# Can drop SMA columns since not needed anymore.
data_df.drop(["SMA_5", "SMA_15"], axis=1, inplace=True)

# shift the price of the target by 1 unit previous in time
```

```

data_df["Diff"] = data_df[target ETF] - data_df[target ETF].shift(1)
data_df["Up"] = data_df["Diff"]
data_df.loc[(data_df["Up"] < 0), "Up"] = 0

data_df["Down"] = data_df["Diff"]
data_df.loc[(data_df["Down"] > 0), "Down"] = 0
data_df["Down"] = abs(data_df["Down"])

data_df["avg_5up"] = data_df["Up"].rolling(5).mean()
data_df["avg_5down"] = data_df["Down"].rolling(5).mean()

data_df["avg_15up"] = data_df["Up"].rolling(15).mean()
data_df["avg_15down"] = data_df["Down"].rolling(15).mean()

data_df["RS_5"] = data_df["avg_5up"] / data_df["avg_5down"]
data_df["RS_15"] = data_df["avg_15up"] / data_df["avg_15down"]

data_df["RSI_5"] = 100 - (100 / (1 + data_df["RS_5"]))
data_df["RSI_15"] = 100 - (100 / (1 + data_df["RS_15"]))

data_df["RSI_ratio"] = data_df["RSI_5"] / data_df["RSI_15"]

# Can drop RS Calc columns
data_df.drop(
    ["Diff", "Up", "Down", "avg_5up", "avg_5down", "avg_15up", "avg_15down"],
    axis=1,
    inplace=True,
)

data_df["RC"] = data_df[target ETF].pct_change(15)

# all_feats
feats.append("SMA_ratio")
feats.append("RSI_ratio")
feats.append("RC")

```

Now we can apply all our models to the data. Perform the train/test split of 80/20.

```

In [16]: # Train/Test split
# Train/Test split. No NaNs in the data.
NoNaN_df = data_df.dropna()
X = NoNaN_df[feats]

X = X.iloc[:, :] # .values
y = NoNaN_df.loc[:, "Target"] # .values

del NoNaN_df

# from sklearn.cross_validation import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_

```

We will tune the boosting models and use the hyperparameters from Lesson 2 for the bagging model. We should note that AdaBoost by default has Decision Tree Classifiers as base learners with `max_depth = 1` as default. For the gradient boosting classifier, we assume the default `max_depth` of 3. We only tune on learning rate and `n_estimators` to reduce computational time. We also do the same for the XGBoost model.

```
In [17]: from sklearn.model_selection import GridSearchCV
```

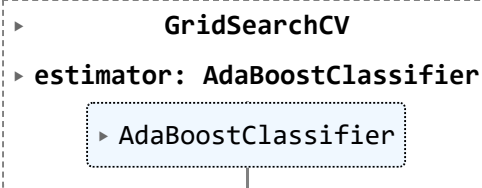
```
In [18]: # parameters for AdaBoost
param_grid = {"n_estimators": [10, 20, 50, 100], "learning_rate": [0.1, 0.25, 0.

gridAdBoost = GridSearchCV(
    AdaBoostClassifier(), param_grid, refit=True, verbose=3, cv=3
)
# fitting the model for grid search
gridAdBoost.fit(X_train, y_train)
```

Fitting 3 folds for each of 16 candidates, totalling 48 fits

```
[CV 1/3] END learning_rate=0.1, n_estimators=10;; score=0.698 total time= 0.0s
[CV 2/3] END learning_rate=0.1, n_estimators=10;; score=0.683 total time= 0.0s
[CV 3/3] END learning_rate=0.1, n_estimators=10;; score=0.710 total time= 0.0s
[CV 1/3] END learning_rate=0.1, n_estimators=20;; score=0.698 total time= 0.0s
[CV 2/3] END learning_rate=0.1, n_estimators=20;; score=0.603 total time= 0.0s
[CV 3/3] END learning_rate=0.1, n_estimators=20;; score=0.694 total time= 0.0s
[CV 1/3] END learning_rate=0.1, n_estimators=50;; score=0.698 total time= 0.0s
[CV 2/3] END learning_rate=0.1, n_estimators=50;; score=0.571 total time= 0.0s
[CV 3/3] END learning_rate=0.1, n_estimators=50;; score=0.677 total time= 0.0s
[CV 1/3] END learning_rate=0.1, n_estimators=100;; score=0.667 total time= 0.1s
[CV 2/3] END learning_rate=0.1, n_estimators=100;; score=0.571 total time= 0.1s
[CV 3/3] END learning_rate=0.1, n_estimators=100;; score=0.661 total time= 0.1s
[CV 1/3] END learning_rate=0.25, n_estimators=10;; score=0.698 total time= 0.0s
[CV 2/3] END learning_rate=0.25, n_estimators=10;; score=0.587 total time= 0.0s
[CV 3/3] END learning_rate=0.25, n_estimators=10;; score=0.694 total time= 0.0s
[CV 1/3] END learning_rate=0.25, n_estimators=20;; score=0.683 total time= 0.0s
[CV 2/3] END learning_rate=0.25, n_estimators=20;; score=0.571 total time= 0.0s
[CV 3/3] END learning_rate=0.25, n_estimators=20;; score=0.677 total time= 0.0s
[CV 1/3] END learning_rate=0.25, n_estimators=50;; score=0.635 total time= 0.0s
[CV 2/3] END learning_rate=0.25, n_estimators=50;; score=0.603 total time= 0.1s
[CV 3/3] END learning_rate=0.25, n_estimators=50;; score=0.645 total time= 0.1s
[CV 1/3] END learning_rate=0.25, n_estimators=100;; score=0.651 total time= 0.1
s
[CV 2/3] END learning_rate=0.25, n_estimators=100;; score=0.540 total time= 0.1
s
[CV 3/3] END learning_rate=0.25, n_estimators=100;; score=0.677 total time= 0.1
s
[CV 1/3] END learning_rate=0.5, n_estimators=10;; score=0.683 total time= 0.0s
[CV 2/3] END learning_rate=0.5, n_estimators=10;; score=0.635 total time= 0.0s
[CV 3/3] END learning_rate=0.5, n_estimators=10;; score=0.629 total time= 0.0s
[CV 1/3] END learning_rate=0.5, n_estimators=20;; score=0.619 total time= 0.0s
[CV 2/3] END learning_rate=0.5, n_estimators=20;; score=0.603 total time= 0.0s
[CV 3/3] END learning_rate=0.5, n_estimators=20;; score=0.645 total time= 0.0s
[CV 1/3] END learning_rate=0.5, n_estimators=50;; score=0.587 total time= 0.0s
[CV 2/3] END learning_rate=0.5, n_estimators=50;; score=0.571 total time= 0.0s
[CV 3/3] END learning_rate=0.5, n_estimators=50;; score=0.629 total time= 0.1s
[CV 1/3] END learning_rate=0.5, n_estimators=100;; score=0.619 total time= 0.1s
[CV 2/3] END learning_rate=0.5, n_estimators=100;; score=0.603 total time= 0.1s
[CV 3/3] END learning_rate=0.5, n_estimators=100;; score=0.613 total time= 0.1s
[CV 1/3] END learning_rate=1.0, n_estimators=10;; score=0.683 total time= 0.0s
[CV 2/3] END learning_rate=1.0, n_estimators=10;; score=0.603 total time= 0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=10;; score=0.597 total time= 0.0s
[CV 1/3] END learning_rate=1.0, n_estimators=20;; score=0.619 total time= 0.0s
[CV 2/3] END learning_rate=1.0, n_estimators=20;; score=0.651 total time= 0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=20;; score=0.629 total time= 0.0s
[CV 1/3] END learning_rate=1.0, n_estimators=50;; score=0.587 total time= 0.1s
[CV 2/3] END learning_rate=1.0, n_estimators=50;; score=0.540 total time= 0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=50;; score=0.629 total time= 0.1s
[CV 1/3] END learning_rate=1.0, n_estimators=100;; score=0.635 total time= 0.1s
[CV 2/3] END learning_rate=1.0, n_estimators=100;; score=0.587 total time= 0.1s
[CV 3/3] END learning_rate=1.0, n_estimators=100;; score=0.661 total time= 0.1s
```

Out[18]:



```

In [19]: # print best parameter after tuning
print(gridAdBoost.best_params_)

# print how our AdaBoost model looks after hyper-parameter tuning
print(gridAdBoost.best_estimator_)

{'learning_rate': 0.1, 'n_estimators': 10}
AdaBoostClassifier(learning_rate=0.1, n_estimators=10)

In [20]: # parameters for Gradient Boosting Classifier
param_grid = {"n_estimators": [10, 20, 50, 100], "learning_rate": [0.1, 0.25, 0.

GBgrid = GridSearchCV(
    GradientBoostingClassifier(), param_grid, refit=True, verbose=3, cv=3
)

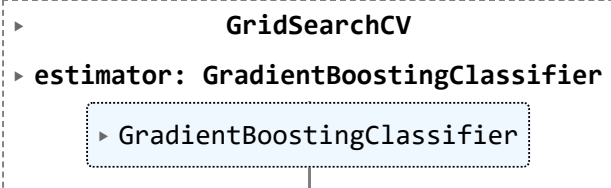
# fitting the model for grid search
GBgrid.fit(X_train, y_train)

```

Fitting 3 folds for each of 16 candidates, totalling 48 fits

```
[CV 1/3] END learning_rate=0.1, n_estimators=10;; score=0.683 total time= 0.0s
[CV 2/3] END learning_rate=0.1, n_estimators=10;; score=0.619 total time= 0.0s
[CV 3/3] END learning_rate=0.1, n_estimators=10;; score=0.645 total time= 0.0s
[CV 1/3] END learning_rate=0.1, n_estimators=20;; score=0.667 total time= 0.0s
[CV 2/3] END learning_rate=0.1, n_estimators=20;; score=0.619 total time= 0.0s
[CV 3/3] END learning_rate=0.1, n_estimators=20;; score=0.661 total time= 0.0s
[CV 1/3] END learning_rate=0.1, n_estimators=50;; score=0.667 total time= 0.0s
[CV 2/3] END learning_rate=0.1, n_estimators=50;; score=0.619 total time= 0.0s
[CV 3/3] END learning_rate=0.1, n_estimators=50;; score=0.677 total time= 0.0s
[CV 1/3] END learning_rate=0.1, n_estimators=100;; score=0.619 total time= 0.1s
[CV 2/3] END learning_rate=0.1, n_estimators=100;; score=0.635 total time= 0.1s
[CV 3/3] END learning_rate=0.1, n_estimators=100;; score=0.694 total time= 0.1s
[CV 1/3] END learning_rate=0.25, n_estimators=10;; score=0.635 total time= 0.0s
[CV 2/3] END learning_rate=0.25, n_estimators=10;; score=0.619 total time= 0.0s
[CV 3/3] END learning_rate=0.25, n_estimators=10;; score=0.645 total time= 0.0s
[CV 1/3] END learning_rate=0.25, n_estimators=20;; score=0.603 total time= 0.0s
[CV 2/3] END learning_rate=0.25, n_estimators=20;; score=0.635 total time= 0.0s
[CV 3/3] END learning_rate=0.25, n_estimators=20;; score=0.661 total time= 0.0s
[CV 1/3] END learning_rate=0.25, n_estimators=50;; score=0.556 total time= 0.0s
[CV 2/3] END learning_rate=0.25, n_estimators=50;; score=0.651 total time= 0.0s
[CV 3/3] END learning_rate=0.25, n_estimators=50;; score=0.694 total time= 0.0s
[CV 1/3] END learning_rate=0.25, n_estimators=100;; score=0.603 total time= 0.1
s
[CV 2/3] END learning_rate=0.25, n_estimators=100;; score=0.587 total time= 0.1
s
[CV 3/3] END learning_rate=0.25, n_estimators=100;; score=0.645 total time= 0.1
s
[CV 1/3] END learning_rate=0.5, n_estimators=10;; score=0.587 total time= 0.0s
[CV 2/3] END learning_rate=0.5, n_estimators=10;; score=0.587 total time= 0.0s
[CV 3/3] END learning_rate=0.5, n_estimators=10;; score=0.645 total time= 0.0s
[CV 1/3] END learning_rate=0.5, n_estimators=20;; score=0.603 total time= 0.0s
[CV 2/3] END learning_rate=0.5, n_estimators=20;; score=0.619 total time= 0.0s
[CV 3/3] END learning_rate=0.5, n_estimators=20;; score=0.629 total time= 0.0s
[CV 1/3] END learning_rate=0.5, n_estimators=50;; score=0.571 total time= 0.0s
[CV 2/3] END learning_rate=0.5, n_estimators=50;; score=0.587 total time= 0.0s
[CV 3/3] END learning_rate=0.5, n_estimators=50;; score=0.645 total time= 0.0s
[CV 1/3] END learning_rate=0.5, n_estimators=100;; score=0.603 total time= 0.1s
[CV 2/3] END learning_rate=0.5, n_estimators=100;; score=0.603 total time= 0.1s
[CV 3/3] END learning_rate=0.5, n_estimators=100;; score=0.645 total time= 0.1s
[CV 1/3] END learning_rate=1.0, n_estimators=10;; score=0.556 total time= 0.0s
[CV 2/3] END learning_rate=1.0, n_estimators=10;; score=0.556 total time= 0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=10;; score=0.613 total time= 0.0s
[CV 1/3] END learning_rate=1.0, n_estimators=20;; score=0.571 total time= 0.0s
[CV 2/3] END learning_rate=1.0, n_estimators=20;; score=0.476 total time= 0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=20;; score=0.645 total time= 0.0s
[CV 1/3] END learning_rate=1.0, n_estimators=50;; score=0.587 total time= 0.0s
[CV 2/3] END learning_rate=1.0, n_estimators=50;; score=0.571 total time= 0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=50;; score=0.645 total time= 0.0s
[CV 1/3] END learning_rate=1.0, n_estimators=100;; score=0.571 total time= 0.1s
[CV 2/3] END learning_rate=1.0, n_estimators=100;; score=0.603 total time= 0.1s
[CV 3/3] END learning_rate=1.0, n_estimators=100;; score=0.710 total time= 0.1s
```

Out[20]:



```
In [21]: # print best parameter after tuning
print(GBgrid.best_params_)

# print how our Gradient Boost model looks after hyper-parameter tuning
print(GBgrid.best_estimator_)

{'learning_rate': 0.1, 'n_estimators': 50}
GradientBoostingClassifier(n_estimators=50)
```

```
In [22]: # import library for XGBoost
import xgboost as xgb
```

```
In [23]: # parameters for XG Boosting Classifier
param_grid = {"n_estimators": [10, 20, 50, 100], "learning_rate": [0.1, 0.25, 0.

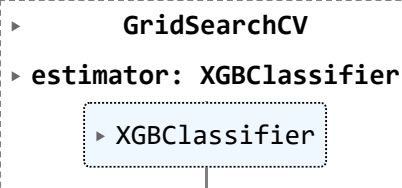
XGB_model = xgb.XGBClassifier()
XGBgrid = GridSearchCV(XGB_model, param_grid, refit=True, verbose=3, cv=3)

# fitting the model for grid search
XGBgrid.fit(X_train, y_train)
```

Fitting 3 folds for each of 16 candidates, totalling 48 fits

```
[CV 1/3] END learning_rate=0.1, n_estimators=10;; score=0.603 total time= 0.0s
[CV 2/3] END learning_rate=0.1, n_estimators=10;; score=0.683 total time= 0.0s
[CV 3/3] END learning_rate=0.1, n_estimators=10;; score=0.677 total time= 0.0s
[CV 1/3] END learning_rate=0.1, n_estimators=20;; score=0.635 total time= 0.0s
[CV 2/3] END learning_rate=0.1, n_estimators=20;; score=0.698 total time= 0.0s
[CV 3/3] END learning_rate=0.1, n_estimators=20;; score=0.661 total time= 0.0s
[CV 1/3] END learning_rate=0.1, n_estimators=50;; score=0.651 total time= 0.0s
[CV 2/3] END learning_rate=0.1, n_estimators=50;; score=0.683 total time= 0.0s
[CV 3/3] END learning_rate=0.1, n_estimators=50;; score=0.661 total time= 0.0s
[CV 1/3] END learning_rate=0.1, n_estimators=100;; score=0.635 total time= 0.0s
[CV 2/3] END learning_rate=0.1, n_estimators=100;; score=0.619 total time= 0.0s
[CV 3/3] END learning_rate=0.1, n_estimators=100;; score=0.661 total time= 0.0s
[CV 1/3] END learning_rate=0.25, n_estimators=10;; score=0.635 total time= 0.0s
[CV 2/3] END learning_rate=0.25, n_estimators=10;; score=0.683 total time= 0.0s
[CV 3/3] END learning_rate=0.25, n_estimators=10;; score=0.677 total time= 0.0s
[CV 1/3] END learning_rate=0.25, n_estimators=20;; score=0.635 total time= 0.0s
[CV 2/3] END learning_rate=0.25, n_estimators=20;; score=0.667 total time= 0.0s
[CV 3/3] END learning_rate=0.25, n_estimators=20;; score=0.661 total time= 0.0s
[CV 1/3] END learning_rate=0.25, n_estimators=50;; score=0.635 total time= 0.0s
[CV 2/3] END learning_rate=0.25, n_estimators=50;; score=0.635 total time= 0.0s
[CV 3/3] END learning_rate=0.25, n_estimators=50;; score=0.629 total time= 0.0s
[CV 1/3] END learning_rate=0.25, n_estimators=100;; score=0.651 total time= 0.0
s
[CV 2/3] END learning_rate=0.25, n_estimators=100;; score=0.635 total time= 0.0
s
[CV 3/3] END learning_rate=0.25, n_estimators=100;; score=0.629 total time= 0.0
s
[CV 1/3] END learning_rate=0.5, n_estimators=10;; score=0.651 total time= 0.0s
[CV 2/3] END learning_rate=0.5, n_estimators=10;; score=0.619 total time= 0.0s
[CV 3/3] END learning_rate=0.5, n_estimators=10;; score=0.661 total time= 0.0s
[CV 1/3] END learning_rate=0.5, n_estimators=20;; score=0.667 total time= 0.0s
[CV 2/3] END learning_rate=0.5, n_estimators=20;; score=0.651 total time= 0.0s
[CV 3/3] END learning_rate=0.5, n_estimators=20;; score=0.661 total time= 0.0s
[CV 1/3] END learning_rate=0.5, n_estimators=50;; score=0.667 total time= 0.0s
[CV 2/3] END learning_rate=0.5, n_estimators=50;; score=0.571 total time= 0.0s
[CV 3/3] END learning_rate=0.5, n_estimators=50;; score=0.645 total time= 0.0s
[CV 1/3] END learning_rate=0.5, n_estimators=100;; score=0.651 total time= 0.0s
[CV 2/3] END learning_rate=0.5, n_estimators=100;; score=0.571 total time= 0.0s
[CV 3/3] END learning_rate=0.5, n_estimators=100;; score=0.613 total time= 0.0s
[CV 1/3] END learning_rate=1.0, n_estimators=10;; score=0.603 total time= 0.0s
[CV 2/3] END learning_rate=1.0, n_estimators=10;; score=0.635 total time= 0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=10;; score=0.597 total time= 0.0s
[CV 1/3] END learning_rate=1.0, n_estimators=20;; score=0.667 total time= 0.0s
[CV 2/3] END learning_rate=1.0, n_estimators=20;; score=0.619 total time= 0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=20;; score=0.629 total time= 0.0s
[CV 1/3] END learning_rate=1.0, n_estimators=50;; score=0.619 total time= 0.0s
[CV 2/3] END learning_rate=1.0, n_estimators=50;; score=0.587 total time= 0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=50;; score=0.613 total time= 0.0s
[CV 1/3] END learning_rate=1.0, n_estimators=100;; score=0.619 total time= 0.0s
[CV 2/3] END learning_rate=1.0, n_estimators=100;; score=0.556 total time= 0.0s
[CV 3/3] END learning_rate=1.0, n_estimators=100;; score=0.645 total time= 0.0s
```

Out[23]:



```
In [24]: # print best parameter after tuning
print(XGBgrid.best_params_)

# print how our XGBoost model looks after hyper-parameter tuning
print(XGBgrid.best_estimator_)

{'learning_rate': 0.25, 'n_estimators': 10}
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric=None, feature_types=None,
              feature_weights=None, gamma=None, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=0.25, max_bin=None, max_cat_threshold=None,
              max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
              max_leaves=None, min_child_weight=None, missing=nan,
              monotone_constraints=None, multi_strategy=None, n_estimators=10,
              n_jobs=None, num_parallel_tree=None, ...)
```

So we find that AdaBoost and Gradient Boosting have the same optimal hyperparameters; however, XGBoost has a different learning rate. We can train all of our ensemble models and then compare performance afterwards.

```
In [25]: # Train with Tuned Random Forest
# Create a tuned RF Classifier
bagmodel_tuned = RandomForestClassifier(
    max_depth=2, min_samples_split=8, n_estimators=10, random_state=10
)

bagmodel_tuned.fit(X_train, y_train)
```

```
Out[25]: ▼ RandomForestClassifier

RandomForestClassifier(max_depth=2, min_samples_split=8, n_estimators=10,
                       random_state=10)
```

This is the stacking model.

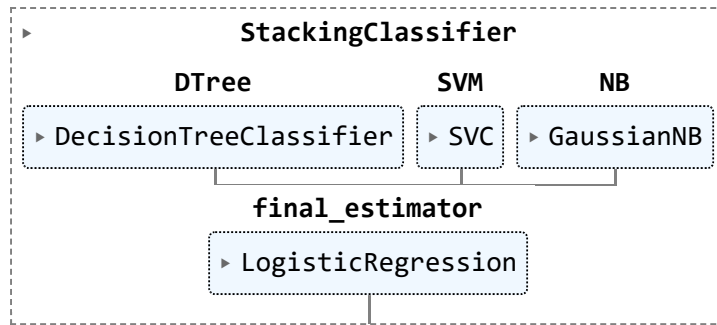
```
In [26]: clf1 = DecisionTreeClassifier() # Decision Tree

clf2 = SVC(kernel="rbf") # Support Vector Classifier

clf3 = GaussianNB() # Gaussian Naive Bayes

est_rs = [("DTTree", clf1), ("SVM", clf2), ("NB", clf3)]
# Meta model
mylr = LogisticRegression()
# creating a stacking classifier
stackingCLF = StackingClassifier(
    estimators=est_rs, final_estimator=mylr, stack_method="auto", cv=3
)
stackingCLF.fit(X_train, y_train)
```

Out[26]:



Fit all tuned boosting models to the training set.

```

In [27]: # Create a tuned AdaBoost Classifier
AdaBoost_tuned = AdaBoostClassifier(learning_rate=0.1, n_estimators=10)

# Create a tuned Gradient Boosting Classifier
GB_tuned = GradientBoostingClassifier(learning_rate=0.1, n_estimators=10)

# Create a tuned XGBoost Classifier
XGB_tuned = xgb.XGBClassifier(learning_rate=0.25, n_estimators=10)

# train boosting models
AdaBoost_tuned.fit(X_train, y_train)
GB_tuned.fit(X_train, y_train)
XGB_tuned.fit(X_train, y_train)
print("Training complete")

```

Training complete

Lastly, we compare all models with an ROC curve.

```

In [28]: from sklearn.metrics import roc_auc_score, roc_curve

```

```

In [29]: # predicted probabilities generated by models
y_pred_probaStack = stackingCLF.predict_proba(X_test) # stacking
y_pred_probaRF = bagmodel_tuned.predict_proba(X_test) # RF
y_pred_probaAdB = AdaBoost_tuned.predict_proba(X_test) # AdaBoost
y_pred_probaGb = GB_tuned.predict_proba(X_test) # `GradBoost`
y_pred_probaXGB = XGB_tuned.predict_proba(X_test) # XGBoost

# Stacking ROC dependencies
fpr, tpr, _ = roc_curve(y_test, y_pred_probaStack[:, 1])
auc = round(roc_auc_score(y_test, y_pred_probaStack[:, 1]), 4)

# RF ROC dependencies
fpr_RF, tpr_RF, _ = roc_curve(y_test, y_pred_probaRF[:, 1])
auc_RF = round(roc_auc_score(y_test, y_pred_probaRF[:, 1]), 4)

# AdaBoost ROC dependencies
fpr_AB, tpr_AB, _ = roc_curve(y_test, y_pred_probaAdB[:, 1])
auc_AB = round(roc_auc_score(y_test, y_pred_probaAdB[:, 1]), 4)

# Gradient Boost ROC dependencies
fpr_GB, tpr_GB, _ = roc_curve(y_test, y_pred_probaGb[:, 1])
auc_GB = round(roc_auc_score(y_test, y_pred_probaGb[:, 1]), 4)

# XGB ROC dependencies
fpr_XGB, tpr_XGB, _ = roc_curve(y_test, y_pred_probaXGB[:, 1])

```

```

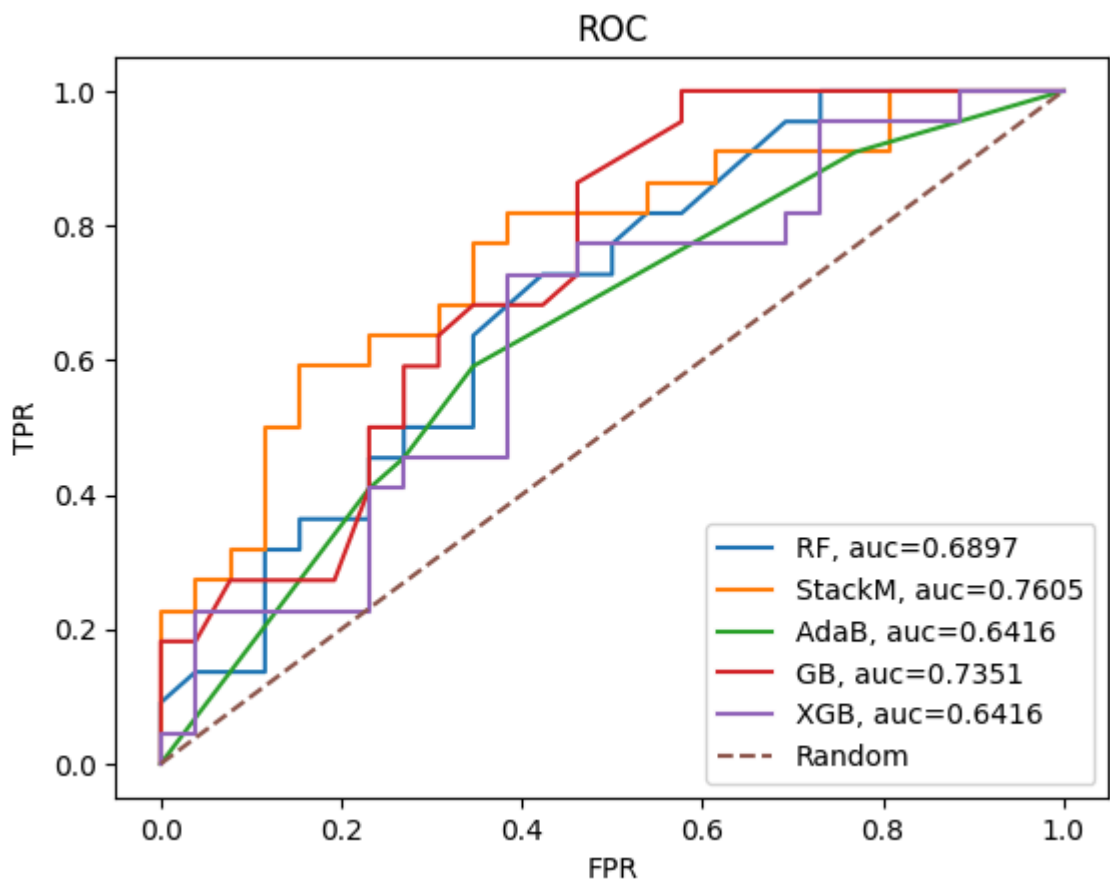
auc_XGB = round(roc_auc_score(y_test, y_pred_probaXGB[:, 1]), 4)

# RF Model
plt.plot(fpr_RF, tpr_RF, label="RF, auc=" + str(auc_RF))
# Stacking Model
plt.plot(fpr, tpr, label="StackM, auc=" + str(auc))
# AdaBoost Model
plt.plot(fpr_AB, tpr_AB, label="AdaB, auc=" + str(auc_AB))
# `GradBoost` Model
plt.plot(fpr_GB, tpr_GB, label="GB, auc=" + str(auc_GB))
# XGBoost Model
plt.plot(fpr_XGB, tpr_XGB, label="XGB, auc=" + str(auc_XGB))

# Random guess model
plt.plot(fpr, fpr, "--", label="Random")
plt.title("ROC")
plt.ylabel("TPR")
plt.xlabel("FPR")

plt.legend(loc=4)
plt.show()

```



Great! All models performed reasonably well with some models exceeding 70% AUC. Keep in mind that we did not explore all hyperparameters for models, but this is a good enough starting point to create your own models and explore the hyperparameter space and effect on model performance.

3. Conclusion

This lesson explored the last ensemble learning algorithm, namely gradient boosting in more detail after briefly mentioning it in Module 3. We compared all ensemble learning algorithms covered in this module in a common classification problem and obtained results significantly better than a no-skill model, thus showing the added value in using the combined predictive power of weak learners.

References

1. Murphy, Kevin P. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
2. University of California, Irvine Machine Learning Repository. "Real Estate Valuation Data Set." <https://archive.ics.uci.edu/ml/datasets/Real+estate+valuation+data+set>

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Popular Ensemble Methods: An Empirical Study

David Opitz

*Department of Computer Science
University of Montana
Missoula, MT 59812 USA*

OPITZ@CS.UMT.EDU

Richard Maclin

*Computer Science Department
University of Minnesota
Duluth, MN 55812 USA*

RMACLIN@D.UMN.EDU

Abstract

An ensemble consists of a set of individually trained classifiers (such as neural networks or decision trees) whose predictions are combined when classifying novel instances. Previous research has shown that an ensemble is often more accurate than any of the single classifiers in the ensemble. Bagging (Breiman, 1996c) and Boosting (Freund & Schapire, 1996; Schapire, 1990) are two relatively new but popular methods for producing ensembles. In this paper we evaluate these methods on 23 data sets using both neural networks and decision trees as our classification algorithm. Our results clearly indicate a number of conclusions. First, while Bagging is almost always more accurate than a single classifier, it is sometimes much less accurate than Boosting. On the other hand, Boosting can create ensembles that are less accurate than a single classifier – especially when using neural networks. Analysis indicates that the performance of the Boosting methods is dependent on the characteristics of the data set being examined. In fact, further results show that Boosting ensembles may overfit noisy data sets, thus decreasing its performance. Finally, consistent with previous studies, our work suggests that most of the gain in an ensemble's performance comes in the first few classifiers combined; however, relatively large gains can be seen up to 25 classifiers when Boosting decision trees.

1. Introduction

Many researchers have investigated the technique of combining the predictions of multiple classifiers to produce a single classifier (Breiman, 1996c; Clemen, 1989; Perrone, 1993; Wolpert, 1992). The resulting classifier (hereafter referred to as an *ensemble*) is generally more accurate than any of the individual classifiers making up the ensemble. Both theoretical (Hansen & Salamon, 1990; Krogh & Vedelsby, 1995) and empirical (Hashem, 1997; Opitz & Shavlik, 1996a, 1996b) research has demonstrated that a good ensemble is one where the individual classifiers in the ensemble are both accurate and make their errors on different parts of the input space. Two popular methods for creating accurate ensembles are Bagging (Breiman, 1996c) and Boosting (Freund & Schapire, 1996; Schapire, 1990). These methods rely on “resampling” techniques to obtain different training sets for each of the classifiers. In this paper we present a comprehensive evaluation of both Bagging and Boosting on 23 data sets using two basic classification methods: decision trees and neural networks.

Previous work has demonstrated that Bagging and Boosting are very effective for decision trees (Bauer & Kohavi, 1999; Drucker & Cortes, 1996; Breiman, 1996c, 1996b; Freund & Schapire, 1996; Quinlan, 1996); however, there has been little empirical testing with neural networks (especially with the new Boosting algorithm). Discussions with previous researchers reveal that many authors concentrated on decision trees due to their fast training speed and well-established default parameter settings. Neural networks present difficulties for testing both in terms of the significant processing time required and in selecting training parameters; however, we feel there are distinct advantages to including neural networks in our study. First, previous empirical studies have demonstrated that individual neural networks produce highly accurate classifiers that are sometimes more accurate than corresponding decision trees (Fisher & McKusick, 1989; Mooney, Shavlik, Towell, & Gove, 1989). Second, neural networks have been extensively applied across numerous domains (Arbib, 1995). Finally, by studying neural networks in addition to decision trees we can examine how Bagging and Boosting are influenced by the learning algorithm, giving further insight into the general characteristics of these approaches. Bauer and Kohavi (1999) also study Bagging and Boosting applied to two learning methods, in their case decision trees using a variant of C4.5 and naive-Bayes classifiers, but their study mainly concentrated on the decision tree results.

Our neural network and decision tree results led us to a number of interesting conclusions. The first is that a Bagging ensemble generally produces a classifier that is more accurate than a standard classifier. Thus one should feel comfortable always Bagging their decision trees or neural networks. For Boosting, however, we note more widely varying results. For a few data sets Boosting produced dramatic reductions in error (even compared to Bagging), but for other data sets it actually increases in error over a single classifier (particularly with neural networks). In further tests we examined the effects of noise and support Freund and Schapire's (1996) conjecture that Boosting's sensitivity to noise may be partly responsible for its occasional increase in error.

An alternate baseline approach we investigated was the creation of a simple neural network ensemble where each network used the full training set and differed only in its random initial weight settings. Our results indicate that this ensemble technique is surprisingly effective, often producing results as good as Bagging. Research by Ali and Pazzani (1996) demonstrated similar results using randomized decision tree algorithms.

Our results also show that the ensemble methods are generally consistent (in terms of their effect on accuracy) when applied either to neural networks or to decision trees; however, there is little inter-correlation between neural networks and decision trees except for the Boosting methods. This suggests that some of the increases produced by Boosting are dependent on the particular characteristics of the data set rather than on the component classifier. In further tests we demonstrate that Bagging is more resilient to noise than Boosting.

Finally, we investigated the question of how many component classifiers should be used in an ensemble. Consistent with previous research (Freund & Schapire, 1996; Quinlan, 1996), our results show that most of the reduction in error for ensemble methods occurs with the first few additional classifiers. With Boosting decision trees, however, relatively large gains may be seen up until about 25 classifiers.

This paper is organized as follows. In the next section we present an overview of classifier ensembles and discuss Bagging and Boosting in detail. Next we present an extensive empirical analysis of Bagging and Boosting. Following that we present future research and additional related work before concluding.

2. Classifier Ensembles

Figure 1 illustrates the basic framework for a classifier ensemble. In this example, neural networks are the basic classification method, though conceptually any classification method (e.g., decision trees) can be substituted in place of the networks. Each network in Figure 1's ensemble (network 1 through network N in this case) is trained using the training instances for that network. Then, for each example, the predicted output of each of these networks (o_i in Figure 1) is combined to produce the output of the ensemble ($\hat{o}$ in Figure 1). Many researchers (Alpaydin, 1993; Breiman, 1996c; Krogh & Vedelsby, 1995; Lincoln & Skrzypek, 1989) have demonstrated that an effective combining scheme is to simply average the predictions of the network.

Combining the output of several classifiers is useful only if there is disagreement among them. Obviously, combining several identical classifiers produces no gain. Hansen and Salamon (1990) proved that if the average error rate for an example is less than 50% and the component classifiers in the ensemble are independent in the production of their errors, the expected error for that example can be reduced to zero as the number of classifiers combined goes to infinity; however, such assumptions rarely hold in practice. Krogh and Vedelsby (1995) later proved that the ensemble error can be divided into a term measuring the average generalization error of each individual classifier and a term measuring the disagreement among the classifiers. What they formally showed was that an ideal ensemble consists of highly correct classifiers that disagree as much as possible. Opitz and Shavlik (1996a, 1996b) empirically verified that such ensembles generalize well.

As a result, methods for creating ensembles center around producing classifiers that disagree on their predictions. Generally, these methods focus on altering the training process in

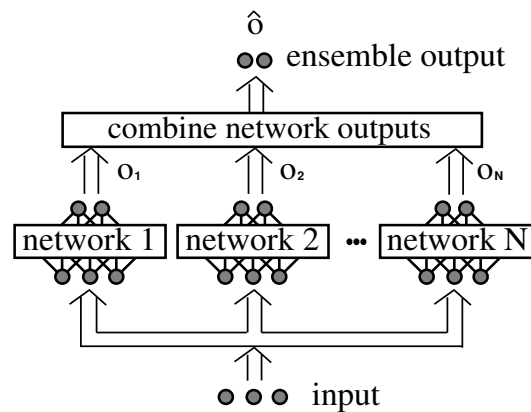


Figure 1: A classifier ensemble of neural networks.

the hope that the resulting classifiers will produce different predictions. For example, neural network techniques that have been employed include methods for training with different topologies, different initial weights, different parameters, and training only on a portion of the training set (Alpaydin, 1993; Drucker, Cortes, Jackel, LeCun, & Vapnik, 1994; Hansen & Salamon, 1990; Maclin & Shavlik, 1995). In this paper we concentrate on two popular methods (Bagging and Boosting) that try to generate disagreement among the classifiers by altering the training set each classifier sees.

2.1 Bagging Classifiers

Bagging (Breiman, 1996c) is a “bootstrap” (Efron & Tibshirani, 1993) ensemble method that creates individuals for its ensemble by training each classifier on a random redistribution of the training set. Each classifier’s training set is generated by randomly drawing, with replacement, N examples – where N is the size of the original training set; many of the original examples may be repeated in the resulting training set while others may be left out. Each individual classifier in the ensemble is generated with a different random sampling of the training set.

Figure 2 gives a sample of how Bagging might work on a imaginary set of data. Since Bagging resamples the training set with replacement, some instance are represented multiple times while others are left out. So Bagging’s training-set-1 might contain examples 3 and 7 twice, but does not contain either example 4 or 5. As a result, the classifier trained on training-set-1 might obtain a higher test-set error than the classifier using all of the data. In fact, all four of Bagging’s component classifiers could result in higher test-set error; however, when combined, these four classifiers can (and often do) produce test-set error lower than that of the single classifier (the diversity among these classifiers generally compensates for the increase in error rate of any individual classifier).

Breiman (1996c) showed that Bagging is effective on “unstable” learning algorithms where small changes in the training set result in large changes in predictions. Breiman (1996c) claimed that neural networks and decision trees are examples of unstable learning algorithms. We study the effectiveness of Bagging on both these learning methods in this article.

2.2 Boosting Classifiers

Boosting (Freund & Schapire, 1996; Schapire, 1990) encompasses a family of methods. The focus of these methods is to produce a *series* of classifiers. The training set used for each member of the series is chosen based on the performance of the earlier classifier(s) in the series. In Boosting, examples that are incorrectly predicted by previous classifiers in the series are chosen more often than examples that were correctly predicted. Thus Boosting attempts to produce new classifiers that are better able to predict examples for which the current ensemble’s performance is poor. (Note that in Bagging, the resampling of the training set is not dependent on the performance of the earlier classifiers.)

In this work we examine two new and powerful forms of Boosting: Arcing (Breiman, 1996b) and Ada-Boosting (Freund & Schapire, 1996). Like Bagging, Arcing chooses a training set of size N for classifier $K + 1$ by probabilistically selecting (with replacement) examples from the original N training examples. Unlike Bagging, however, the probability

A sample of a single classifier on an imaginary set of data.	
(Original) Training Set	
Training-set-1:	1, 2, 3, 4, 5, 6, 7, 8

A sample of Bagging on the same data.	
(Resampled) Training Set	
Training-set-1:	2, 7, 8, 3, 7, 6, 3, 1
Training-set-2:	7, 8, 5, 6, 4, 2, 7, 1
Training-set-3:	3, 6, 2, 7, 5, 6, 2, 2
Training-set-4:	4, 5, 1, 4, 6, 4, 3, 8

A sample of Boosting on the same data.	
(Resampled) Training Set	
Training-set-1:	2, 7, 8, 3, 7, 6, 3, 1
Training-set-2:	1, 4, 5, 4, 1, 5, 6, 4
Training-set-3:	7, 1, 5, 8, 1, 8, 1, 4
Training-set-4:	1, 1, 6, 1, 1, 3, 1, 5

Figure 2: Hypothetical runs of Bagging and Boosting. Assume there are eight training examples. Assume example 1 is an “outlier” and is hard for the component learning algorithm to classify correctly. With Bagging, each training set is an independent sample of the data; thus, some examples are missing and others occur multiple times. The Boosting training sets are also samples of the original data set, but the “hard” example (example 1) occurs more in later training sets since Boosting concentrates on correctly predicting it.

of selecting an example is not equal across the training set. This probability depends on how often that example was misclassified by the previous K classifiers. Ada-Boosting can use the approach of (a) selecting a set of examples based on the probabilities of the examples, or (b) simply using all of the examples and weight the error of each example by the probability for that example (i.e., examples with higher probabilities have more effect on the error). This latter approach has the clear advantage that each example is incorporated (at least in part) in the training set. Furthermore, Friedman et al. (1998) have demonstrated that this form of Ada-Boosting can be viewed as a form of additive modeling for optimizing a logistic loss function. In this work, however, we have chosen to use the approach of subsampling the data to ensure a fair empirical comparison (in part due to the restarting reason discussed below).

Both Arcing and Ada-Boosting initially set the probability of picking each example to be $1/N$. These methods then recalculate these probabilities after each trained classifier is added to the ensemble. For Ada-Boosting, let ϵ_k be the sum of the probabilities of the

misclassified instances for the currently trained classifier C_k . The probabilities for the next trial are generated by multiplying the probabilities of C_k 's incorrectly classified instances by the factor $\beta_k = (1 - \epsilon_k)/\epsilon_k$ and then renormalizing all probabilities so that their sum equals 1. Ada-Boosting combines the classifiers $C_1, \dots, C_k$ using weighted voting where C_k has weight $\log(\beta_k)$. These weights allow Ada-Boosting to discount the predictions of classifiers that are not very accurate on the overall problem. Friedman et al (1998) have also suggested an alternative mechanism that fits together the predictions of the classifiers as an additive model using a maximum likelihood criterion.

In this work, we use the revision described by Breiman (1996b) where we reset all the weights to be equal and restart if either ϵ_k is not less than 0.5 or ϵ_k becomes 0.¹ By resetting the weights we do not disadvantage the Ada-Boosting learner in those cases where it reaches these values of ϵ_k ; the Ada-Boosting learner always incorporates the same number of classifiers as other methods we tested. To make this feasible, we are forced to use the approach of selecting a data set probabilistically rather than weighting the examples, otherwise a deterministic method such as C4.5 would cycle and generate duplicate members of the ensemble. That is, resetting the weights to $1/N$ would cause the learner to repeat the decision tree learned as the first member of the ensemble, and this would lead to reweighting the data set the same as for the second member of the ensemble, and so on. Randomly selecting examples for the data set based on the example probabilities alleviates this problem.

Arcing-x4 (Breiman, 1996b) (which we will refer to simply as Arcing) started out as a simple mechanism for evaluating the effect of Boosting methods where the resulting classifiers were combined without weighting the votes. Arcing uses a simple mechanism for determining the probabilities of including examples in the training set. For the i th example in the training set, the value m_i refers to the number of times that example was misclassified by the previous K classifiers. The probability p_i for selecting example i to be part of classifier $K + 1$'s training set is defined as

$$p_i = \frac{1 + m_i^4}{\sum_{j=1}^N (1 + m_j^4)} \quad (1)$$

Breiman chose the value of the power (4) empirically after trying several different values (Breiman, 1996b). Although this mechanism does not have the weighted voting of Ada-Boosting it still produces accurate ensembles and is simple to implement; thus we include this method (along with Ada-Boosting) in our empirical evaluation.

Figure 2 shows a hypothetical run of Boosting. Note that the first training set would be the same as Bagging; however, later training sets accentuate examples that were misclassified by the earlier member of the ensembles. In this figure, example 1 is a “hard” example that previous classifiers tend to misclassify. With the second training set, example 1 occurs multiple times, as do examples 4 and 5 since they were left out of the first training set and, in this case, misclassified by the first learner. For the final training set, example 1

1. For those few cases where ϵ_k becomes 0 (less than 0.12% of our results) we simply use a large positive value, $\log(\beta_k) = 3.0$, to weight these networks. For the more likely cases where ϵ_k is larger than 0.5 (approximately 5% of our results) we chose to weight the predictions by a very small positive value (0.001) rather than using a negative or 0 weight factor (this produced slightly better results than the alternate approaches in pilot studies).

becomes the predominant example chosen (whereas no single example is accentuated with Bagging); thus, the overall test-set error for this classifier might become very high. Despite this, however, Boosting will probably obtain a lower error rate when it combines the output of these four classifiers since it focuses on correctly predicting previously misclassified examples and weights the predictions of the different classifiers based on their accuracy for the training set. But Boosting can also overfit in the presence of noise (as we empirically show in Section 3).

2.3 The Bias plus Variance Decomposition

Recently, several authors (Breiman, 1996b; Friedman, 1996; Kohavi & Wolpert, 1996; Kong & Dietterich, 1995) have proposed theories for the effectiveness of Bagging and Boosting based on Geman et al.'s (1992) bias plus variance decomposition of classification error. In this decomposition we can view the expected error of a learning algorithm on a particular *target function* and *training set size* as having three components:

1. A *bias* term measuring how close the average classifier produced by the learning algorithm will be to the target function;
2. A *variance* term measuring how much each of the learning algorithm's guesses will vary with respect to each other (how often they disagree); and
3. A term measuring the minimum classification error associated with the Bayes optimal classifier for the target function (this term is sometimes referred to as the intrinsic target noise).

Using this framework it has been suggested (Breiman, 1996b) that both Bagging and Boosting reduce error by reducing the variance term. Freund and Schapire (1996) argue that Boosting also attempts to reduce the error in the bias term since it focuses on misclassified examples. Such a focus may cause the learner to produce an ensemble function that differs significantly from the single learning algorithm. In fact, Boosting may construct a function that is not even producible by its component learning algorithm (e.g., changing linear predictions into a classifier that contains non-linear predictions). It is this capability that makes Boosting an appropriate algorithm for combining the predictions of “weak” learning algorithms (i.e., algorithms that have a simple learning bias). In their recent paper, Bauer and Kohavi (1999) demonstrated that Boosting does indeed seem to reduce bias for certain real world problems. More surprisingly, they also showed that Bagging can also reduce the bias portion of the error, often for the same data sets for which Boosting reduces the bias.

Though the bias-variance decomposition is interesting, there are certain limitations to applying it to real-world data sets. To be able to estimate the bias, variance, and target noise for a particular problem, we need to know the actual function being learned. This is unavailable for most real-world problems. To deal with this problem Kohavi and Wolpert (1996) suggest holding out some of the data, the approach used by Bauer and Kohavi (1999) in their study. The main problem with this technique is that the training set size is greatly reduced in order to get good estimates of the bias and variance terms. We have chosen to strictly focus on generalization accuracy in our study, in part because Bauer and Kohavi's work has answered the question about whether Boosting and Bagging reduce the

bias for real world problems (they both do), and because their experiments demonstrate that while this decomposition gives some insight into ensemble methods, it is only a small part of the equation. For different data sets they observe cases where Boosting and Bagging both decrease mostly the variance portion of the error, and other cases where Boosting and Bagging both reduce the bias and variance of the error. Their tests also seem to indicate that Boosting’s generalization error increases on the domains where Boosting increases the variance portion of the error; but, it is difficult to determine what aspects of the data sets led to these results.

3. Results

This section describes our empirical study of Bagging, Ada-Boosting, and Arcing. Each of these three methods was tested with both decision trees and neural networks.

3.1 Data Sets

To evaluate the performance of Bagging and Boosting, we obtained a number of data sets from the University of Wisconsin Machine Learning repository as well as the UCI data set repository (Murphy & Aha, 1994). These data sets were hand selected such that they (a) came from real-world problems, (b) varied in characteristics, and (c) were deemed useful by previous researchers. Table 1 gives the characteristics of our data sets. The data sets chosen vary across a number of dimensions including: the type of the features in the data set (i.e., continuous, discrete, or a mix of the two); the number of output classes; and the number of examples in the data set. Table 1 also shows the architecture and training parameters used in our neural networks experiments.

3.2 Methodology

Results, unless otherwise noted, are averaged over five standard 10-fold cross validation experiments. For each 10-fold cross validation the data set is first partitioned into 10 equal-sized sets, then each set is in turn used as the test set while the classifier trains on the other nine sets. For each fold an ensemble of 25 classifiers is created. Cross validation folds were performed independently for each algorithm. We trained the neural networks using standard backpropagation learning (Rumelhart, Hinton, & Williams, 1986). Parameter settings for the neural networks include a learning rate of 0.15, a momentum term of 0.9, and weights are initialized randomly to be between -0.5 and 0.5. The number of hidden units and epochs used for training are given in the next section. We chose the number of hidden units based on the number of input and output units. This choice was based on the criteria of having at least one hidden unit per output, at least one hidden unit for every ten inputs, and five hidden units being a minimum. The number of epochs was based both on the number of examples and the number of parameters (i.e., topology) of the network. Specifically, we used 60 to 80 epochs for small problems involving fewer than 250 examples; 40 epochs for the mid-sized problems containing between 250 to 500 examples; and 20 to 40 epochs for larger problems. For the decision trees we used the C4.5 tool (Quinlan, 1993) and pruned trees (which empirically produce better performance) as suggested in Quinlan’s work.

Data Set	Cases	Class	Features		Neural Network			
			Cont	Disc	Inputs	Outputs	Hiddens	Epochs
breast-cancer-w	699	2	9	-	9	1	5	20
credit-a	690	2	6	9	47	1	10	35
credit-g	1000	2	7	13	63	1	10	30
diabetes	768	2	9	-	8	1	5	30
glass	214	6	9	-	9	6	10	80
heart-cleveland	303	2	8	5	13	1	5	40
hepatitis	155	2	6	13	32	1	10	60
house-votes-84	435	2	-	16	16	1	5	40
hypo	3772	5	7	22	55	5	15	40
ionosphere	351	2	34	-	34	1	10	40
iris	159	3	4	-	4	3	5	80
kr-vs-kp	3196	2	-	36	74	1	15	20
labor	57	2	8	8	29	1	10	80
letter	20000	26	16	-	16	26	40	30
promoters-936	936	2	-	57	228	1	20	30
ribosome-bind	1877	2	-	49	196	1	20	35
satellite	6435	6	36	-	36	6	15	30
segmentation	2310	7	19	-	19	7	15	20
sick	3772	2	7	22	55	1	10	40
sonar	208	2	60	-	60	1	10	60
soybean	683	19	-	35	134	19	25	40
splice	3190	3	-	60	240	2	25	30
vehicle	846	4	18	-	18	4	10	40

Table 1: Summary of the data sets used in this paper. Shown are the number of examples in the data set; the number of output classes; the number of continuous and discrete input features; the number of input, output, and hidden units used in the neural networks tested; and how many epochs each neural network was trained.

3.3 Data Set Error Rates

Table 2 shows test-set error rates for the data sets described in Table 1 for five neural network methods and four decision tree methods. (In Tables 4 and 5 we show these error rates as well as the standard deviation for each of these values.) Along with the test-set errors for Bagging, Arcing, and Ada-boosting, we include the test-set error rate for a single neural-network and a single decision-tree classifier. We also report results for a simple (baseline) neural-network ensemble approach – creating an ensemble of networks where each network varies only by randomly initializing the weights of the network. We include these results in certain comparisons to demonstrate their similarity to Bagging. One obvious conclusion drawn from the results is that each ensemble method appears to reduce the error rate for almost all of the data sets, and in many cases this reduction is large. In fact, the two-tailed sign test indicates that every ensemble method is significantly better than its single

Data Set	Neural Network					C4.5			
	Stan	Simp	Bag	Boosting		Stan	Bag	Boosting	
				Arc	Ada			Arc	Ada
breast-cancer-w	3.4	3.5	3.4	3.8	4.0	5.0	3.7	3.5	3.5
credit-a	14.8	13.7	13.8	15.8	15.7	14.9	13.4	14.0	13.7
credit-g	27.9	24.7	24.2	25.2	25.3	29.6	25.2	25.9	26.7
diabetes	23.9	23.0	22.8	24.4	23.3	27.8	24.4	26.0	25.7
glass	38.6	35.2	33.1	32.0	31.1	31.3	25.8	25.5	23.3
heart-cleveland	18.6	17.4	17.0	20.7	21.1	24.3	19.5	21.5	20.8
hepatitis	20.1	19.5	17.8	19.0	19.7	21.2	17.3	16.9	17.2
house-votes-84	4.9	4.8	4.1	5.1	5.3	3.6	3.6	5.0	4.8
hypo	6.4	6.2	6.2	6.2	6.2	0.5	0.4	0.4	0.4
ionosphere	9.7	7.5	9.2	7.6	8.3	8.1	6.4	6.0	6.1
iris	4.3	3.9	4.0	3.7	3.9	5.2	4.9	5.1	5.6
kr-vs-kp	2.3	0.8	0.8	0.4	0.3	0.6	0.6	0.3	0.4
labor	6.1	3.2	4.2	3.2	3.2	16.5	13.7	13.0	11.6
letter	18.0	12.8	10.5	5.7	4.6	14.0	7.0	4.1	3.9
promoters-936	5.3	4.8	4.0	4.5	4.6	12.8	10.6	6.8	6.4
ribosome-bind	9.3	8.5	8.4	8.1	8.2	11.2	10.2	9.3	9.6
satellite	13.0	10.9	10.6	9.9	10.0	13.8	9.9	8.6	8.4
segmentation	6.6	5.3	5.4	3.5	3.3	3.7	3.0	1.7	1.5
sick	5.9	5.7	5.7	4.7	4.5	1.3	1.2	1.1	1.0
sonar	16.6	15.9	16.8	12.9	13.0	29.7	25.3	21.5	21.7
soybean	9.2	6.7	6.9	6.7	6.3	8.0	7.9	7.2	6.7
splice	4.7	4.0	3.9	4.0	4.2	5.9	5.4	5.1	5.3
vehicle	24.9	21.2	20.7	19.1	19.7	29.4	27.1	22.5	22.9

Table 2: Test set error rates for the data sets using (1) a single neural network classifier; (2) an ensemble where each individual network is trained using the original training set and thus only differs from the other networks in the ensemble by its random initial weights; (3) an ensemble where the networks are trained using randomly resampled training sets (Bagging); an ensemble where the networks are trained using weighted resampled training sets (Boosting) where the resampling is based on the (4) Arcing method and (5) Ada method; (6) a single decision tree classifier; (7) a Bagging ensemble of decision trees; and (8) Arcing and (9) Ada Boosting ensembles of decision trees.

component classifier at the 95% confidence level; however, none of the ensemble methods are significantly better than any other ensemble approach at the 95% confidence level.

To better analyze Table 2's results, Figures 3 and 4 plot the percentage reduction in error for the Ada-Boosting, Arcing, and Bagging method as a function of the original error rate. Examining these figures we note that many of the gains produced by the ensemble methods are much larger than the standard deviation values. In terms of comparisons of different methods, it is apparent from both figures that the Boosting methods (Ada-

POPULAR ENSEMBLE METHODS

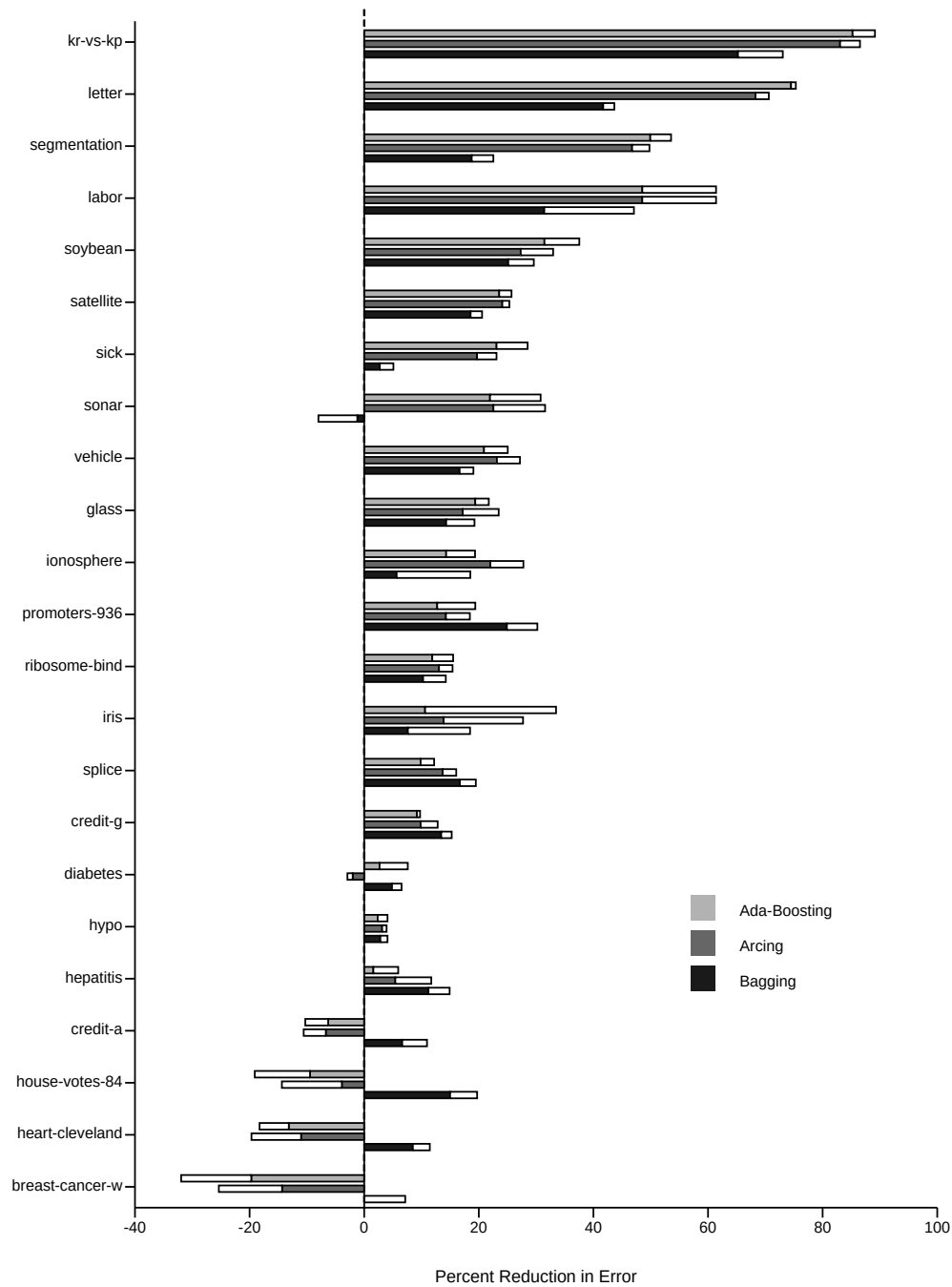


Figure 3: Reduction in error for Ada-Boosting, Arcing, and Bagging neural network ensembles as a percentage of the original error rate (i.e., a reduction from an error rate of 2.5% to 1.25% would be a 50% reduction in error rate, just as a reduction from 10.0% to 5.0% would also be a 50% reduction). Also shown (white portion of each bar) is one standard deviation for these results. The standard deviation is shown as an addition to the error reduction.

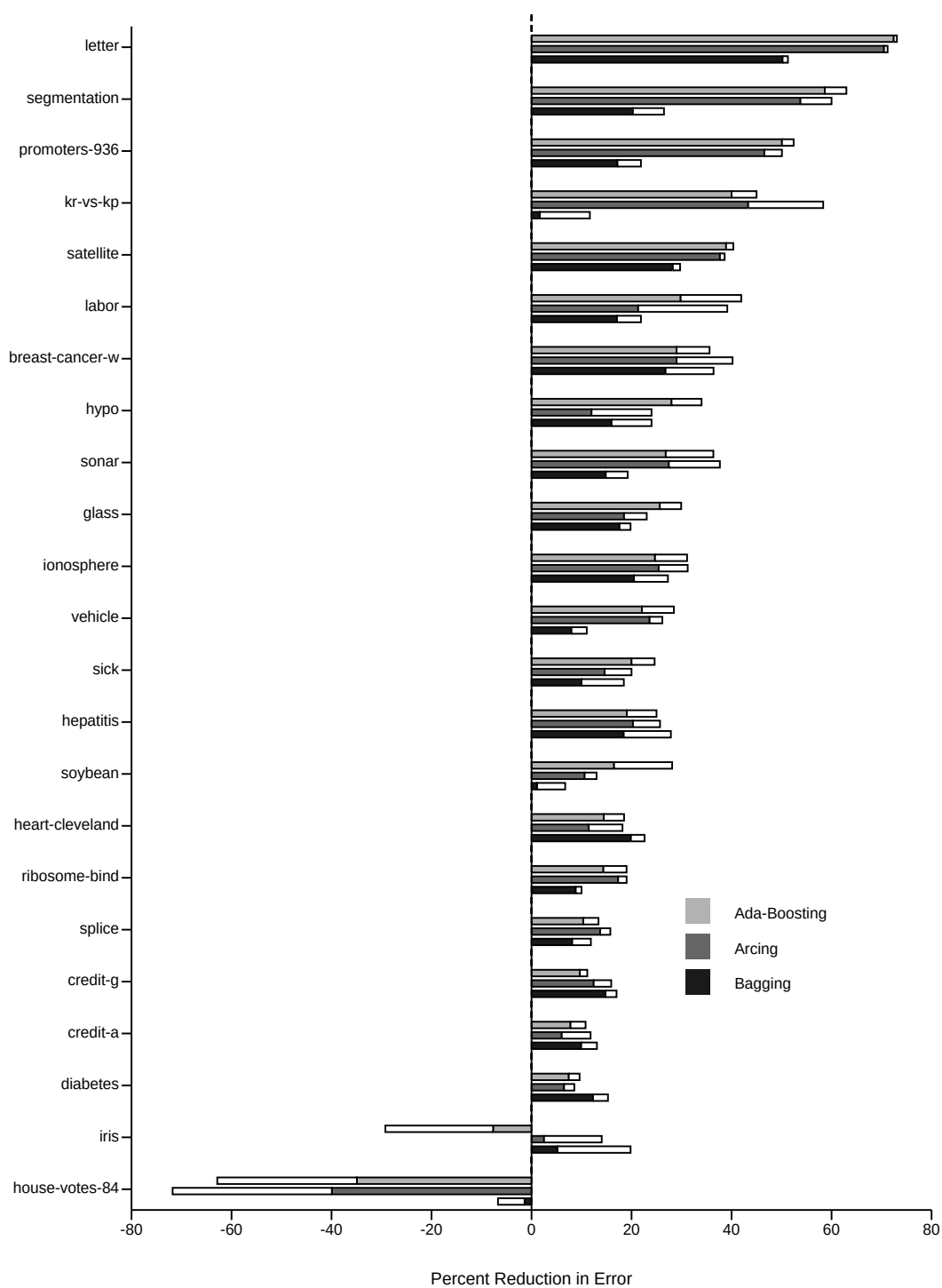


Figure 4: Reduction in error for Ada-Boosting, Arcing, and Bagging decision tree ensembles as a percentage of the original error rate. Also shown (white portion of each bar) is one standard deviation for these results.

Boosting and Arcing) are similar in their results, both for neural networks and decision trees. Furthermore, the Ada-Boosting and Arcing methods produce some of the largest reductions in error. On the other hand, while the Bagging method consistently produces reductions in error for almost all of the cases, with neural networks the Boosting methods can sometimes result in an increase in error.

Looking at the ordering of the data sets in the two figures (the results are sorted by the percentage of reduction using the Ada-Boosting method), we note that the data sets for which the ensemble methods seem to work well are somewhat consistent across both neural networks and decision trees. For the few domains which see increases in error, it is difficult to reach strong conclusions since the ensemble methods seem to do well for a large number of domains. One domain on which the Boosting methods do uniformly poorly is the house-votes-84 domain. As we discuss later, there may noise in this domain’s examples that causes the Boosting methods significant problems.

3.4 Ensemble Size

Early work (Hansen & Salamon, 1990) on ensembles suggested that ensembles with as few as ten members were adequate to sufficiently reduce test-set error. While this claim may be true for the earlier proposed ensembles, the Boosting literature (Schapire, Freund, Bartlett, & Lee, 1997) has recently suggested (based on a few data sets with decision trees) that it is possible to further reduce test-set error even after ten members have been added to an ensemble (and they note that this result also applies to Bagging). In this section, we perform additional experiments to further investigate the appropriate size of an ensemble. Figure 5 shows the composite error rate over all of our data sets for neural network and decision tree ensembles using up to 100 classifiers. Our experiments indicate that most of the methods produce similarly shaped curves. As expected, much of the reduction in error due to adding classifiers to an ensemble comes with the first few classifiers; however, there is some variation with respect to where the error reduction finally asymptotes.

For both Bagging and Boosting applied to neural networks, much of the reduction in error appears to have occurred after ten to fifteen classifiers. A similar conclusion can be reached for Bagging and decision trees, which is consistent with Breiman (1996a). But Ada-boosting and Arcing continue to measurably improve their test-set error until around 25 classifiers for decision trees. At 25 classifiers the error reduction for both methods appears to have nearly asymptoted to a plateau. Therefore, the results reported in this paper are of an ensemble size of 25 (i.e., a sufficient yet manageable size for qualitative analysis). It was traditionally believed (Freund & Schapire, 1996) that small reductions in test-set error may continue indefinitely for boosting; however, Grove and Schuurmans (1998) demonstrate that Ada-boosting can indeed begin to overfit with *very* large ensemble sizes (10,000 or more members).

3.5 Correlation Among Methods

As suggested above, it appears that the performance of many of the ensemble methods are highly correlated with one another. To help identify these consistencies, Table 3 presents the correlation coefficients of the performance of all seven ensemble methods. For each data set, performance is measured as the ensemble error rate divided by the single-classifier error

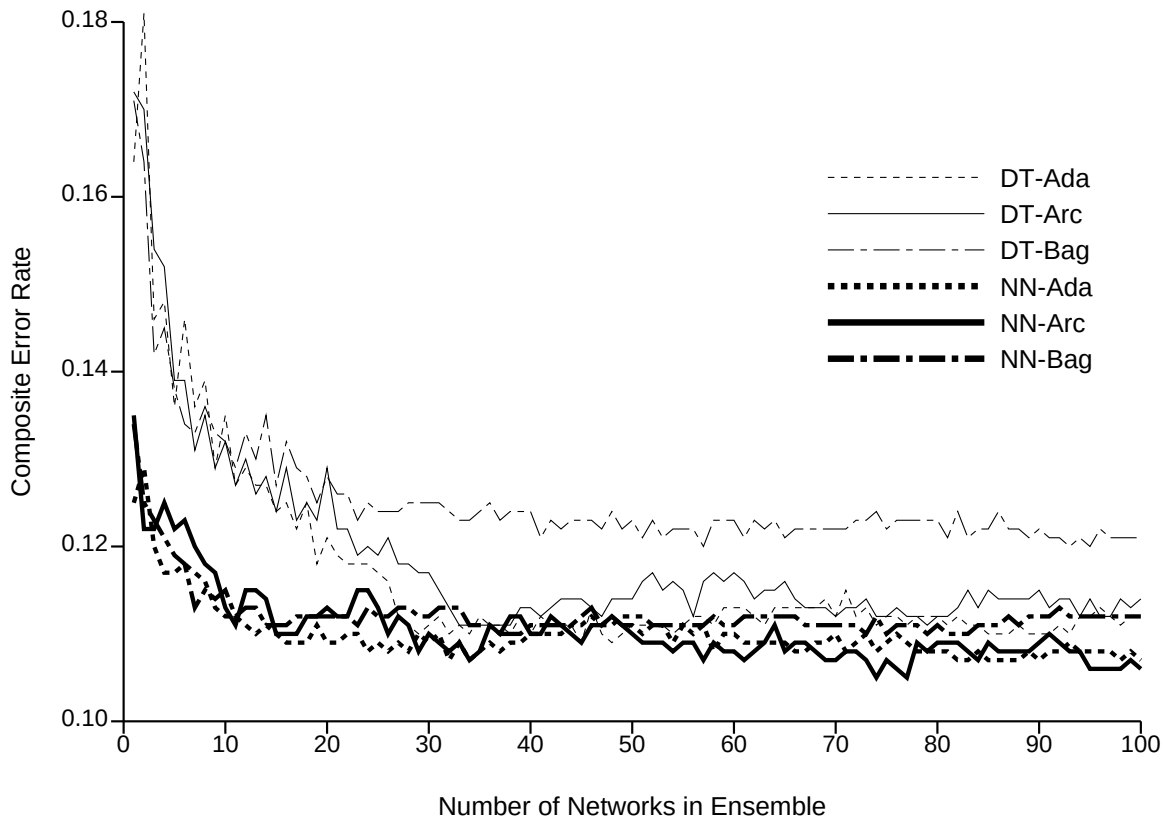


Figure 5: Average test-set error over all 23 data sets used in our studies for ensembles incorporating from one to 100 decision trees or neural networks. The error rate graphed is simply the average of the error rates of the 23 data sets. The alternative of averaging the error over all data points (i.e., weighting a data set’s error rate by its sample size) produces similarly shaped curves.

rate. Thus a high correlation (i.e., one near 1.0) suggests that two methods are consistent in the domains in which they have the greatest impact on test-set error reduction.

Table 3 provides numerous interesting insights. The first is that the neural-network ensemble methods are strongly correlated with one another and the decision-tree ensemble methods are strongly correlated with one another; however, there is less correlation between any neural-network ensemble method and any decision-tree ensemble method. Not surprisingly, Ada-boosting and Arcing are strongly correlated, even across different component learning algorithms. This suggests that Boosting’s effectiveness depends more on the data set than whether the component learning algorithm is a neural network or decision tree. Bagging on the other hand, is not correlated across component learning algorithms. These results are consistent with our later claim that while Boosting is a powerful ensemble method, it is more susceptible to a noisy data set than Bagging.

	Neural Network				Decision Tree		
	Simple	Bagging	Arcing	Ada	Bagging	Arcing	Ada
Simple-NN	1.00	0.88	0.87	0.85	-0.10	0.38	0.37
Bagging-NN	0.88	1.00	0.78	0.78	-0.11	0.35	0.35
Arcing-NN	0.87	0.78	1.00	0.99	0.14	0.61	0.60
Ada-NN	0.85	0.78	0.99	1.00	0.17	0.62	0.63
Bagging-DT	-0.10	-0.11	0.14	0.17	1.00	0.68	0.69
Arcing-DT	0.38	0.35	0.61	0.62	0.68	1.00	0.96
Ada-DT	0.37	0.35	0.60	0.63	0.69	0.96	1.00

Table 3: Performance correlation coefficients across ensemble learning methods. Performance is measured by the ratio of the ensemble method’s test-set error divided by the single component classifier’s test-set error.

3.6 Bagging versus Simple network ensembles

Figure 6 shows the Bagging and Simple network ensemble results from Table 2. These results indicate that often a Simple Ensemble approach will produce results that are as accurate as Bagging (correlation results from Table 3 also support this statement). This suggests that any mechanism which causes a learning method to produce some randomness in the formation of its classifiers can be used to form accurate ensembles, and indeed, Ali and Pazzani (1996) have demonstrated similar results for randomized decision trees.

3.7 Neural Networks versus Decision Trees

Another interesting question is how effective the different methods are for neural networks and decision trees. Figures 7, 8, and 9 compare the error rates and reduction in error values for Ada-Boosting, Arcing, and Bagging respectively. Note that we graph error rate rather than percent reduction in error rate because the baseline for each method (decision trees for Ada-Boosting on decision trees versus neural networks for Ada-Boosting on neural networks) may partially explain the differences in percent reduction. For example, in the promoters-936 problem using Ada-Boosting, the much larger reduction in error for the decision tree approach may be due to the fact that decision trees do not seem to be as effective for this problem, and Ada-Boosting therefore produces a larger percent reduction in the error for decision trees.

The results show that in many cases if a single decision tree had lower (or higher) error than a single neural network on a data set, then the decision-tree ensemble methods also had lower (or higher) error than their neural network counterpart. The exceptions to this rule generally happened on the same data set for all three ensemble methods (e.g., hepatitis, soybean, satellite, credit-a, and heart-cleveland). These results suggest that (a) the performance of the ensemble methods is dependent on both the data set and classifier method, and (b) ensembles can, at least in some cases, overcome the inductive bias of its component learning algorithm.

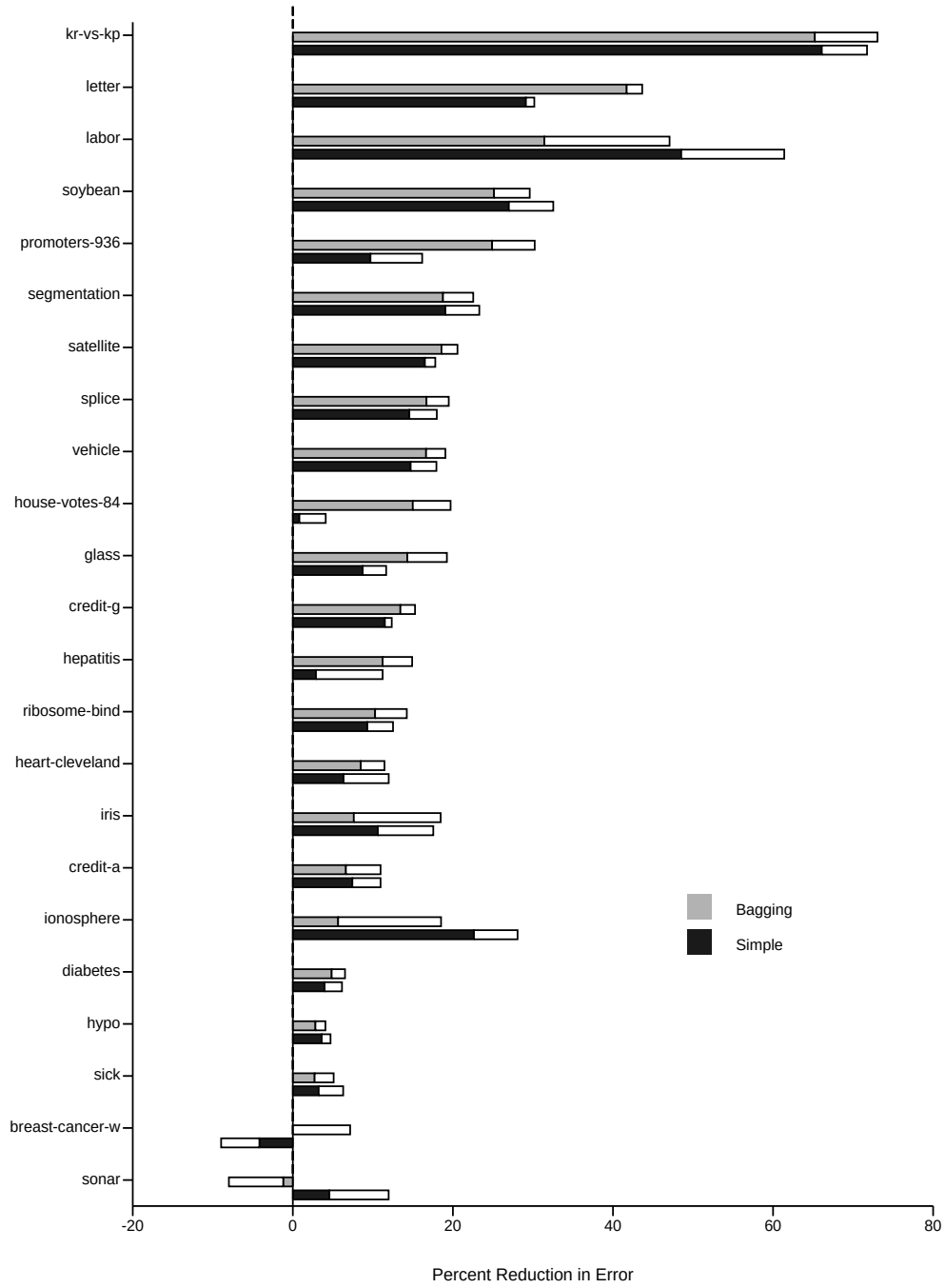


Figure 6: Reduction in error for Bagging and Simple neural network ensembles as a percentage of the original error rate. Also shown (white portion of each bar) is one standard deviation for these results.

POPULAR ENSEMBLE METHODS

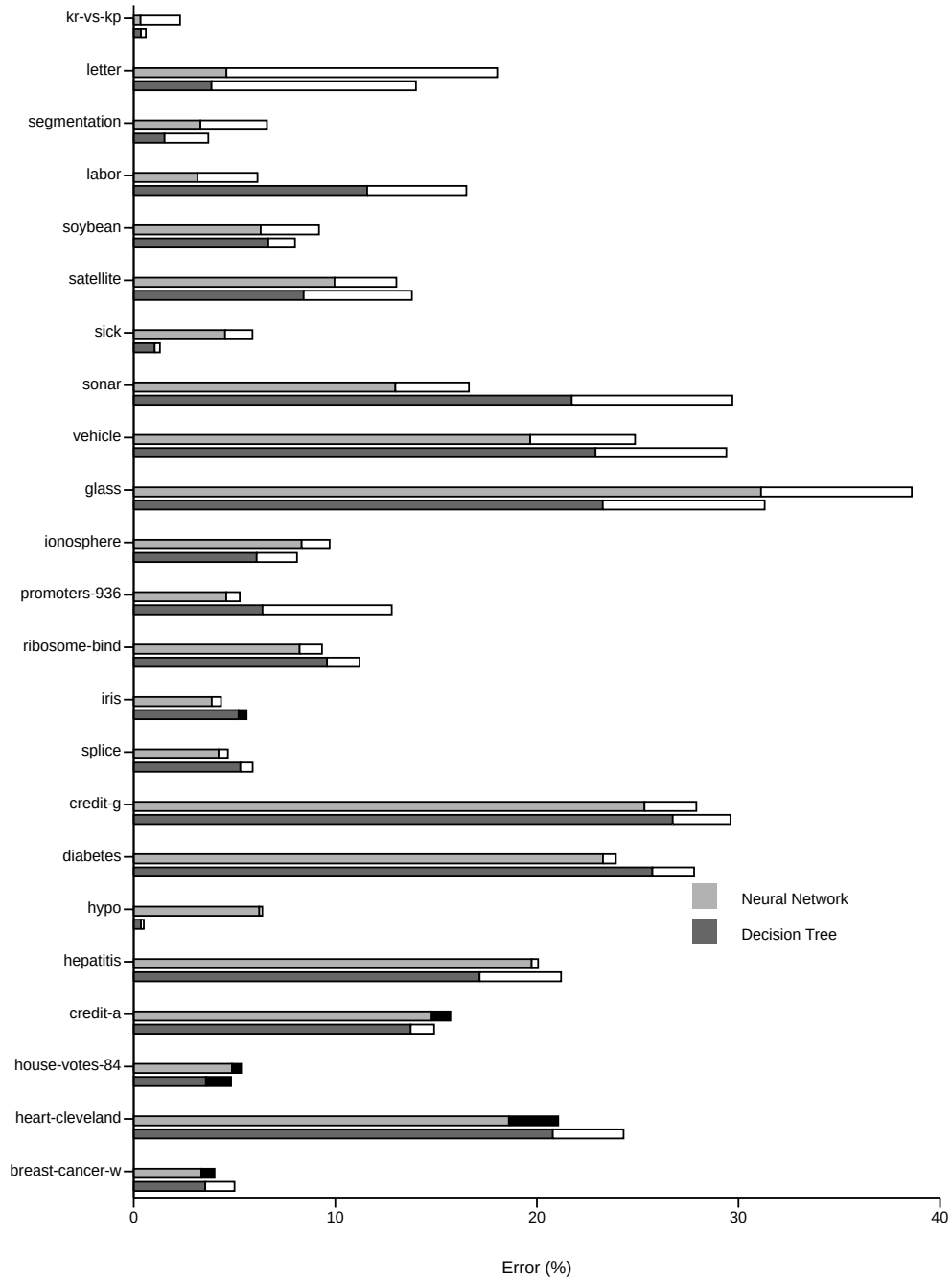


Figure 7: Error rates for Ada-Boosting ensembles. The white portion shows the reduction in error of Ada-Boosting compared to a single classifier while increases in error are shown in black. The data sets are sorted by the ratio of reduction in ensemble error to overall error for neural networks.

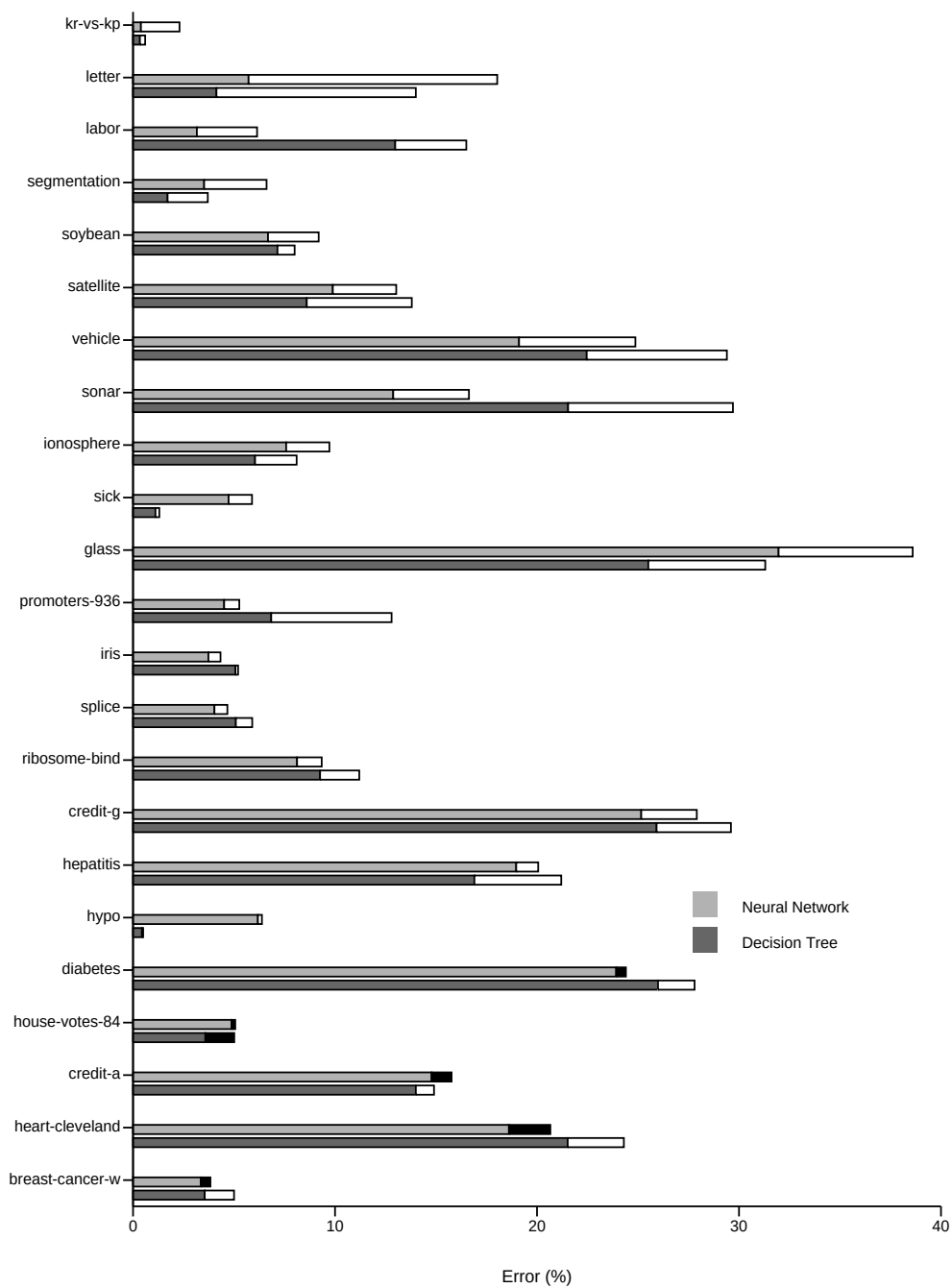


Figure 8: Error rates for Arcing ensembles. The white portion shows the reduction in error of Arcing compared to a single classifier while increases in error are shown in black. The data sets are sorted by the ratio of reduction in ensemble error to overall error for neural networks.

POPULAR ENSEMBLE METHODS

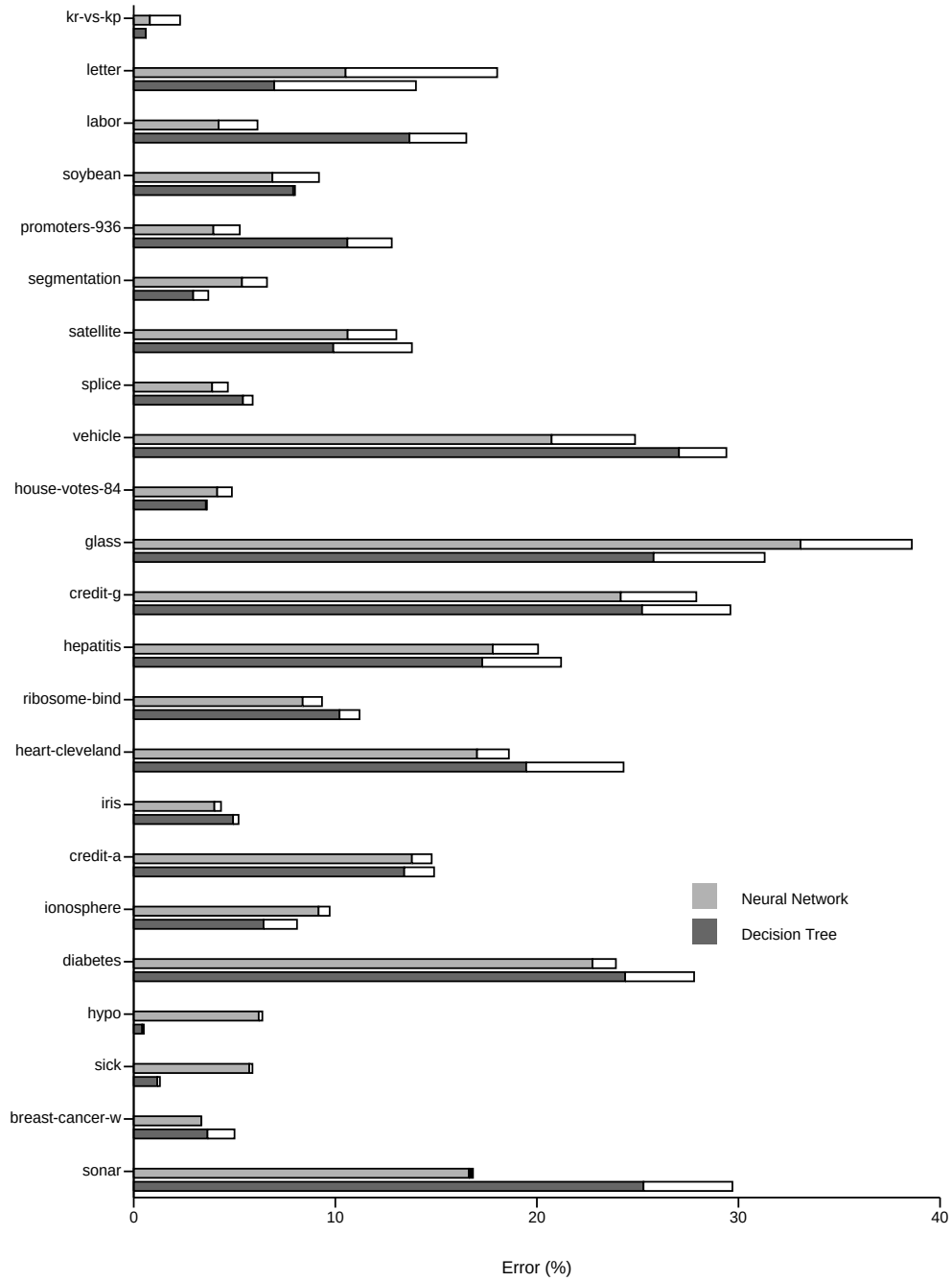


Figure 9: Error rates for Bagging ensembles. The white portion shows the reduction in error of Bagging compared to a single classifier while increases in error are shown in black. The data sets are sorted by the ratio of reduction in ensemble error to overall error for neural networks.

3.8 Boosting and Noise

Freund and Shapire (1996) suggested that the sometimes poor performance of Boosting results from overfitting the training set since later training sets may be over-emphasizing examples that are noise (thus creating extremely poor classifiers). This argument seems especially pertinent to Boosting for two reasons. The first and most obvious reason is that their method for updating the probabilities may be over-emphasizing noisy examples. The second reason is that the classifiers are combined using weighted voting. Previous work (Sollich & Krogh, 1996) has shown that optimizing the combining weights can lead to overfitting while an unweighted voting scheme is generally resilient to overfitting. Friedman et al. (1998) hypothesize that Boosting methods, as additive models, may see increases in error in those situations where the bias of the base classifier is appropriate for the problem being learned. We test this hypothesis in our second set of results presented in this section.

To evaluate the hypothesis that Boosting may be prone to overfitting we performed a set of experiments using the four ensemble neural network methods. We introduced 5%, 10%, 20%, and 30% noise² into four different data sets. At each level we created five different noisy data sets, performed a 10-fold cross validation on each, then averaged over the five results. In Figure 10 we show the reduction in error rate for each of the ensemble methods compared to using a single neural network classifier. These results demonstrate that as the noise level grows, the efficacy of the Simple and Bagging ensembles generally increases while the Arcing and Ada-Boosting ensembles gains in performance are much smaller (or may actually decrease). Note that this effect is more extreme for Ada-Boosting which supports our hypothesis that Ada-Boosting is more affected by noise. This suggests that Boosting’s poor performance for certain data sets may be partially explained by overfitting noise.

To further demonstrate the effect of noise on Boosting we created several sets of artificial data specifically designed to mislead Boosting methods. For each data set we created a simple hyperplane concept based on a set of the features (and also included some irrelevant features). A set of random points were then generated and labeled based on which side of the hyperplane they fell. Then a certain percentage of the points on one side of the hyperplane were mislabeled as being part of the other class. For the experiments shown below we generated five data sets where the concept was based on two linear features, had four irrelevant features, and 20% of the data was mislabeled. We trained five ensembles of neural networks (perceptrons) for each data set and averaged the ensembles’ predictions. Thus these experiments involve learning in situations where the original bias of the learner (a single hyperplane produced by a perceptron) is appropriate for the problem, and as Friedman et al. (1998) suggest, using an additive model may harm performance. Figure 11 shows the resulting error rates for Ada-Boosting, Arcing, and Bagging by the number of networks being combined in the ensemble. These results indicate clearly that in cases where there is noise Bagging’s error rate will not increase as the ensemble size increases whereas the error rate of the Boosting methods may indeed increase as ensemble size increases.

2. X% noise indicates that each feature of the training examples, both input and output features, had X% chance of being randomly perturbed to another feature value for that feature (for continuous features, the set of possible other values was chosen by examining all of the training examples).

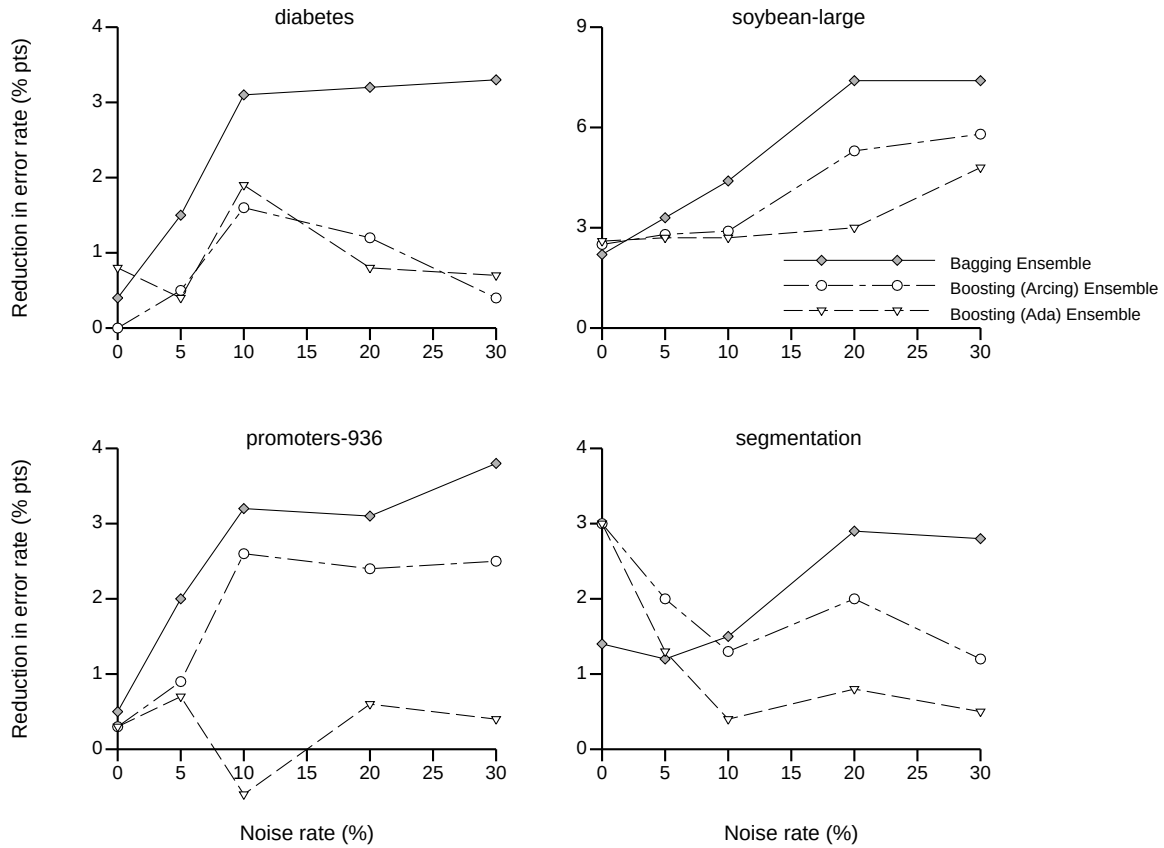


Figure 10: Simple, Bagging, and Boosting (Arcing and Ada) neural network ensemble reduction in error as compared to using a single neural network. Graphed is the percentage *point* reduction in error (e.g., for 5% noise in the segmentation data set, if the single network method had an error rate of 15.9% and the Bagging method had an error rate of 14.7%, then this is graphed as a 1.2 percentage point reduction in the error rate).

Additional tests (not shown here) show that Ada-Boosting's error rate becomes worse when restarting is not employed.

This conclusion dovetails nicely with Schapire et al.'s (1997) recent discussion where they note that the effectiveness of a voting method can be measured by examining the *margins* of the examples. (The margin is the difference between the number of correct and incorrect votes for an example.) In a simple resampling method such as Bagging, each resulting classifier focuses on increasing the margin for as many of the examples as possible. But in a Boosting method, later classifiers focus on increasing the margins for examples with poor current margins. As Schapire et al. (1997) note, this is a very effective strategy if the overall accuracy of the resulting classifier does not drop significantly. For a problem with noise, focusing on misclassified examples may cause a classifier to focus on boosting the margins of (noisy) examples that would in fact be misleading in overall classification.

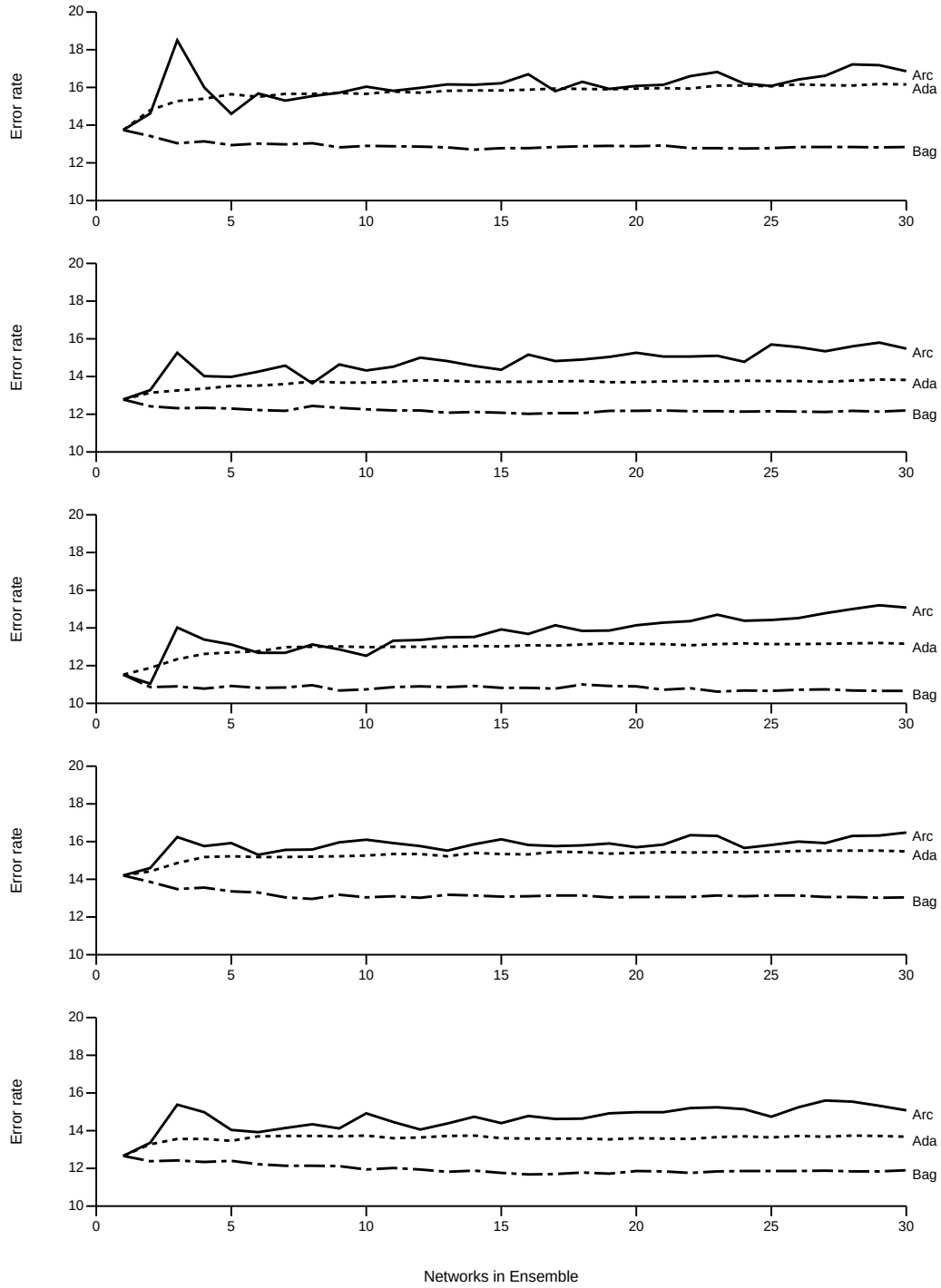


Figure 11: Error rates by the size of ensemble for Ada-Boosting, Arcing, and Bagging ensembles for five different artificial data sets containing one-sided noise (see text for description).

4. Future Work

One interesting question we plan to investigate is how effective a single classifier approach might be if it was allowed to use the time it takes the ensemble method to train multiple classifiers to explore its concept space. For example, a neural network approach could perform pilot studies using the training set to select appropriate values of parameters such as hidden units, learning rate, etc.

We plan to compare Bagging and Boosting methods to other methods introduced recently. In particular we intend to examine the use of Stacking (Wolpert, 1992) as a method of *training* a combining function, so as to avoid the effect of having to weight classifiers. We also plan to compare Bagging and Boosting to other methods such as Opitz and Shavlik's (1996b) approach to creating an ensemble. This approach uses genetic search to find classifiers that are accurate and differ in their predictions.

Finally, since the Boosting methods are extremely successful in many domains, we plan to investigate novel approaches that will retain the benefits of Boosting. The goal will be to create a learner where you can essentially push a start button and let it run. To do this we would try to preserve the benefits of Boosting while preventing overfitting on noisy data sets. One possible approach would be to use a holdout training set (a tuning set) to evaluate the performance of the Boosting ensemble to determine when the accuracy is no longer increasing. Another approach would be to use pilot studies to determine an "optimal" number of classifiers to use in an ensemble.

5. Additional Related Work

As mentioned before, the idea of using an ensemble of classifiers rather than the single best classifier has been proposed by several people. In Section 2, we present a framework for these systems, some theories of what makes an effective ensemble, an extensive covering of the Bagging and Boosting algorithms, and a discussion on the bias plus variance decomposition. Section 3 referred to empirical studies similar to ours; these methods differ from ours in that they were limited to decision trees, generally with fewer data sets. We cover additional related work in this section.

Lincoln and Skrzypek (1989), Mani (1991) and the forecasting literature (Clemen, 1989; Granger, 1989) indicate that a simple averaging of the predictors generates a very good composite model; however, many later researchers (Alpaydin, 1993; Asker & Maclin, 1997a, 1997b; Breiman, 1996c; Hashem, 1997; Maclin, 1998; Perrone, 1992; Wolpert, 1992; Zhang, Mesirov, & Waltz, 1992) have further improved generalization with voting schemes that are complex combinations of each predictor's output. One must be careful in this case, since optimizing the combining weights can easily lead to the problem of overfitting which simple averaging seems to avoid (Sollich & Krogh, 1996).

Most approaches only *indirectly* try to generate highly correct classifiers that disagree as much as possible. These methods try to create diverse classifiers by training classifiers with dissimilar learning parameters (Alpaydin, 1993), different classifier architectures (Hashem, 1997), various initial neural-network weight settings (Maclin & Opitz, 1997; Maclin & Shavlik, 1995), or separate partitions of the training set (Breiman, 1996a; Krogh & Vedelsby, 1995). Boosting on the other hand is active in trying to generate highly correct networks

since it accentuates examples currently classified incorrectly by previous members of the ensemble.

ADDEMUP (Opitz & Shavlik, 1996a, 1996b) is another example of an approach that directly tries to create a diverse ensemble. ADDEMUP uses genetic algorithms to search explicitly for a highly diverse set of accurate trained networks. ADDEMUP works by first creating an initial population, then uses genetic operators to create new networks continually, keeping the set of networks that are highly accurate while disagreeing with each other as much as possible. ADDEMUP is also effective at incorporating prior knowledge, if available, to improve the quality of its ensemble.

An alternate approach to the ensemble framework is to train individual networks on a subtask, and to then combine these predictions with a “gating” function that depends on the input. Jacobs et al.’s (1991) adaptive mixtures of local experts, Baxt’s (1992) method for identifying myocardial infarction, and Nowlan and Sejnowski’s (1992) visual model all train networks to learn specific subtasks. The key idea of these techniques is that a decomposition of the problem into specific subtasks might lead to more efficient representations and training (Hampshire & Waibel, 1989).

Once a problem is broken into subtasks, the resulting solutions need to be combined. Jacobs et al. (1991) propose having the gating function be a network that *learns* how to allocate examples to the experts. Thus the gating network allocates each example to one or more experts, and the backpropagated errors and resulting weight changes are then restricted to these networks (and the gating function). Tresp and Taniguchi (1995) propose a method for determining the gating function *after* the problem has been decomposed and the experts trained. Their gating function is an input-dependent, linear-weighting function that is determined by a combination of the networks’ diversity on the current input with the likelihood that these networks have seen data “near” that input.

Although the mixtures of experts and ensemble paradigms seem very similar, they are in fact quite distinct from a statistical point of view. The mixtures-of-experts model makes the assumption that a single expert is responsible for each example. In this case, each expert is a model of a region of the input space, and the job of the gating function is to decide from which model the data point originates. Since each network in the ensemble approach learns the whole task rather than just some subtask and thus makes no such mutual exclusivity assumption, ensembles are appropriate when no one model is highly likely to be correct for any one point in the input space.

6. Conclusions

This paper presents a comprehensive empirical evaluation of Bagging and Boosting for neural networks and decision trees. Our results demonstrate that a Bagging ensemble nearly always outperforms a single classifier. Our results also show that a Boosting ensemble can greatly outperform both Bagging and a single classifier. However, for some data sets Boosting may show zero gain or even a decrease in performance from a single classifier. Further tests indicate that Boosting may suffer from overfitting in the presence of noise which may explain some of the decreases in performance for Boosting. We also found that a simple ensemble approach of using neural networks that differ only in their random initial weight settings performed surprisingly well, often doing as well as the Bagging.

Analysis of our results suggests that the performance of both Boosting methods (Ada-Boosting and Arcing) is at least partly dependent on the data set being examined, where Bagging shows much less correlation. The strong correlations for Boosting may be partially explained by its sensitivity to noise, a claim supported by additional tests. Finally, we show that much of the performance enhancement for an ensemble comes with the first few classifiers combined, but that Boosting decision trees may continue to further improve with larger ensemble sizes.

In conclusion, as a general technique for decision trees and neural networks, Bagging is probably appropriate for most problems, but when appropriate, Boosting (either Arcing or Ada) may produce larger gains in accuracy.

Acknowledgments

This research was partially supported by University of Minnesota Grants-in-Aid to both authors. Dave Opitz was also supported by National Science Foundation grant IRI-9734419, the Montana DOE/EPSCoR Petroleum Reservoir Characterization Project, a MONTS grant supported by the University of Montana, and a Montana Science Technology Alliance grant. This is an extended version of a paper published in the *Fourteenth National Conference on Artificial Intelligence*.

References

- Ali, K., & Pazzani, M. (1996). Error reduction through learning multiple descriptions. *Machine Learning*, 24, 173–202.
- Alpaydin, E. (1993). Multiple networks for function learning. In *Proceedings of the 1993 IEEE International Conference on Neural Networks*, Vol. I, pp. 27–32 San Francisco.
- Arbib, M. (Ed.). (1995). *The Handbook of Brain Theory and Neural Networks*. MIT Press.
- Asker, L., & Maclin, R. (1997a). Ensembles as a sequence of classifiers. In *Proceedings of the Fifteenth International Joint Conference on Artificial Intelligence*, pp. 860–865 Nagoya, Japan.
- Asker, L., & Maclin, R. (1997b). Feature engineering and classifier selection: A case study in Venusian volcano detection. In *Proceedings of the Fourteenth International Conference on Machine Learning*, pp. 3–11 Nashville, TN.
- Bauer, E., & Kohavi, R. (1999). An empirical comparison of voting classification algorithms: Bagging, boosting, and variants. *Machine Learning*, 36, 105–139.
- Baxt, W. (1992). Improving the accuracy of an artificial neural network using multiple differently trained networks. *Neural Computation*, 4, 772–780.
- Breiman, L. (1996a). Bagging predictors. *Machine Learning*, 24(2), 123–140.
- Breiman, L. (1996b). Bias, variance, and arcing classifiers. Tech. rep. 460, UC-Berkeley, Berkeley, CA.

- Breiman, L. (1996c). Stacked regressions. *Machine Learning*, 24(1), 49–64.
- Clemen, R. (1989). Combining forecasts: A review and annotated bibliography. *Journal of Forecasting*, 5, 559–583.
- Drucker, H., & Cortes, C. (1996). Boosting decision trees. In Touretsky, D., Mozer, M., & Hasselmo, M. (Eds.), *Advances in Neural Information Processing Systems*, Vol. 8, pp. 479–485 Cambridge, MA. MIT Press.
- Drucker, H., Cortes, C., Jackel, L., LeCun, Y., & Vapnik, V. (1994). Boosting and other machine learning algorithms. In *Proceedings of the Eleventh International Conference on Machine Learning*, pp. 53–61 New Brunswick, NJ.
- Efron, B., & Tibshirani, R. (1993). *An Introduction to the Bootstrap*. Chapman and Hall, New York.
- Fisher, D., & McKusick, K. (1989). An empirical comparison of ID3 and back-propagation. In *Proceedings of the Eleventh International Joint Conference on Artificial Intelligence*, pp. 788–793 Detroit, MI.
- Freund, Y., & Schapire, R. (1996). Experiments with a new boosting algorithm. In *Proceedings of the Thirteenth International Conference on Machine Learning*, pp. 148–156 Bari, Italy.
- Friedman, J. (1996). On bias, variance, 0/1-loss, and the curse-of-dimensionality. *Journal of Data Mining and Knowledge Discovery*, 1.
- Friedman, J., Hastie, T., & Tibshirani, R. (1998). Additive logistic regression: A statistical view of boosting. (<http://www-stat.stanford.edu/~jhf>).
- Geman, S., Bienenstock, E., & Doursat, R. (1992). Neural networks and the bias/variance dilemma. *Neural Computation*, 4, 1–58.
- Granger, C. (1989). Combining forecasts: Twenty years later. *Journal of Forecasting*, 8, 167–173.
- Grove, A., & Schuurmans, D. (1998). Boosting in the limit: Maximizing the margin of learned ensembles. In *Proceedings of the Fifteenth National Conference on Artificial Intelligence*, pp. 692–699 Madison, WI.
- Hampshire, J., & Waibel, A. (1989). The meta-pi network: Building distributed knowledge representations for robust pattern recognition. Tech. rep. CMU-CS-89-166, CMU, Pittsburgh, PA.
- Hansen, L., & Salamon, P. (1990). Neural network ensembles. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 12, 993–1001.
- Hashem, S. (1997). Optimal linear combinations of neural networks. *Neural Networks*, 10(4), 599–614.

- Jacobs, R., Jordan, M., Nowlan, S., & Hinton, G. (1991). Adaptive mixtures of local experts. *Neural Computation*, 3, 79–87.
- Kohavi, R., & Wolpert, D. (1996). Bias plus variance decomposition for zero-one loss functions. In *Proceedings of the Thirteenth International Conference on Machine Learning*, pp. 275–283 Bari, Italy.
- Kong, E., & Dietterich, T. (1995). Error-correcting output coding corrects bias and variance. In *Proceedings of the Twelfth International Conference on Machine Learning*, pp. 313–321 Tahoe City, CA.
- Krogh, A., & Vedelsby, J. (1995). Neural network ensembles, cross validation, and active learning. In Tesauero, G., Touretzky, D., & Leen, T. (Eds.), *Advances in Neural Information Processing Systems*, Vol. 7, pp. 231–238 Cambridge, MA. MIT Press.
- Lincoln, W., & Skrzypek, J. (1989). Synergy of clustering multiple back propagation networks. In Touretzky, D. (Ed.), *Advances in Neural Information Processing Systems*, Vol. 2, pp. 650–659 San Mateo, CA. Morgan Kaufmann.
- Maclin, R. (1998). Boosting classifiers regionally. In *Proceedings of the Fifteenth National Conference on Artificial Intelligence*, pp. 700–705 Madison, WI.
- Maclin, R., & Opitz, D. (1997). An empirical evaluation of bagging and boosting. In *Proceedings of the Fourteenth National Conference on Artificial Intelligence*, pp. 546–551 Providence, RI.
- Maclin, R., & Shavlik, J. (1995). Combining the predictions of multiple classifiers: Using competitive learning to initialize neural networks. In *Proceedings of the Fourteenth International Joint Conference on Artificial Intelligence*, pp. 524–530 Montreal, Canada.
- Mani, G. (1991). Lowering variance of decisions by using artificial neural network portfolios. *Neural Computation*, 3, 484–486.
- Mooney, R., Shavlik, J., Towell, G., & Gove, A. (1989). An experimental comparison of symbolic and connectionist learning algorithms. In *Proceedings of the Eleventh International Joint Conference on Artificial Intelligence*, pp. 775–780 Detroit, MI.
- Murphy, P. M., & Aha, D. W. (1994). UCI repository of machine learning databases (machine-readable data repository). University of California-Irvine, Department of Information and Computer Science.
- Nowlan, S., & Sejnowski, T. (1992). Filter selection model for generating visual motion signals. In Hanson, S., Cowan, J., & Giles, C. (Eds.), *Advances in Neural Information Processing Systems*, Vol. 5, pp. 369–376 San Mateo, CA. Morgan Kaufmann.
- Opitz, D., & Shavlik, J. (1996a). Actively searching for an effective neural-network ensemble. *Connection Science*, 8(3/4), 337–353.

- Opitz, D., & Shavlik, J. (1996b). Generating accurate and diverse members of a neural-network ensemble. In Touretsky, D., Mozer, M., & Hasselmo, M. (Eds.), *Advances in Neural Information Processing Systems*, Vol. 8, pp. 535–541 Cambridge, MA. MIT Press.
- Perrone, M. (1992). A soft-competitive splitting rule for adaptive tree-structured neural networks. In *Proceedings of the International Joint Conference on Neural Networks*, pp. 689–693 Baltimore, MD.
- Perrone, M. (1993). *Improving Regression Estimation: Averaging Methods for Variance Reduction with Extension to General Convex Measure Optimization*. Ph.D. thesis, Brown University, Providence, RI.
- Quinlan, J. (1993). *C4.5: Programs for Machine Learning*. Morgan Kaufmann, San Mateo, CA.
- Quinlan, J. R. (1996). Bagging, boosting, and c4.5. In *Proceedings of the Thirteenth National Conference on Artificial Intelligence*, pp. 725–730. Portland, OR.
- Rumelhart, D., Hinton, G., & Williams, R. (1986). Learning internal representations by error propagation. In Rumelhart, D., & McClelland, J. (Eds.), *Parallel Distributed Processing: Explorations in the microstructure of cognition. Volume 1: Foundations*, pp. 318–363. MIT Press, Cambridge, MA.
- Schapire, R. (1990). The strength of weak learnability. *Machine Learning*, 5(2), 197–227.
- Schapire, R., Freund, Y., Bartlett, P., & Lee, W. (1997). Boosting the margin: A new explanation for the effectiveness of voting methods. In *Proceedings of the Fourteenth International Conference on Machine Learning*, pp. 322–330 Nashville, TN.
- Sollich, P., & Krogh, A. (1996). Learning with ensembles: How over-fitting can be useful. In Touretsky, D., Mozer, M., & Hasselmo, M. (Eds.), *Advances in Neural Information Processing Systems*, Vol. 8, pp. 190–196 Cambridge, MA. MIT Press.
- Tresp, V., & Taniguchi, M. (1995). Combining estimators using non-constant weighting functions. In Tesauro, G., Touretzky, D., & Leen, T. (Eds.), *Advances in Neural Information Processing Systems*, Vol. 7, pp. 419–426 Cambridge, MA. MIT Press.
- Wolpert, D. (1992). Stacked generalization. *Neural Networks*, 5, 241–259.
- Zhang, X., Mesirov, J., & Waltz, D. (1992). Hybrid system for protein secondary structure prediction. *Journal of Molecular Biology*, 225, 1049–1063.

Appendix

Tables 4 and 5 show the complete results for the first set of experiments used in this paper.

Data Set	Single			Simple		Bagging		Arcing		Boosting	
	Err	SD	Best	Err	SD	Err	SD	Err	SD	Err	SD
breast-cancer-w	3.4	0.3	2.9	3.5	0.2	3.4	0.2	3.8	0.4	4.0	0.4
credit-a	14.8	0.7	13.6	13.7	0.5	13.8	0.6	15.8	0.6	15.7	0.6
credit-g	27.9	0.8	26.2	24.7	0.2	24.2	0.5	25.2	0.8	25.3	0.1
diabetes	23.9	0.9	22.6	23.0	0.5	22.8	0.4	24.4	0.2	23.3	1.2
glass	38.6	1.5	36.9	35.2	1.1	33.1	1.9	32.0	2.4	31.1	0.9
heart-cleveland	18.6	1.0	16.8	17.4	1.1	17.0	0.6	20.7	1.6	21.1	0.9
hepatitis	20.1	1.6	19.1	19.5	1.7	17.8	0.7	19.0	1.3	19.7	0.9
house-votes-84	4.9	0.6	4.1	4.8	0.2	4.1	0.2	5.1	0.5	5.3	0.5
hypo	6.4	0.2	6.2	6.2	0.1	6.2	0.1	6.2	0.1	6.2	0.1
ionosphere	9.7	1.3	7.4	7.5	0.5	9.2	1.2	7.6	0.6	8.3	0.5
iris	4.3	1.7	2.0	3.9	0.3	4.0	0.5	3.7	0.6	3.9	1.0
kr-vs-kp	2.3	0.7	1.5	0.8	0.1	0.8	0.2	0.4	0.1	0.3	0.1
labor	6.1	1.5	3.5	3.2	0.8	4.2	1.0	3.2	0.8	3.2	0.8
letter	18.0	0.3	17.6	12.8	0.2	10.5	0.3	5.7	0.4	4.6	0.1
promoters-936	5.3	0.6	4.5	4.8	0.3	4.0	0.3	4.5	0.2	4.6	0.3
ribosome-bind	9.3	0.4	8.9	8.5	0.3	8.4	0.4	8.1	0.2	8.2	0.3
satellite	13.0	0.3	12.6	10.9	0.2	10.6	0.3	9.9	0.2	10.0	0.3
segmentation	6.6	0.7	5.7	5.3	0.3	5.4	0.2	3.5	0.2	3.3	0.2
sick	5.9	0.5	5.2	5.7	0.2	5.7	0.1	4.7	0.2	4.5	0.3
sonar	16.6	1.5	14.9	15.9	1.2	16.8	1.1	12.9	1.5	13.0	1.5
soybean	9.2	1.1	7.0	6.7	0.5	6.9	0.4	6.7	0.5	6.3	0.6
splice	4.7	0.2	4.5	4.0	0.2	3.9	0.1	4.0	0.1	4.2	0.1
vehicle	24.9	1.2	22.9	21.2	0.8	20.7	0.6	19.1	1.0	19.7	1.0

Table 4: Neural network test set error rates and standard deviation values for those error rates for (1) a single neural network classifier; (2) a simple neural network ensemble; (3) a Bagging ensemble; (4) an Arcing ensemble; and (5) and Ada-Boosting ensemble. Also shown (results column 3) is the “best” result produced from all of the single network results run using all of the training data.

Data Set	Single			Bagging		Arcing		Boosting	
	Err	SD	Best	Err	SD	Err	SD	Err	SD
breast-cancer-w	5.0	0.7	4.0	3.7	0.5	3.5	0.6	3.5	0.3
credit-a	14.9	0.8	14.2	13.4	0.5	14.0	0.9	13.7	0.5
credit-g	29.6	1.0	28.7	25.2	0.7	25.9	1.0	26.7	0.4
diabetes	27.8	1.0	26.7	24.4	0.8	26.0	0.6	25.7	0.6
glass	31.3	2.1	28.5	25.8	0.7	25.5	1.4	23.3	1.3
heart-cleveland	24.3	1.3	22.7	19.5	0.7	21.5	1.6	20.8	1.0
hepatitis	21.2	1.2	20.0	17.3	2.0	16.9	1.1	17.2	1.3
house-votes-84	3.6	0.3	3.2	3.6	0.2	5.0	1.1	4.8	1.0
hypo	0.5	0.1	0.4	0.4	0.0	0.4	0.1	0.4	0.0
ionosphere	8.1	0.7	7.1	6.4	0.6	6.0	0.5	6.1	0.5
iris	5.2	0.7	5.3	4.9	0.8	5.1	0.6	5.6	1.1
kr-vs-kp	0.6	0.1	0.5	0.6	0.1	0.3	0.1	0.4	0.0
labor	16.5	3.4	12.7	13.7	0.8	13.0	2.9	11.6	2.0
letter	14.0	0.8	12.2	7.0	0.1	4.1	0.1	3.9	0.1
promoters-936	12.8	0.4	12.5	10.6	0.6	6.8	0.5	6.4	0.3
ribosome-bind	11.2	0.6	10.8	10.2	0.1	9.3	0.2	9.6	0.5
satellite	13.8	0.4	13.5	9.9	0.2	8.6	0.1	8.4	0.2
segmentation	3.7	0.2	3.4	3.0	0.2	1.7	0.2	1.5	0.2
sick	1.3	0.9	1.1	1.2	0.1	1.1	0.1	1.0	0.1
sonar	29.7	1.9	26.9	25.3	1.3	21.5	3.0	21.7	2.8
soybean	8.0	0.5	7.5	7.9	0.5	7.2	0.2	6.7	0.9
splice	5.9	0.3	5.7	5.4	0.2	5.1	0.1	5.3	0.2
vehicle	29.4	0.7	28.6	27.1	0.9	22.5	0.8	22.9	1.9

Table 5: Decision tree test set error rates and standard deviation values for those error rates for (1) a single decision tree classifier; (2) a Bagging ensemble; (3) an Arcing ensemble; and (4) and Ada-Boosting ensemble. Also shown (results column 3) is the “best” result produced from all of the single tree results run using all of the training data.

- ▼ Unfortunately, trees generally do not have the same level of predictive accuracy as some of the other regression and classification approaches seen in this book.
- ▼ Additionally, trees can be very non-robust. In other words, a small change in the data can cause a large change in the final estimated tree.

However, by aggregating many decision trees, using methods like *bagging*, *random forests*, and *boosting*, the predictive performance of trees can be substantially improved. We introduce these concepts in the next section.

8.2 Bagging, Random Forests, Boosting, and Bayesian Additive Regression Trees

An *ensemble* method is an approach that combines many simple “building block” models in order to obtain a single and potentially very powerful model. These simple building block models are sometimes known as *weak learners*, since they may lead to mediocre predictions on their own.

We will now discuss bagging, random forests, boosting, and Bayesian additive regression trees. These are ensemble methods for which the simple building block is a regression or a classification tree.

ensemble
weak
learners

8.2.1 Bagging

The bootstrap, introduced in Chapter 5, is an extremely powerful idea. It is used in many situations in which it is hard or even impossible to directly compute the standard deviation of a quantity of interest. We see here that the bootstrap can be used in a completely different context, in order to improve statistical learning methods such as decision trees.

The decision trees discussed in Section 8.1 suffer from *high variance*. This means that if we split the training data into two parts at random, and fit a decision tree to both halves, the results that we get could be quite different. In contrast, a procedure with *low variance* will yield similar results if applied repeatedly to distinct data sets; linear regression tends to have low variance, if the ratio of n to p is moderately large. *Bootstrap aggregation*, or *bagging*, is a general-purpose procedure for reducing the variance of a statistical learning method; we introduce it here because it is particularly useful and frequently used in the context of decision trees.

bagging

Recall that given a set of n independent observations $Z_1, \dots, Z_n$, each with variance σ^2 , the variance of the mean $\bar{Z}$ of the observations is given by σ^2/n . In other words, *averaging a set of observations reduces variance*. Hence a natural way to reduce the variance and increase the test set accuracy of a statistical learning method is to take many training sets from

the population, build a separate prediction model using each training set, and average the resulting predictions. In other words, we could calculate $\hat{f}^1(x), \hat{f}^2(x), \dots, \hat{f}^B(x)$ using B separate training sets, and average them in order to obtain a single low-variance statistical learning model, given by

$$\hat{f}_{\text{avg}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^b(x).$$

Of course, this is not practical because we generally do not have access to multiple training sets. Instead, we can bootstrap, by taking repeated samples from the (single) training data set. In this approach we generate B different bootstrapped training data sets. We then train our method on the b th bootstrapped training set in order to get $\hat{f}^{*b}(x)$, and finally average all the predictions, to obtain

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}^{*b}(x).$$

This is called bagging.

While bagging can improve predictions for many regression methods, it is particularly useful for decision trees. To apply bagging to regression trees, we simply construct B regression trees using B bootstrapped training sets, and average the resulting predictions. These trees are grown deep, and are not pruned. Hence each individual tree has high variance, but low bias. Averaging these B trees reduces the variance. Bagging has been demonstrated to give impressive improvements in accuracy by combining together hundreds or even thousands of trees into a single procedure.

Thus far, we have described the bagging procedure in the regression context, to predict a quantitative outcome Y . How can bagging be extended to a classification problem where Y is qualitative? In that situation, there are a few possible approaches, but the simplest is as follows. For a given test observation, we can record the class predicted by each of the B trees, and take a *majority vote*: the overall prediction is the most commonly occurring class among the B predictions.

majority
vote

Figure 8.8 shows the results from bagging trees on the **Heart** data. The test error rate is shown as a function of B , the number of trees constructed using bootstrapped training data sets. We see that the bagging test error rate is slightly lower in this case than the test error rate obtained from a single tree. The number of trees B is not a critical parameter with bagging; using a very large value of B will not lead to overfitting. In practice we use a value of B sufficiently large that the error has settled down. Using $B = 100$ is sufficient to achieve good performance in this example.

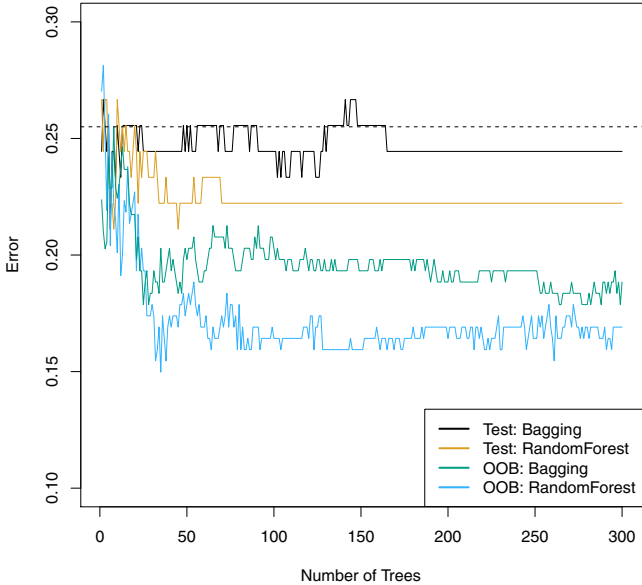


FIGURE 8.8. Bagging and random forest results for the **Heart** data. The test error (black and orange) is shown as a function of B , the number of bootstrapped training sets used. Random forests were applied with $m = \sqrt{p}$. The dashed line indicates the test error resulting from a single classification tree. The green and blue traces show the OOB error, which in this case is — by chance — considerably lower.

Out-of-Bag Error Estimation

It turns out that there is a very straightforward way to estimate the test error of a bagged model, without the need to perform cross-validation or the validation set approach. Recall that the key to bagging is that trees are repeatedly fit to bootstrapped subsets of the observations. One can show that on average, each bagged tree makes use of around two-thirds of the observations.³ The remaining one-third of the observations not used to fit a given bagged tree are referred to as the *out-of-bag* (OOB) observations. We can predict the response for the i th observation using each of the trees in which that observation was OOB. This will yield around $B/3$ predictions for the i th observation. In order to obtain a single prediction for the i th observation, we can average these predicted responses (if regression is the goal) or can take a majority vote (if classification is the goal). This leads to a single OOB prediction for the i th observation. An OOB prediction can be obtained in this way for each of the n observations, from which the

out-of-bag

³This relates to Exercise 2 of Chapter 5.

overall OOB MSE (for a regression problem) or classification error (for a classification problem) can be computed. The resulting OOB error is a valid estimate of the test error for the bagged model, since the response for each observation is predicted using only the trees that were not fit using that observation. Figure 8.8 displays the OOB error on the **Heart** data. It can be shown that with B sufficiently large, OOB error is virtually equivalent to leave-one-out cross-validation error. The OOB approach for estimating the test error is particularly convenient when performing bagging on large data sets for which cross-validation would be computationally onerous.

Variable Importance Measures

As we have discussed, bagging typically results in improved accuracy over prediction using a single tree. Unfortunately, however, it can be difficult to interpret the resulting model. Recall that one of the advantages of decision trees is the attractive and easily interpreted diagram that results, such as the one displayed in Figure 8.1. However, when we bag a large number of trees, it is no longer possible to represent the resulting statistical learning procedure using a single tree, and it is no longer clear which variables are most important to the procedure. Thus, bagging improves prediction accuracy at the expense of interpretability.

Although the collection of bagged trees is much more difficult to interpret than a single tree, one can obtain an overall summary of the importance of each predictor using the RSS (for bagging regression trees) or the Gini index (for bagging classification trees). In the case of bagging regression trees, we can record the total amount that the RSS (8.1) is decreased due to splits over a given predictor, averaged over all B trees. A large value indicates an important predictor. Similarly, in the context of bagging classification trees, we can add up the total amount that the Gini index (8.6) is decreased by splits over a given predictor, averaged over all B trees.

A graphical representation of the *variable importances* in the **Heart** data is shown in Figure 8.9. We see the mean decrease in Gini index for each variable, relative to the largest. The variables with the largest mean decrease in Gini index are **Thal**, **Ca**, and **ChestPain**.

variable
importance

8.2.2 Random Forests

Random forests provide an improvement over bagged trees by way of a small tweak that *decorrelates* the trees. As in bagging, we build a number of decision trees on bootstrapped training samples. But when building these decision trees, each time a split in a tree is considered, a *random sample of m predictors* is chosen as split candidates from the full set of p predictors. The split is allowed to use only one of those m predictors. A fresh sample of m predictors is taken at each split, and typically we choose $m \approx \sqrt{p}$ —that is, the number of predictors considered at each split is approximately equal

random
forest

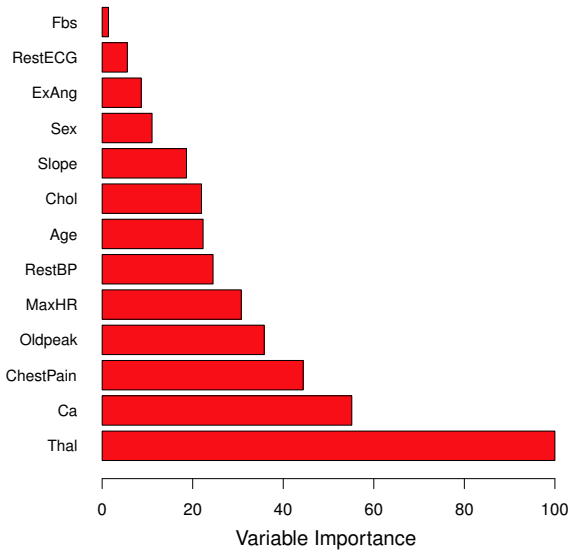


FIGURE 8.9. A variable importance plot for the **Heart** data. Variable importance is computed using the mean decrease in Gini index, and expressed relative to the maximum.

to the square root of the total number of predictors (4 out of the 13 for the **Heart** data).

In other words, in building a random forest, at each split in the tree, the algorithm is *not even allowed to consider* a majority of the available predictors. This may sound crazy, but it has a clever rationale. Suppose that there is one very strong predictor in the data set, along with a number of other moderately strong predictors. Then in the collection of bagged trees, most or all of the trees will use this strong predictor in the top split. Consequently, all of the bagged trees will look quite similar to each other. Hence the predictions from the bagged trees will be highly correlated. Unfortunately, averaging many highly correlated quantities does not lead to as large of a reduction in variance as averaging many uncorrelated quantities. In particular, this means that bagging will not lead to a substantial reduction in variance over a single tree in this setting.

Random forests overcome this problem by forcing each split to consider only a subset of the predictors. Therefore, on average $(p - m)/p$ of the splits will not even consider the strong predictor, and so other predictors will have more of a chance. We can think of this process as *decorrelating* the trees, thereby making the average of the resulting trees less variable and hence more reliable.

The Success of AdaBoost and Its Application in Portfolio Management

Yijian Chuan[‡], Chaoyi Zhao[‡], Zhenrui He[‡] and Lan Wu^{†,*}

[‡]School of Mathematical Sciences, Peking University, Beijing, China

[†]LMEQF, School of Mathematical Sciences, Peking University, Beijing, China

Abstract

We develop a novel approach to explain why AdaBoost is a successful classifier. By introducing a measure of the influence of the noise points (ION) in the training data for the binary classification problem, we prove that there is a strong connection between the ION and the test error. We further identify that the ION of AdaBoost decreases as the iteration number or the complexity of the base learners increases. We confirm that it is impossible to obtain a consistent classifier without deep trees as the base learners of AdaBoost in some complicated situations. We apply AdaBoost in portfolio management via empirical studies in the Chinese market, which corroborates our theoretical propositions.

Keywords: AdaBoost, interpolation, noise point, base learner, equal-weighted portfolio

*Corresponding author. lwu@pku.edu.cn

1 Introduction

Equal-weighted portfolios are one of the most important strategies in portfolio management. They are portfolios with weights equally distributed across the selected securities in the long and/or short positions. In academic research, numerous studies have suggested that equal-weighted portfolios have a better out-of-sample performance than other portfolios (e.g., [Jobson and Korkie 1981; James 2003; DeMiguel et al. 2007]. [Michaud (1989) and DeMiguel et al. (2007)] argued that, the equal-weighted strategies do not suffer from the estimation error of the covariance matrix, which is vulnerable to outliers ([Tu and Zhou, 2011]). In industry, equal-weighted portfolios are popular across portfolio management in practice, particularly in the hedge funds. The MSCI has issued many equal-weighted indexes, which are “some of the oldest and best-known factor strategies that have aimed to identify specific characteristics of stocks generating excess return”¹].

The core of constructing equal-weighted portfolios is to forecast the long and/or short positions, which is a classification problem. Machine learning usually outstands when dealing with classification problems, and the research of machine learning on classification is diverse. [Devroye et al. (1996)] lists numerous traditional pattern recognition research in computer science, where the pattern recognition is an alias for the classification problem. [Hastie et al. (2009)] analyzes and summaries many machine learning methods, among which include lots of classification methods, such as the Linear/Quadratic Discriminant Analysis, Support Vector Machine, Boosting, Random Forest, etc. The application of machine learning in classification succeeds in various fields, such as email spam identification, handwritten digit recognition, etc. In portfolio management, [López de Prado (2018)] explained how to apply machine learning to managing funds for some of the most demanding institutional investors. Specifically, [Creamer and Freund (2005, 2010)] applied the Boosting method in finance, and revealed its practical value. [Creamer (2012)] used LogitBoost in high-frequency data for Euro futures, and generated positive returns. [Wang et al. (2012)] invented the N-LASR (non-linear adaptive

¹<https://www.msci.com/msci-equal-weighted-indexes>

style rotation) model by applying AdaBoost in stock’s factor strategy. They wisely incorporated benefits of different factors by the N-LASR model, and the empirical study on the component stocks in Russel 3000 showed a significant risk-adjusted portfolio return. [Fiévet and Sornette \(2018\)](#) proposed a decision tree forecasting model and applied it to S&P 500, which is capable of capturing arbitrary interaction patterns and generating positive returns. [Rasekhschaffe and Jones \(2019\)](#) provided an example of the machine learning techniques to forecast the cross-section of stock returns. [Gu et al. \(2020\)](#) and [D’Hondt et al. \(2020\)](#) gave comprehensive analysis of machine learning methods for the canonical problem of empirical asset pricing, attributing their predicted gains to the non-linearity.

AdaBoost is a classification method in machine learning, inspiring a tremendous amount of innovations. AdaBoost has developed for more than two decades since [Freund and Schapire \(1996\)](#). Since it is less prone to overfit, Breiman praised AdaBoost—“the best off-the-shelf classifier in the world”—at the 1996 NeurIPS conference ([Friedman et al., 2000](#)). AdaBoost has made a significant impact on machine learning and statistics. To explain AdaBoost’s overfitting resistance, [Schapire et al. \(1998\)](#) proposed the “margin” theory. Meanwhile, [Breiman \(1998, 1999\)](#) and [Friedman et al. \(2000\)](#) discovered the fact that AdaBoost is equivalent to an optimization algorithm, so [Friedman \(2001\)](#) put forward the Gradient Boosting. Inspired by AdaBoost, [Breiman \(2001\)](#) invented the Random Forest(RF) and believed that there are some similarities between RF and AdaBoost. Subsequently, people generalized the Boosting methods, in which two Boosting algorithms are widely-adopted—the XGBoost ([Chen and Guestrin, 2016](#)) and the LightGBM ([Ke et al., 2017](#)). Till now, the family of Boosting flourishes, and becomes a considerable part in machine learning.

Although there are many studies explaining why AdaBoost is a successful method, people are still curious about its excellent achievement till now. [Wyner \(2003\)](#); [Mease and Wyner \(2008\)](#) believed that the available interpretation of AdaBoost is “incomplete”, particularly on the explanation of its overfitting resistance property. [Wyner et al. \(2017\)](#), p. 10) introduced a novel perspective on AdaBoost and RF, and conjectured that their success could be explained

by the “spiked-smooth”, where *spiked* is “in the sense that it is allowed to adapt in very small regions to noise points”, and *smooth* is “in the sense that it has been produced through averaging”. In other words, AdaBoost is a self-averaging interpolating method, localizing the effect of the noise points as the iteration number increases.

Our work is motivated by the questions from the industry: “May machine learning strategies outperform other traditional strategies in quantitative investment? Why and how do they work?” [López de Prado \(2018\)](#) gave a comprehensive and systematic approach to apply machine learning methods, and highly appraised Boosting: “We explored a number of standard black-box approaches. Among machine learning methods, we found gradient tree boosting to be more effective than others.” Besides, [Wang et al. \(2012\)](#) applied AdaBoost to select and combine factors with consistent and interpretative performance, and [Zhang et al. \(2019\)](#) proposed a Boosting method to compose portfolios which performs well. These findings answered the first question. There is limited research concerning the mechanism or the interpretability of machine learning in portfolio management. However, interpretability is essential in investment ([Feng et al., 2020](#)). In detail, [Harvey et al. \(2016\)](#), p. 37) argued: “... a factor developed from first [economic] principles should have a lower threshold t-statistic than a factor that is discovered as a purely empirical exercise.” [Harvey \(2017\)](#) proposed an example. They constructed portfolios based on the first, second, and third letters of the ticker symbols, gaining significant excess returns. Nevertheless, most people would not like to adopt this symbol-based portfolio, as they implied. Thus, without interpretability, portfolio investment is vulnerable. We must pay more attention to the second/third questions.

To answer the “why and how” questions, we should investigate AdaBoost in the framework of statistics to find a theoretical explanation of its outperformance and apply them in portfolio management. [Wyner et al. \(2017\)](#) pointed out that: “The statistics community was especially confounded by two properties of AdaBoost: 1) interpolation (perfect prediction in sample) was achieved after relatively few iterations, 2) generalization error continues to drop even after interpolation is achieved and maintained.” They innovated the concept

of “spiked-smooth” classifier created by a self-averaging and interpolating algorithm. They conjectured that the “spiked-smooth” property renders the success of AdaBoost, and provided many delicate examples by simulation to support their viewpoints. Thus, we would like to narrow the gap between the theory and the simulation by strengthening their work from a statistical perspective. To explain the “spiked-smooth” mathematically, we need to distinguish the signal and the noise within the training set in a statistical framework first. Then, one should connect the relationship between the “spiked-smooth” and the test error, explaining the property of overfitting resistance.

In addition, [Wyner et al. \(2017\)](#) pointed out that “boosting should be used like random forests: with large decision trees, without regularization or early stopping”. The point is that larger and deeper decision trees are preferred to be used as the “weak” classifiers (base learners) of AdaBoost, since they can both “interpolate” the training set and realize the goal of “spiked-smooth”. This point contradicts with the common sense about machine learning and statistics, and statisticians usually believe complexity leads to overfitting. Therefore, we wonder if the AdaBoost method can boost shallow trees when the true model is very complicated, just as we cannot “make bricks without straw”. We try to find out that under what populations will the AdaBoost method be unable to achieve good performance if the base learners are very weak. We want to demonstrate the viewpoints of [Wyner et al. \(2017\)](#) in a mathematical framework.

In this paper, we show that how AdaBoost can dig out more non-linear information in the training set without increasing the test error. Our work is composed of three parts. First, to concrete the abstract concept “spiked-smooth” into a measurable value, we define a measure of the influence of the noise points in the training set for a given method. The measure can also be regarded as a measure of the localization of the given method. We discover the connection between the measure and the out-of-sample performance. That is, under certain conditions, if the influence of the noise points is not essential, then the test error will be low. A toy example clarifies the theorem, intuitively illustrating the influence

of the noise and explaining why it controls the test error. For AdaBoost, we show that, as the number of iterations increases or the depth of the base learners grows, it becomes more robust to the influence of the noise, and thus lead to a lower test error. Therefore, we give a theoretical explanation about why AdaBoost has a good performance without overfitting in noisy training sets.

Second, we confirm that it is a better choice to use deeper/larger decision trees as base learners of AdaBoost in the sense of digging out complex information. Specifically, we propose several counterexamples that AdaBoost based on shallow decision trees fails to handle, even after iterating infinite times. We generalize the results and indicate that AdaBoost based on shallow decision trees would fail in recognizing a certain kind of information, while the one based on deep decision trees could easily solve out. Therefore, these findings suggest that AdaBoost based on deep decision trees maybe better.

Third, the empirical studies in the Chinese market corroborate our theoretical propositions. The theoretical results about the interpolation and the localization of AdaBoost in the previous parts of this paper is verified by constructing an optimal portfolio strategy. Besides, the result also illustrates the good performance of the equal-weighted portfolio generated by the selected optimal classifier trained by AdaBoost.

The outline of this paper is as follows. Section 2 introduces a measure of “spiked-smooth”, illustrates the relationship between the measure and the test error, and explains the success of AdaBoost. Section 3 identifies that AdaBoost based on deep trees can dig out more information, while the one based on shallow trees fails. Section 4 provides empirical studies of AdaBoost in the Chinese stock market. Section 5 concludes.

2 The influence of the noise points and AdaBoost

In this section, we give a strict definition for the “spiked-smooth” suggested by [Wyner et al. \(2017\)](#) in the framework of the Bayes classifier. Under the framework, we explain the success

of AdaBoost by developing a concrete measure.

First, we describe a background model of the binary classification problem and the Bayes classifier, and define the signal/noise points for a given training set. Based on these concepts, we build a bridge between the Bayes classifier and the interpolating classifier. We define a measure of the influence of the noise points, and specify its property. Second, we explore the connection between the measure and the test error. Last, we explain the success of AdaBoost as the minimization of the influence of the noise points in the sense of the “spiked-smooth”, and reveal its potential applications in portfolio management.

2.1 The model of the binary classification

A prediction model consists of an input vector $X \in \mathcal{X}$, an output $Y \in \mathcal{Y} = \{\pm 1\}$, and a prediction classifier $f : \mathcal{X} \rightarrow \mathcal{Y}$. For simplicity, let us assume that the distribution of X is absolutely continuous with respect to the Lebesgue measure. We restate the definition of the Bayes classifier/error ([Devroye et al., 1996], p. 2) in Definition 1 below.

Definition 1 (Bayes Classifier/Error). Given the population X and $Y \in \mathcal{Y} = \{\pm 1\}$, the Bayes classifier is

$$f^B := \operatorname{argmin}_{f: \mathcal{X} \rightarrow \mathcal{Y}} \mathbb{P}\{f(X) \neq Y\},$$

and the minimum is the Bayes error q , i.e.,

$$q := \min_{f: \mathcal{X} \rightarrow \mathcal{Y}} \mathbb{P}\{f(X) \neq Y\}.$$

According to the definition, f^B is the classifier minimizing the test error, which can be represented by the conditional distribution $\mathbb{P}_{Y|X}$ in the population. One can show $f^B(\mathbf{x}) = \operatorname{sign}(\mathbb{P}\{Y = 1|X = \mathbf{x}\} - 0.5)$, when the population satisfies certain canonical conditions. There is no classifier having lower test error than f^B . We give a general representation of

the test error of a classifier by the Bayes classifier in Lemma 1 while the noise and X are independent.

Lemma 1. *Given the population (X, Y) , the Bayes classifier f^B , and the Bayes error q , if $\mathbf{1}_{\{Y \neq f^B(X)\}}$ and X are independent, then, for any classifier f ,*

$$\mathbb{P}\{f(X) \neq Y\} = q\mathbb{P}_X\{f(X) = f^B(X)\} + (1 - q)\mathbb{P}_X\{f(X) \neq f^B(X)\}. \quad (1)$$

Proof. We have

$$\begin{aligned} & \mathbb{P}\{f(X) \neq Y\} \\ &= \mathbb{P}\{f(X) \neq Y, f(X) = f^B(X)\} + \mathbb{P}\{f(X) \neq Y, f(X) \neq f^B(X)\} \\ &= \mathbb{P}\{f^B(X) \neq Y, f(X) = f^B(X)\} + \mathbb{P}\{f^B(X) = Y, f(X) \neq f^B(X)\} \\ &= \mathbb{P}\{f^B(X) \neq Y\}\mathbb{P}_X\{f(X) = f^B(X)\} + \mathbb{P}\{f^B(X) = Y\}\mathbb{P}_X\{f(X) \neq f^B(X)\} \\ &= q\mathbb{P}_X\{f(X) = f^B(X)\} + (1 - q)\mathbb{P}_X\{f(X) \neq f^B(X)\}. \end{aligned}$$

□

A natural corollary of (1) is $\mathbb{P}\{f(X) \neq Y\} = q + (1 - 2q)\mathbb{P}_X\{f(X) \neq f^B(X)\}$. In other words, $\mathbb{P}\{f(X) \neq Y\}$ is a linear function of $\mathbb{P}_X\{f(X) \neq f^B(X)\}$.

Next, we introduce the concept of the signal/noise points of the training set T in the following Definition 2.

Definition 2 (Signal/Noise Points). Given a training set $T = (\mathbf{x}_i, y_i)_{i=1}^n$ generated from the population (X, Y) and the Bayes classifier f^B , a point $(\mathbf{x}_i, y_i)$ is a signal point, if $f^B(\mathbf{x}_i) = y_i$; and it is a noise point, if $f^B(\mathbf{x}_i) \neq y_i$.

In short, the Bayes classifier f^B distinguishes the signal/noise points of a training set. Heuristically, the signal points are the points that equal to the output of the Bayes classifier, while the noise points are not.

We recall the definition of the interpolation classifier proposed by [Wyner et al. \(2017\)](#) for coherence.

Definition 3 (Interpolating Classifier). A classifier f is an interpolating classifier on the training set $T = (\mathbf{x}_i, y_i)_{i=1}^n$, if $f(\mathbf{x}_i) = y_i$ for all $(\mathbf{x}_i, y_i), i = 1, \dots, n$.

Immediately, we can obtain a property of the interpolating classifier, i.e., its training error is 0 on the training set $T = (\mathbf{x}_i, y_i)_{i=1}^n$.

Though the Bayes classifier is the best classifier in the sense of minimizing the test error, it does not necessarily interpolate the given training set $T = (\mathbf{x}_i, y_i)_{i=1}^n$. The Bayes classifier f^B violates interpolation at and only at the noise points, as implied in Definition [3](#). Thus, in view of the training set, the difference between an interpolating classifier and the Bayes classifier is only on the noise points. So, we propose a definition of a purified training set of T by converting the noise points into the “signal” points.

Definition 4 (Purified Training Set). Given a training set $T = (\mathbf{x}_i, y_i)_{i=1}^n$ from the population (X, Y) and the Bayes classifier f^B , the purified training set of T is defined as $T_p := (\mathbf{x}_i, f^B(\mathbf{x}_i))_{i=1}^n$.

There is no noise point in T_p . In other words, the Bayes classifier must interpolate the purified training set T_p . We can also rewrite the definition of the purified training set as

$$T_p = (\mathbf{x}_i, \theta_i)_{i=1}^n, \quad \theta_i = \begin{cases} y_i, & i \text{ is a signal point;} \\ -y_i, & i \text{ is a noise point.} \end{cases}$$

The two training sets T and T_p share the same input $\mathbf{x}_i$ ’s but different outputs, and the difference between the outputs of the two sets is only on the noise points. The purpose is to separate out the influence of the noise points from the whole information contained in the training set T .

Last, based on the previous preparations, we propose a measure of the influence of the noise points (ION) for a given training set T and a given method $\mathcal{M}$. It helps us to compare

the properties of different methods, such as one nearest neighbor (1NN) or AdaBoost, on a given training set.

Definition 5 (ION). Given the marginal probability measure of X ($\mathbb{P}_X$), we define the influence of noise (ION), a function of the training set T and the method $\mathcal{M}$:

$$\text{ION}(\mathcal{M}, T) := \mathbb{P}_X \{f_T(X) \neq f_{T_p}(X)\}, \quad (2)$$

where f_T is the classifier trained on the training set T using method $\mathcal{M}$, and f_{T_p} is on the purified training set T_p using method $\mathcal{M}$.

We interpret Definition 5. $\mathcal{M}$ represents a specific method. For instance, for 1NN, one can apply it on the training set T and T_p , which generates two classifiers f_T and f_{T_p} . Then, by comparing the two classifiers, we can get the value of $\text{ION}(1\text{NN}, T)$. The ION is defined according to two sets: the training set T and the proxy of the training set generated by f^B .

Although Definition 5 does not require the interpolation of the classifier f , ION usually characterizes the performance of the method which generates interpolating classifiers on a given training set. Meanwhile, $0 \leq \text{ION} \leq 1$. If ION is low, then the classifier is robust to the noise points on the training set of the given method, and vice versa.

Interpolation is not necessarily bad, if it subjects to some “mechanism” (Wyner et al., 2017). Although some interpolating classifiers “can be shown to be inconsistent and have poor generalization error in environments with noise”, “the claim that all interpolating classifiers overfit is problematic”. The classifiers generated by 1NN or random forest are both interpolating classifiers, but their ION may not be the same. Furthermore, Wyner et al. (2017) suggested: “an interpolated classifier, if sufficiently local, minimizes the influence of noise points in other parts of the data.” The next question is, what is the relationship between the ION and the so-called “spiked-smooth” classifier.

2.2 The ION and the test error

In this section, we reveal the connection between the ION and the test error from theoretical and numerical perspectives.

First, we prove that, under certain conditions, the lower the ION, the lower the test error.

Proposition 1. *Given the population (X, Y) such that $\mathbf{1}\{Y \neq f^B(X)\}$ is independent of X , let $f_T^{(1)}$ and $f_T^{(2)}$ denote the classifiers generated from two different methods $\mathcal{M}^{(1)}$ and $\mathcal{M}^{(2)}$ on the training set T , and $f_{T_p}^{(1)}$ and $f_{T_p}^{(2)}$ denote the ones generated from $\mathcal{M}^{(1)}$ and $\mathcal{M}^{(2)}$ on the purified training set T_p , respectively. If*

$$f_{T_p}^{(1)}(x) = f_{T_p}^{(2)}(x) = f^B(x), \quad a.s., \quad (3)$$

and

$$\text{ION}(\mathcal{M}^{(1)}, T) < \text{ION}(\mathcal{M}^{(2)}, T),$$

then

$$\mathbb{P} \left\{ f_T^{(1)}(X) \neq Y \right\} < \mathbb{P} \left\{ f_T^{(2)}(X) \neq Y \right\}. \quad (4)$$

Proof. Because of (3), we have

$$\begin{aligned} \mathbb{P}_X \left\{ f_T^{(1)}(X) \neq f^B(X) \right\} &= \mathbb{P}_X \left\{ f_T^{(1)}(X) \neq f_{T_p}^{(1)}(X) \right\} = \text{ION}(\mathcal{M}^{(1)}, T) \\ &< \text{ION}(\mathcal{M}^{(2)}, T) = \mathbb{P}_X \left\{ f_T^{(2)}(X) \neq f_{T_p}^{(2)}(X) \right\} = \mathbb{P}_X \left\{ f_T^{(2)}(X) \neq f^B(X) \right\}. \end{aligned}$$

Thus, by Lemma 1, (4) holds. □

Proposition 1 shows that the ION controls the test error. Specifically, it means that, if the two methods could reach the Bayes classifier in the purified training set, then the method

with lower ION outperforms the others in the sense of the test error. For instance, $\mathcal{M}^{(1)}$ might indicate 1NN, while $\mathcal{M}^{(2)}$ indicate AdaBoost.

However, the condition (3) is slightly unnatural. It is so strong that it could only hold in several particular training sets. We therefore weaken the original condition (3) and establish a more natural condition in Theorem 1 below.

Theorem 1. *Given the population (X, Y) such that $\mathbf{1}_{\{Y \neq f^B(X)\}}$ is independent of X , let $f_T^{(1)}$ and $f_T^{(2)}$ denote the classifiers generated from two different methods $\mathcal{M}^{(1)}$ and $\mathcal{M}^{(2)}$ on the training set T , and $f_{T_p}^{(1)}$ and $f_{T_p}^{(2)}$ denote the ones generated from $\mathcal{M}^{(1)}$ and $\mathcal{M}^{(2)}$ on the purified training set T_p , respectively. The size of the training set T or T_p is n . If*

$$\lim_{n \rightarrow \infty} \mathbb{P} \left\{ f_{T_p}^{(j)}(X) \neq Y \right\} = q, \quad j = 1, 2, \quad (5)$$

and

$$\limsup_{n \rightarrow \infty} \left[\text{ION}(\mathcal{M}^{(1)}, T) - \text{ION}(\mathcal{M}^{(2)}, T) \right] < 0, \quad (6)$$

then

$$\limsup_{n \rightarrow \infty} \left[\mathbb{P} \left\{ f_T^{(1)}(X) \neq Y \right\} - \mathbb{P} \left\{ f_T^{(2)}(X) \neq Y \right\} \right] < 0. \quad (7)$$

Proof. To begin with, by Lemma 1, (5) is equivalent to

$$\lim_{n \rightarrow \infty} \mathbb{P}_X \left\{ f_{T_p}^{(j)}(X) \neq f^B(X) \right\} = 0, \quad j = 1, 2, \quad (8)$$

Then,

$$\begin{aligned}
 & \mathbb{P}_X \left\{ f_T^{(1)}(X) \neq f^B(X) \right\} - \mathbb{P}_X \left\{ f_T^{(2)}(X) \neq f^B(X) \right\} \\
 & - \left(\mathbb{P}_X \left\{ f_T^{(2)}(X) \neq f^B(X), f_T^{(2)}(X) \neq f_{T_p}^{(2)}(X) \right\} + \mathbb{P}_X \left\{ f_T^{(2)}(X) \neq f^B(X), f_T^{(2)}(X) = f_{T_p}^{(2)}(X) \right\} \right) \\
 & = \mathbb{P}_X \left\{ f_T^{(1)}(X) \neq f^B(X), f_T^{(1)}(X) \neq f_{T_p}^{(1)}(X) \right\} - \mathbb{P}_X \left\{ f_T^{(2)}(X) \neq f^B(X), f_T^{(2)}(X) \neq f_{T_p}^{(2)}(X) \right\} \\
 & + \mathbb{P}_X \left\{ f_T^{(1)}(X) \neq f^B(X), f_T^{(1)}(X) = f_{T_p}^{(1)}(X) \right\} - \mathbb{P}_X \left\{ f_T^{(2)}(X) \neq f^B(X), f_T^{(2)}(X) = f_{T_p}^{(2)}(X) \right\} \\
 & = \mathbb{P}_X \left\{ f_{T_p}^{(1)}(X) = f^B(X), f_T^{(1)}(X) \neq f_{T_p}^{(1)}(X) \right\} - \mathbb{P}_X \left\{ f_{T_p}^{(2)}(X) = f^B(X), f_T^{(2)}(X) \neq f_{T_p}^{(2)}(X) \right\} \\
 & + \mathbb{P}_X \left\{ f_{T_p}^{(1)}(X) \neq f^B(X), f_T^{(1)}(X) = f_{T_p}^{(1)}(X) \right\} - \mathbb{P}_X \left\{ f_{T_p}^{(2)}(X) \neq f^B(X), f_T^{(2)}(X) = f_{T_p}^{(2)}(X) \right\} \\
 & =: A_n^{(1)} - A_n^{(2)} + B_n^{(1)} - B_n^{(2)}.
 \end{aligned}$$

By [8], we have

$$\lim_{n \rightarrow \infty} \mathbb{P}_X \left\{ f_{T_p}^{(j)}(X) \neq f^B(X), f_T^{(j)}(X) = f_{T_p}^{(j)}(X) \right\} = 0, \quad j = 1, 2,$$

and thus $\lim_{n \rightarrow \infty} B_n^{(j)} = 0, j = 1, 2$. By [8], we also have

$$\lim_{n \rightarrow \infty} \left(A_n^{(j)} - \mathbb{P}_X \left\{ f_T^{(j)}(X) \neq f_{T_p}^{(j)}(X) \right\} \right) = 0, \quad j = 1, 2,$$

so, by [2], one can show that $A_n^{(j)}$ and $\text{ION}(\mathcal{M}^{(j)}, T)$ share the same limit, $j = 1, 2$. Therefore,

$$\lim_{n \rightarrow \infty} \left[\left(\mathbb{P}_X \left\{ f_T^{(1)}(X) \neq f^B(X) \right\} - \mathbb{P}_X \left\{ f_T^{(2)}(X) \neq f^B(X) \right\} \right) - \left(\text{ION}(\mathcal{M}^{(1)}, T) - \text{ION}(\mathcal{M}^{(2)}, T) \right) \right] = 0.$$

Because of [6], we have

$$\limsup_{n \rightarrow \infty} \left[\mathbb{P}_X \left\{ f_T^{(1)}(X) \neq f^B(X) \right\} - \mathbb{P}_X \left\{ f_T^{(2)}(X) \neq f^B(X) \right\} \right] < 0.$$

Further, by Lemma [1], [7] holds. □

We interpret Theorem 1. First, (5) is a very weak condition. It assumes the two methods are consistent on the purified training set T_p . In fact, many classical methods have been proved to be consistent. Furthermore, because there is no noise point in T_p , the consistency on T_p is easier to achieve than on T . Even the notoriously easy-to-overfit method, 1NN, is consistent in such a good training set T_p but not necessarily consistent in T , according to the Cover-Hart inequality (Cover and Hart, 1967).

Second, (6) is about the property of some methods regarding a certain training set. Instead of subjectively describing the property of the methods, it measures the influence of the noise points in the particular training set objectively.

Third, according to Theorem 1, the decrease of ION implies that the method is minimizing the influence of the noise points and thus enhancing the generalization ability. It means that, for most methods, the ION is a good indicator of the test error.

Fourth, there is no natural conflict between interpolation and lower ION. For a classifier, the purpose of interpolating is to take all information contained in the signal points as much as possible, while the goal of lowering ION is to reduce the impact attributed to the noise points.

In order to have a concrete understanding of Proposition 1 and Theorem 1, we give a 2-dim toy example. First, the population is

$$\mathbb{P}\{Y = 1|X = \mathbf{x}\} = \begin{cases} 0.1, & \text{if } x_1 < 0, \\ 0.9, & \text{if } x_1 \geq 0, \end{cases}$$

where X is uniformly distributed in $(-1, 1]^2$. In other words, only the first dimension of X is relevant to Y , while the second dimension contributes no information. One can easily solve the Bayes classifier $f^B(\mathbf{x}) = \text{sign}\{x_1\}$ and the Bayes error $q = 0.1$.

Second, we randomly generate a training set T with a size $n = 500$, as in Fig. 1. The training set T is composed of 460 signal points and 40 noise points. Roughly speaking, the

yellow triangles to the left and the blue circles to the right are all noise points. Particularly, on the left and bottom side of the graph in Fig. 1, there is a solid triangle $\mathbf{x}_{i_0} = (-0.79, -0.41)$, which is the noise point that would be discussed later.

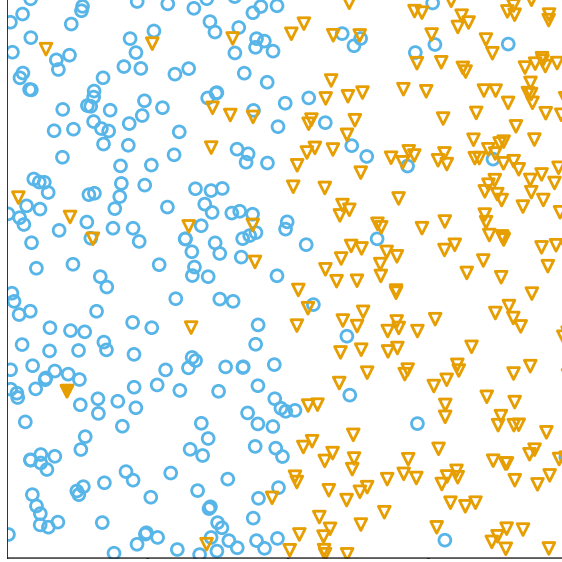


Fig. 1. The training set T .

Third, we apply two methods, $\mathcal{M}^{(1)}$ (1NN) and $\mathcal{M}^{(2)}$ (AdaBoost²), to generate interpolating classifiers on the training set T respectively. Fig. 2(a) is the classifier f_T^{1NN} generated by method 1NN, while 2(b) f_T^{AdaBoost} . The purple vertical dotted line ($x_1 = 0$) is the watershed of f^B , while the black solid lines are the decision boundaries of $f_T^{\mathcal{M}^{(\cdot)}}$. Both classifiers are interpolating, which means that $f_T^{\mathcal{M}^{(\cdot)}}(\mathbf{x}_i) = y_i, \forall i$, even though they are from different methods.

Fourth, we argue that the ION of AdaBoost is lower than that of 1NN on the training set T . the classifiers in Fig. 2 are different: The decision boundary of 1NN in Fig. 2(a) is smooth and natural, while that of AdaBoost in Fig. 2(b) is sharp and uneven. However, we argue that the sharp and uneven is better than the smooth and natural in the sense of minimizing and localizing the influence of the noise points. For the isolated noise points, the regions surround them in 2(b) are smaller and narrower than that in 2(a). In detail,

²To be more specific, for the AdaBoost we used, the base learners are decision trees with a maximum depth 4, and the number of iterations is 50.

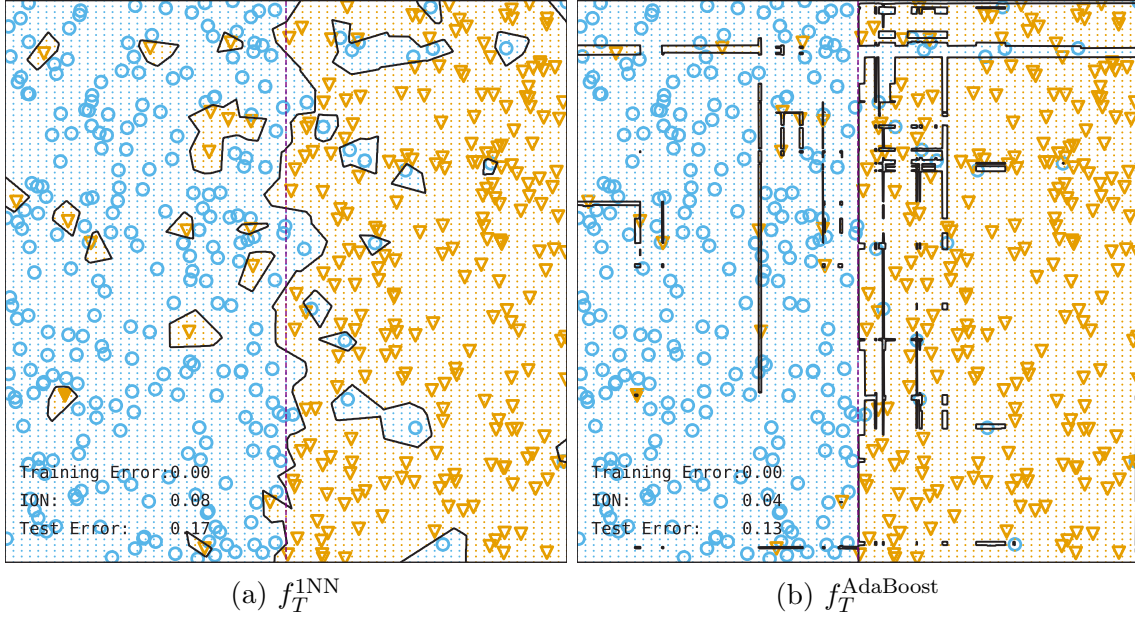


Fig. 2. The training set T and the classifiers: AdaBoost has lower ION than 1NN.

we focus on the particular noise point lies at about $\mathbf{x}_{i_0} = (-0.79, -0.41)$, which is the solid triangle. The area around $\mathbf{x}_{i_0}$ of $f_T^{AdaBoost}$ in 2(b) is very small, while that of f_T^{1NN} in 2(a) is a big irregular polygon—the influence of the noise point $\mathbf{x}_{i_0}$ seems to be lower for AdaBoost than 1NN³

Last, we calculate the ION and the test error, summarized in Table 1, where $\text{ION}(1\text{NN}, T) \approx 0.08$ and $\text{ION}(\text{AdaBoost}, T) \approx 0.04$, $\mathbb{P}\{f_T^{1NN}(X) \neq Y\} \approx 0.17$ and $\mathbb{P}\{f_T^{AdaBoost}(X) \neq Y\} \approx 0.13$. We can observe that the results are in line with our theorem, i.e., the lower the ION, the lower the test error.

Table 1: The ION and the test error.

Method	ION	Training Error	Test Error
f^B	-	-	0.10
$\mathcal{M}^{(1)} = 1\text{NN}$	0.08	0	0.17
$\mathcal{M}^{(2)} = \text{AdaBoost}$	0.04	0	0.13

Overall, this section connects the ION and the test error. Both the theoretical derivation and the toy example of simulation demonstrate the importance of ION. Particularly, the toy

³It is noteworthy that the influence of the noise points are acting jointly rather than individually, but it does not matter in this heuristic case.

example explains why 1NN is easy to overfit, while AdaBoost not. However, AdaBoost is only a general term for a class of methods, since both the base learners and the number of iterations need to be specified. By choosing different kinds of base learners and different numbers of iterations, we can generate a tremendous amount of specific methods. In Section 2.3, we take a close look at the performance of AdaBoost with different hyperparameters from our new perspective: ION.

2.3 The ION and AdaBoost

AdaBoost mainly has two hyperparameters. One of them is the complexity of the base learners. The decision trees are one of the most popular base learners of AdaBoost, which is the classical base learner in the monograph [Hastie et al. \(2009\)](#). In this paper, we use the maximum depth of decision trees to indicate the complexity of the base learners. The deeper the decision trees, the more complex the base learners, and the more complex the AdaBoost. The other is the number of iterations, which is the number of the base learners added in total. The higher the number of iterations, the more complex the AdaBoost.

This section corroborates the conclusion of [Wyner et al. \(2017\)](#) with our newly defined concept ION. We show their conclusion that AdaBoost based on large decision trees without early stopping is better. We want to show that AdaBoost generates interpolating classifiers, and both the ION and the test error decrease as the depth of the base learners and the number of iterations increase. Instead of comparing AdaBoost with 1NN, we digest AdaBoost itself with different parameters in details via high-dimensional population of simulation.

The simulation population is

$$\mathbb{P}\{Y = 1|X = \mathbf{x}\} = \begin{cases} 0.1, & \text{if } x_1 \cdot x_2 \cdot x_3 < 0, \\ 0.9, & \text{if } x_1 \cdot x_2 \cdot x_3 \geq 0, \end{cases}$$

where X is uniformly distributed in $(-1, 1]^6$. We randomly generate a training set T with

$n = 500$, and compare the results of AdaBoost with different hyperparameters.

In order to explain the reason why AdaBoost without early stopping might be better, we compare the results of AdaBoost with different numbers of iterations $m = 1, 2, \dots, 250$ but the same maximum depth of the base decision trees. The maximum depth is set as 5. We denote the corresponding classifiers by $f_T^{(m)}$, $m = 1, 2, \dots, 250$.

The results are in Fig. 3. The x -axis in the figure represents the number of iterations m . We clarify the three lines in detail. The green dashed line is the training error of $f_T^{(m)}$ on its training set T , the red dashed-dotted line is the test error of $f_T^{(m)}$, and the blue solid line is the ION of AdaBoost in the training set: $\text{ION}(\mathcal{M}^{(m)}, T)$. All the three lines decrease sharply when $n < 20$. When $n \geq 20$, the training error remains 0, but the test error and ION keep decreasing.

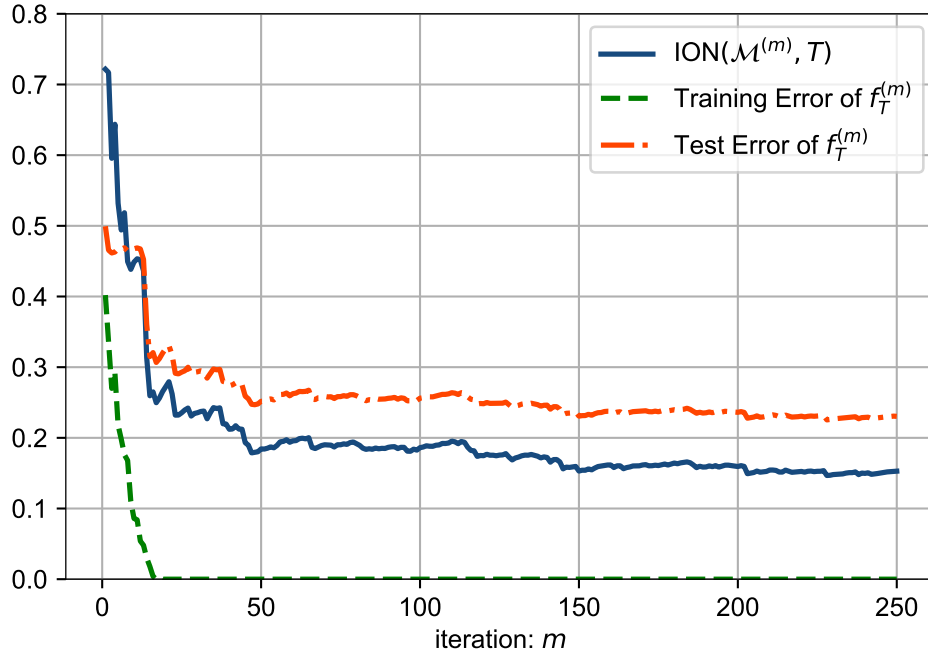


Fig. 3. The performance of AdaBoost regarding to $m = 1, 2, \dots, 250$.

From Fig. 3, we have the following observations. First, AdaBoost is minimizing the influence of the noise points. When $m \geq 20$, the test error decreases but the training error remains 0. A natural question is, what is AdaBoost doing? There are many explanations.

Wyner et al. (2017) believed that AdaBoost is self-averaging and generating a “spiked-smooth” classifier by minimizing the the influence of the noise points. We corroborate their work with the blue solid line ION. When $m \geq 20$, although the training error remains 0, the ION continues to decrease, which reflects the decrease of the influence of the noise points. Thus, as the number of iterations increases, AdaBoost keeps interpolating, and simultaneously minimizes the influence of the noise points. Second, the iteration of AdaBoost can be divided into two stages: The first stage is the sharp decrease of the training error ($m < 20$), and the second stage is the decrease of ION ($m \geq 20$). The first stage can be considered as the formation of the rough skeleton of the classifier, while the second stage can be treated as the process of the details with the minimization of the influence of the noise points.⁴

For another, we show that AdaBoost based on deep/large decision trees is better, and explain it by ION. Specifically, we apply AdaBoost based on different decision trees but the same number of iterations $m = 250$, where the base decision trees have different maximum depths from 1 to 8. In other words, the number of the terminal leaves of the base decision trees varies from 2 to 256. We denote the corresponding classifiers by $f_T^{(j)}, j = 1, 2, \dots, 8$.

The results are presented in Fig. 4. The x -axis in the figure represents the maximum depth, i.e., j for $f_T^{(j)}$. The three lines are the same as those in Fig. 3, and so are the interpretations.

Overall, AdaBoost based on large decision trees without early stopping is better, which can be explained as the decrease of the ION. Given the condition that the training error is 0, the influence of the noise points decreases as the depth of the base decision trees and the number of iterations increase.

Now, return to the main line of the paper, we show that AdaBoost would not overfit even interpolating, when digging out complex structures of factors in constructing equal-weighted portfolios. As it was emphasized by López de Prado (2018, p. 15), the linear

⁴However, the two stages can not be divided arbitrarily, because ION may also play a role in the first stage.

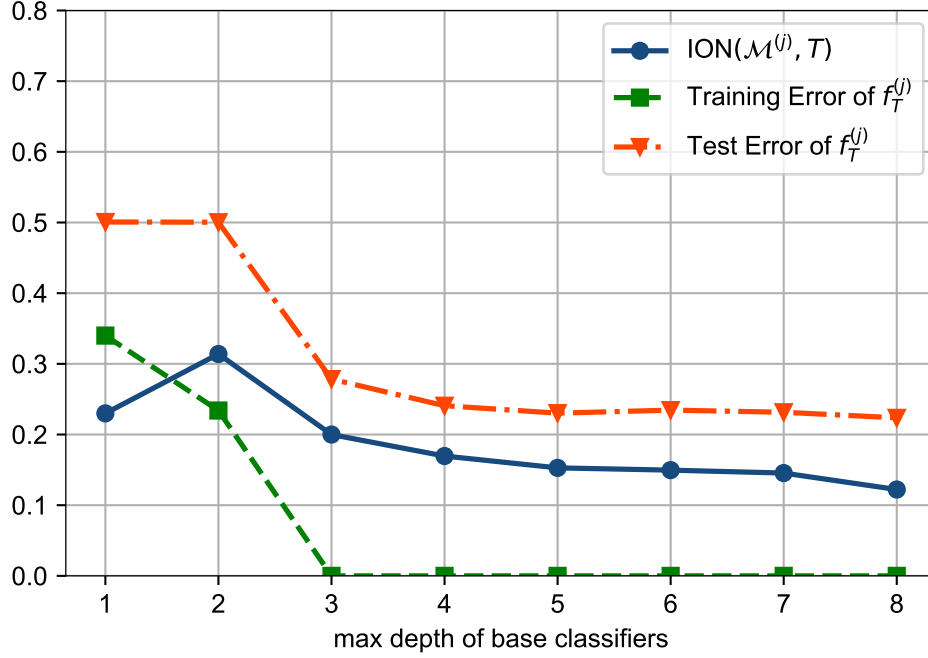


Fig. 4. The performance of AdaBoost regarding to $j = 1, 2, \dots, 8$.

methods are awfully simplistic and useless, and would “fail to recognize the complexity of the data”. The academia and industry shift their focus to the non-linear ones. There are a tremendous amount of machine learning methods applied in various data and fields in finance. Many of the machine learning methods suffer from the overfitting and the low interpretation. However, AdaBoost is not heavily affected by them as illustrated above.

In Section 4, we give empirical studies about specific factors or strategies, and prove the advantage of AdaBoost in portfolio management. But we want to clarify what kind of non-linear information can AdaBoost dig out first.

3 Base learners of AdaBoost

AdaBoost is a boosting method. It boosts the performance of a series of base learners, or “weak classifiers”. People usually choose shallow trees (such as “stumps”, i.e., decision trees with only one layer) as base learners since they are “weak” enough and thus can avoid

overfitting.

However, in many fields, especially in the area of portfolio management, using stumps as base learners may not capture the nature of the population, since the population is usually rather complicated. [Wyner et al. \(2017\)](#) proposed that the deep and large trees will allow the base learners to interpolate the data without overfitting, and it is a better choice to use deep trees as base learners. We have already shown the result mathematically in Section [2.3](#) from the perspective of the ION.

In this section, we discuss the shortcomings of AdaBoost based on stumps. We first show that stumps cannot deal with the “XOR” classification problem. Then, we generalize the result and demonstrate that AdaBoost based on stumps cannot deal with populations without “comonotonicity”. These kinds of populations are common in finance, since the investment activities in the financial world are usually rather complicated and interactive.

3.1 The “XOR” population

In this section, we use a toy example to show that the shallow trees (especially stumps) are not always capable to capture the patterns of the population. We introduce the Boolean operator “exclusive OR” (XOR) first.

Definition 6 (2-XOR). The 2-XOR function is defined as $\text{XOR}_2 : \mathbb{R}^2 \rightarrow \{\pm 1\}$ such that

$$\text{XOR}_2(x_1, x_2) = \begin{cases} -1, & \text{if } x_1 \cdot x_2 \geq 0, \\ 1, & \text{if } x_1 \cdot x_2 < 0. \end{cases}$$

Definition 7 (k -XOR). For $k > 2$, the k -XOR function, denoted as XOR_k , is defined recursively as

$$\text{XOR}_k(x_1, x_2, \dots, x_k) = \text{XOR}_2(\text{XOR}_{k-1}(x_1, x_2, \dots, x_{k-1}), x_k).$$

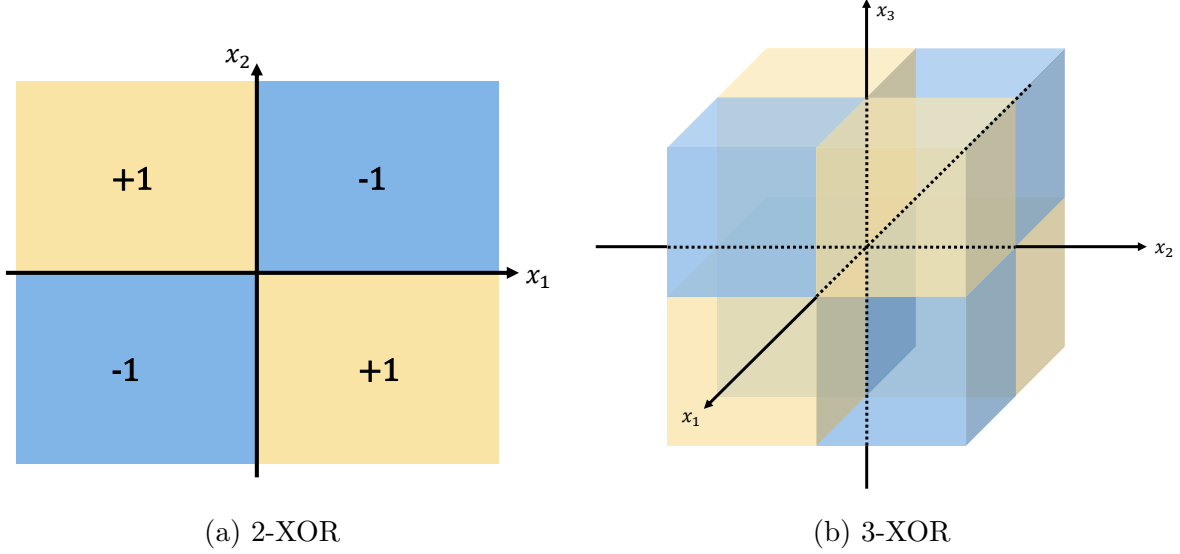


Fig. 5. Intuitions of 2-XOR and 3-XOR functions.

The Boolean operator k -XOR is an important function in computer science (for instance, the parity check), and it is also a classical example in [Hastie et al. \(2009\)](#). It can also bring insights into portfolio management, since the k -XOR can characterize the interaction among different factors. There are many studies focusing on the interaction among various factors ([Asness et al., 2018](#)). Fig. 5 are intuitive illustrations of the 2-XOR and the 3-XOR functions. The outputs are not the same in adjacent quadrants (or octants), which is a common pattern of interaction.

Now we show that the stumps cannot deal with the classification problems with the Bayes classifier $f^B = \text{XOR}_k$, even in the case that the Bayes error is 0. For instance, if we use a stump classifier f to classify the 2-XOR function, we can easily show that $\mathbb{P}_{x_1, x_2} \{f(x_1, x_2) \neq \text{XOR}_2(x_1, x_2)\}$ is always 50% no matter how the stump is trained. That is because, a stump is equivalent to a partition of $\mathbb{R}^2$ along the direction of one axis. After the partition, both half-spaces still contain values of 1 (accounts for 50%) and -1 (50%), which leads to a test error 50%.

The conclusion can also be generalized to high-dimensional spaces. Let $f^{(\leq k)}$ denote a decision tree whose depth is no more than k , and $f^{(k)}$ denote a decision tree whose depth is k . We have the following result in Theorem [2](#).

Theorem 2. *Applying a decision tree $f^{(\leq k)}$ on the $(k+1)$ -XOR classification problem will always lead to $\mathbb{P}_X \{f^{(\leq k)}(X) = \text{XOR}_{k+1}(X)\} = 50\%$, where $X = (x_1, \dots, x_{k+1})$.*

Proof. We prove the theorem by induction. The case of $k = 1$ has already been proved. Now we assume that our conclusion holds for $f^{(\leq k-1)}$. We want to prove that it also holds for $f^{(k)}$.

Without loss of generality, we assume that the splitting variable of $f^{(k)}$'s first layer is the 1st feature x_1 , then

$$f^{(k)} = \mathbf{1}_{\{x_1 \leq c\}} f_1^{(k-1)} + \mathbf{1}_{\{x_1 > c\}} f_2^{(k-1)},$$

where $f_1^{(k-1)}$ and $f_2^{(k-1)}$ represent the left subtree and the right subtree of $f^{(k)}$'s top node respectively, and c is the splitting value. Let $X_{[-1]} = (x_2, \dots, x_{k+1})$, then

$$\begin{aligned} \mathbb{P}_X \{f^{(k)}(X) = \text{XOR}_{k+1}(X)\} &= \mathbb{P}_X \left\{ f_1^{(k-1)}(X) = \text{XOR}_{k+1}(X) \middle| x_1 \leq c \right\} \mathbb{P}_X \{x_1 \leq c\} \\ &\quad + \mathbb{P}_X \left\{ f_2^{(k-1)}(X) = \text{XOR}_{k+1}(X) \middle| x_1 > c \right\} \mathbb{P}_X \{x_1 > c\}. \end{aligned}$$

Assuming $c > 0$ without loss of generality, then we have

$$\begin{aligned} &\mathbb{P}_X \left\{ f_1^{(k-1)}(X) = \text{XOR}_{k+1}(X) \middle| x_1 \leq c \right\} \\ &= \mathbb{P}_X \left\{ f_1^{(k-1)}(X) = \text{XOR}_{k+1}(X) \middle| x_1 \leq c, x_1 \leq 0 \right\} \mathbb{P}_X \{x_1 \leq 0\} \\ &\quad + \mathbb{P}_X \left\{ f_1^{(k-1)}(X) = \text{XOR}_{k+1}(X) \middle| 0 < x_1 \leq c \right\} \mathbb{P}_X \{x_1 > 0\} \\ &= \mathbb{P}_X \left\{ f_1^{(k-1)}(X_{[-1]}) = \text{XOR}_k(X_{[-1]}) \right\} \mathbb{P}_X \{x_1 \leq 0\} \\ &\quad + \mathbb{P}_X \left\{ -f_1^{(k-1)}(X_{[-1]}) = \text{XOR}_k(X_{[-1]}) \right\} \mathbb{P}_X \{x_1 > 0\}, \end{aligned}$$

and

$$\mathbb{P}_X \left\{ f_2^{(k-1)}(X) = \text{XOR}_{k+1}(X) \middle| x_1 > c \right\} = \mathbb{P}_X \left\{ -f_2^{(k-1)}(X_{[-1]}) = \text{XOR}_k(X_{[-1]}) \right\}.$$

Our inductive assumption tells us that, for the XOR_k classification problem, both $f_1^{(k-1)}$ and

$f_2^{(k-1)}$ will have a 50% error. Hence, the three probabilities $\mathbb{P}_X \left\{ f_1^{(k-1)}(X_{[-1]}) = \text{XOR}_k(X_{[-1]}) \right\}$, $\mathbb{P}_X \left\{ -f_1^{(k-1)}(X_{[-1]}) = \text{XOR}_k(X_{[-1]}) \right\}$ and $\mathbb{P}_X \left\{ -f_2^{(k-1)}(X_{[-1]}) = \text{XOR}_k(X_{[-1]}) \right\}$ are all equal to 50%. Finally,

$$\begin{aligned} & \mathbb{P}_X \left\{ f^{(k)}(X) = \text{XOR}_{k+1}(X) \right\} \\ &= [50\% \mathbb{P}_X \{x_1 \leq 0\} + 50\% \mathbb{P}_X \{x_1 > 0\}] \mathbb{P}_X \{x_1 \leq c\} + 50\% \mathbb{P}_X \{x_1 > c\} \\ &= 50\% \mathbb{P}_X \{x_1 \leq c\} + 50\% \mathbb{P}_X \{x_1 > c\} \\ &= 50\%. \end{aligned}$$

□

In the proof above, we suppose that each component of X would be split just only one time. In other words, once the CART algorithm (Hastie et al., 2009, p. 305) split a decision tree at x_1 , it will not split at x_1 again in other layers. It is just for clarity and conciseness, because one can use the total probability formula to deal with the more complicated situations.

Although the k -XOR is a special case that each factor interacts with other factors, it is enough to demonstrate that shallow decision trees (especially for one-layer stumps) may be unable to deal with factors that are not independent of each others.

3.2 The population without “comonotonicity”

In this section, we show the shortcomings of AdaBoost based on stumps by introducing the concept of “comonotonicity”. The conclusion in this section can be regarded as an extension of Section 3.1, since the XOR function do not have the property of comonotonicity, as we will discuss later.

Definition 8 (Comonotonic Population). A population $X \in \mathcal{X} = \mathbb{R}^k, Y \in \mathcal{Y} = \{\pm 1\}$ is comonotonic, if its Bayes classifier f^B satisfies: for any constant c and any $i = 1, \dots, k$, there

exists an $\varepsilon > 0$ such that for each $a \in (c - \varepsilon, c)$ and $b \in (c, c + \varepsilon)$, the elements in

$$\{f^B(x_1, \dots, x_{i-1}, a, x_{i+1}, \dots, x_k) - f^B(x_1, \dots, x_{i-1}, b, x_{i+1}, \dots, x_k) : X_{[-i]} \in \mathbb{R}^{k-1}\}$$

are all non-positive or all non-negative, where $X_{[-i]} = (x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_k)$.

To give an intuition of comonotonicity, in Fig. 6, we give three examples of populations which are not comonotonic. For Fig. 6(a), 6(b) and 6(c), the decision boundaries of their Bayes classifiers form shapes of an XOR, a ring, and a diagonal band, respectively. The yellow (light) region takes a value of +1, and the blue (dark) region -1. Note that, in each figure, there both exist arrows from values of -1 to +1, and arrows from values of +1 to -1. These arrows tell us that the populations are not comonotonic.

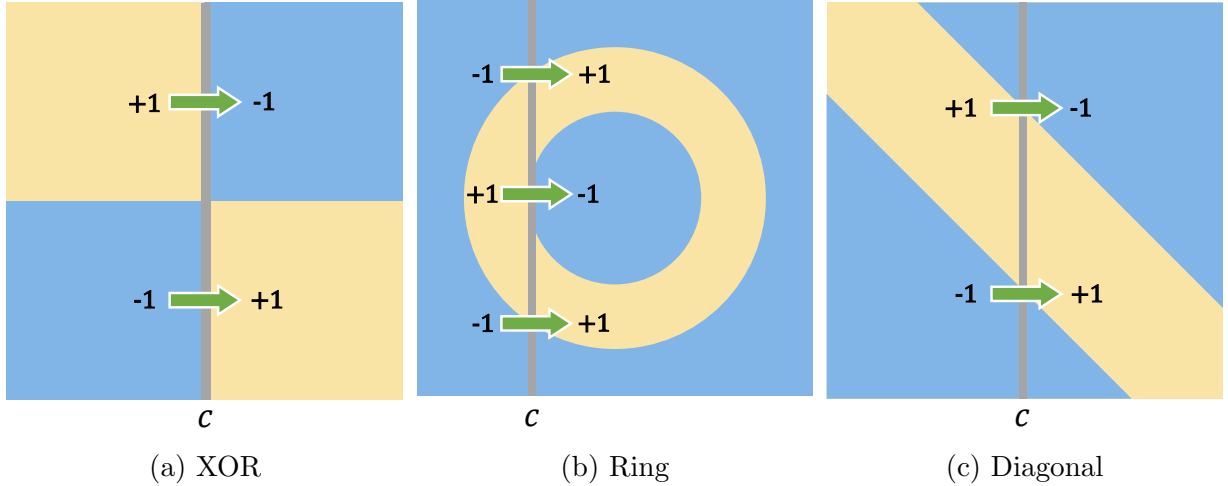


Fig. 6. The populations WITHOUT comonotonicity.

The following theorem shows that AdaBoost based on stumps cannot deal with populations without comonotonicity.

Theorem 3. *For a population in $\mathbb{R}^k$ with a Bayes classifier $f^B(x_1, x_2, \dots, x_k)$, a necessary condition for the classifier trained by AdaBoost based on stumps can converge to f^B as the number of iterations $m \rightarrow \infty$ is: the population is comonotonic.*

Proof. The AdaBoost.M1 algorithm in [Hastie et al. \(2009\)](#), p. 339) shows that, if the number

of iterations is m , the final strong classifier $f^{(m)}$ must takes the form

$$f^{(m)}(x_1, x_2, \dots, x_k) = \text{sign} \left[\sum_{s=1}^m \alpha_s g_s(x_1, x_2, \dots, x_k) \right],$$

where $g_s(\cdot)$, $s = 1, \dots, m$ are base learners (stumps). In other words, $f^{(s)}(\cdot)$ must be a linear combination of base learners.

A stump “A” with k variables can be expressed as

$$\text{stump}_A(x_1, x_2, \dots, x_k) = \text{sign}(x_i \bowtie c) := \begin{cases} 1, & x_i \bowtie c, \\ -1, & \text{otherwise,} \end{cases}$$

where $\bowtie \in \{\geq, \leq, >, <\}$, and the splitting variable of the stump is the i -th feature.

Without loss of generality, we require that $\bowtie$ can only be $\leq$ or $>$. Then the linear combination of stumps trained by AdaBoost can be represented as

$$f^{(m)}(x_1, x_2, \dots, x_k) = \text{sign} \left[L + \sum_{s=1}^m \alpha_s \text{sign}(x_{i_s} \bowtie c_s) \right].$$

where L is a constant, x_{i_s} is the splitting variable of the s -th stump, and c_s is the splitting value of the s -th stump. Since $\text{sign}(x_i > c) = -\text{sign}(x_i \leq c)$, we can adjust all inequality signs to the same direction:

$$f^{(m)}(x_1, x_2, \dots, x_k) = \text{sign} \left[L + \sum_{s=1}^m \alpha_s \text{sign}(x_{i_s} \leq c_s) \right].$$

For simplicity, let us consider the 2-dim case ($k = 2$). One can generalize the following conclusions to high-dimensional spaces similarly. According to the splitting variable of each stump, we can separate the m stumps into two groups as

$$f^{(m)}(x_1, x_2) = \text{sign} \left[L + \left(\sum_{i=1}^{m_1} \beta_i \text{sign}(x_1 \leq a_i) \right) + \left(\sum_{j=1}^{m_2} \gamma_j \text{sign}(x_2 \leq b_j) \right) \right],$$

where m_1 is the number of stumps with x_1 as the splitting variable, m_2 is the number of stumps with x_2 as the splitting variable, and $m_1 + m_2 = m$. Without loss of generality, we assume that $a_1 \leq a_2 \leq \dots \leq a_{m_1}$, and $b_1 \leq b_2 \leq \dots \leq b_{m_2}$.

Recall the definition of comonotonicity (Definition 8). For any constant c , take any $\varepsilon > 0$, $a \in (c - \varepsilon, c)$ and $b \in (c, c + \varepsilon)$. We sort a, b, c and $a_1, \dots, a_{m_1}$ together as

$$a_{m_1,a} \leq a \leq a_{m_1,a+1} \leq \dots \leq a_{m_1,c} \leq c \leq a_{m_1,c+1} \leq \dots \leq a_{m_1,b} \leq b \leq a_{m_1,b+1}.$$

Then, from the expression of $f^{(m)}(x_1, x_2)$, we have

$$f^{(m)}(a, x_2) - f^{(m)}(b, x_2) \equiv \sum_{i=m_1,b}^{m_1,a} \beta_i, \quad \forall x_2,$$

i.e., it does not depend on x_2 . Similarly, we also have that $f^{(m)}(x_1, a) - f^{(m)}(x_1, b)$ does not depend on x_1 . Let $m \rightarrow \infty$, and if the algorithm will converge, then

$$\lim_{m \rightarrow \infty} [f^{(m)}(a, x_2) - f^{(m)}(b, x_2)]$$

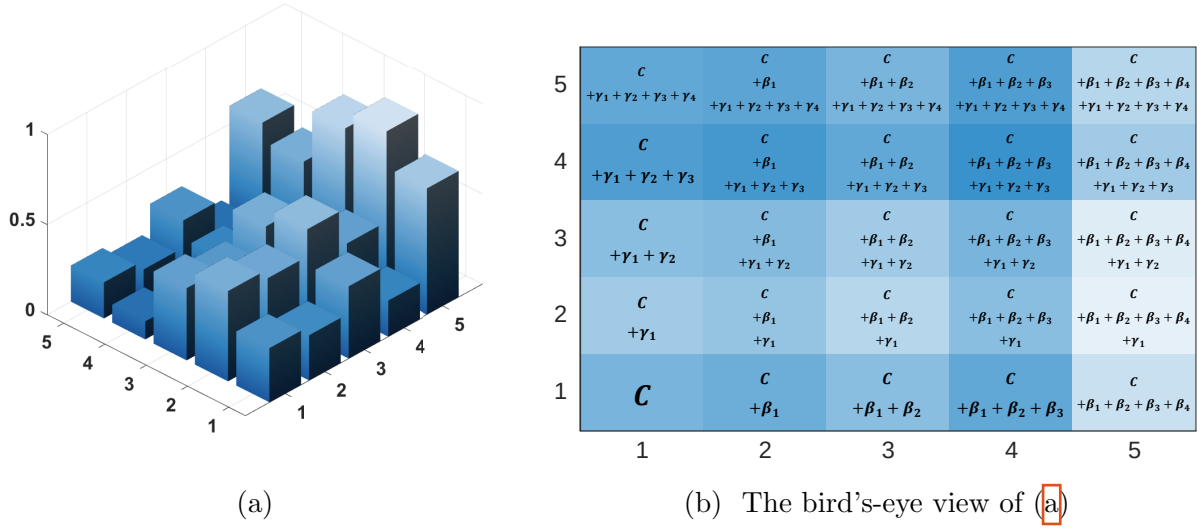
and

$$\lim_{m \rightarrow \infty} [f^{(m)}(x_1, a) - f^{(m)}(x_1, b)]$$

will also be constants which do not depend on x_2 and x_1 , respectively. Therefore, according to the definition of comonotonicity, $f^{(m)}$ cannot converge to f^B if the population is not comonotonic. $\square$

To show the intuition of $f^{(m)}$ in the proof above, let $f(x_1, x_2) = L + \left(\sum_{i=1}^{m_1} \beta_i \text{sign}(x_1 \leq a_i) \right) + \left(\sum_{j=1}^{m_2} \gamma_j \text{sign}(x_2 \leq b_j) \right)$. Fig. 7 illustrates the property of the final strong classifier f .

Fig. 7 is a toy example of the strong classifier f with $M_1 = 5$ and $M_2 = 5$. Fig. 7(a) is the graph of the function $f(x_1, x_2)$, and 7(b) is the bird's-eye view of 7(a). The darker the color is, the smaller the value of f takes. The value of f is written explicitly on 7(b),

Fig. 7. An example of the strong classifier f .

which shows that, the values in row 2 are γ_1 greater than row 1, and the values in row 3 are γ_2 greater than row 2, and so on. Similarly, the values in column 2 are β_1 greater than column 1, and the values in column 3 are β_2 greater than column 2. All numbers in the grids increase or decrease *the same* values, from left to right, and from bottom to top. It is the pattern of comonotonicity.

We have already shown in Fig. 6(a) that the XOR function is not comonotonic. Therefore, if the Bayes classifier of a population is the XOR function, it is impossible to give a good answer to the classification problem by training AdaBoost based on stumps. The conclusion in this section can be regarded as a generalization of Section 3.1.

In portfolio management, it is very common that factors may have interactions among each others. Hence, non-comonotonic populations are not rare. Although AdaBoost based on stumps can achieve good results in some areas, in financial studies, just based on stumps is far from reaching the desired goal. In Section 4, we use empirical studies to show that, using deeper trees as base learners of AdaBoost is usually a better choice in portfolio management.

4 Empirical studies

In this section, we use the data of the Chinese A-share market to give empirical studies about the factor investing strategy based on AdaBoost. How to construct a stock factor strategy is an open problem with long history in portfolio management. From Wang et al. (2012) invented the N-LASR to Fiévet and Sornette (2018) proposed a decision tree forecasting model, and Gu et al. (2020) and D'Hondt et al. (2020) gave a comprehensive analysis of machine learning methods for the canonical problem of empirical asset pricing, all of them agree that it may improve the strategy performance if the prediction model can dig out nonlinear and complex information.

Our empirical studies have two goals. On the one hand, by selecting an optimal portfolio management strategy based on AdaBoost, we want to verify the general theoretical results about the interpolation and localization of AdaBoost in Section 2 and Section 3. On the other hand, we want to illustrate the good performance of the equal-weighted strategy based on AdaBoost.

In order to achieve the first goal, we give a sensitivity analysis about the depth of the base learners (decision trees) and the number of iterations of AdaBoost on the training set and the test set. We specifically explain the performance of AdaBoost that it can dig out useful information efficiently, as well as decrease the test error.

4.1 Data

The empirical data starts in June 2002 and ends in June 2017, 181 months in total. All stocks traded in the Chinese A-share market are included. 60 factors are used in our strategy. The data of the factor exposures and the monthly returns are downloaded from the Wind Financial Terminal⁵. The 60 factors include not only the fundamental factors, but also the technical factors, such as the momentum and the turnover. All 60 factors are listed in Table

²

⁵<https://www.wind.com.cn/en/Default.html>

The original data has been preliminarily cleaned, but we still need to do some preprocessing before training. We remove all stocks which are not traded (or cannot be traded due to the limit-up or limit-down in the Chinese market) during the period we study. We remove the factors with over 10% missing data, and fill in the missing data of other factors with 0. For each month, we assign the response variables Y of all stocks according to their ranks of the next-month returns cross-sectionally. The response variables of the top 50% stocks are +1, and that of the bottom 50% stocks are -1.

We divide all data into a training set and a test set manually. Total 181 months' data is divided into two sets: the first 127 months' data (June 2002–December 2012) is taken as the training set, and the last 54 months' data (January 2013–June 2017) is taken as the test set. Then, the size of the training set is 193455 (sum of the stock numbers in all months), and the size of the test set is 133277. We use the training set to fit models, and then use the test set to evaluate the models and verify our conclusions in Section 2 and Section 3.

Table 2: The 60 factors.

alr	IR_bps_252	IR_netasset_126	IR_net_profit_63	IR_roe_252	net_assets
amount_21	IR_bps_63	IR_netasset_252	IR_oper_rev_126	IR_roe_63	oper_rev_ttm
avg_volume_63	IR_eps_126	IR_netasset_63	IR_oper_rev_252	IR_totasset_126	pb
bps	IR_eps_252	IR_net_profit_126	IR_oper_rev_63	IR_totasset_252	q_eps
IR_bps_126	IR_eps_63	IR_net_profit_252	IR_roe_126	IR_totasset_63	q_grossprofitmargin
q_netprofitmargin	q_ps	rt_252	tot_assets	ttm_pcf	turnover_126
q_oper_rev	q_roa	rt_63	ttm_eps	ttm_pe	turnover_21
q_orps	q_roe	shr_float2tot	ttm_grossprofitmargin	ttm_ps	turnover_252
q_pcf	rt_126	s_dq_mv	ttm_netprofitmargin	ttm_roa	turnover_63
q_pe	rt_21	s_val_mv	ttm_orps	ttm_roe	val_float2tot

4.2 The performance of the AdaBoost classifiers

In this section, we analyze how the performance of the classifiers trained by AdaBoost varies with the two hyperparameters: the depth of the base learners (decision trees), and the number of iterations. Both hyperparameters are typically the source of overfitting in common sense. More specifically, we consider the following hyperparameters:

- Max_Depth: The maximum depth of the base learners (decision trees), takes 2, 4, 6, 8, and 10, respectively;

- N_Steps: The number of iterations, takes 10, 20, 30, 40, 50, 100, 500, and 1000, respectively.

In order to analyze the influence of these two hyperparameters on the fitting ability of AdaBoost, we fix other parameters, and set the learning rates of all models as 0.1. Both in the training set and in the test set, we use the AUC (area under the ROC curve) to measure the performance of the models, which is a supplement to the usual error calculation.

The performance results of all models we studied are summarized in Table 3. Based on the results, we observe that:

- The training/test AUC and the training/test error are consistent, since if the AUC is high, the error will be low in almost every scenarios. For example, when Max_Depth = 2 and N_Steps = 10 (the 1st model), the training AUC is 0.5412 while the training error is 0.4701; And when Max_Depth = 2 and N_Steps = 20 (the 2nd model), the training AUC is 0.5436 while the training error is 0.4700—They vary in different directions.
- The training AUC increases monotonically as the complexity of the model increases. Specifically, from the first model to the last model, the complexity increases. Meanwhile, the training AUC increases from 0.5412 to 0.6828; The training error decreases too.
- The test AUC also almost increases monotonically as the complexity of the model increases. For instance, when Max_Depth = 2 and N_Steps = 10, ..., 50 (the 1st-5th models), the test AUC increases from 0.5433 to 0.5480; When N_Steps = 20 and Max_Depth = 2, 4, 6 (the 2nd, 7th and 12th models), the test AUC also increases from 0.5462 to 0.5490.
- The changes of the test AUC are relatively small and stable with regard to that of the training AUC. For example, for the first 15 models, the test AUC changes from 0.5433 to 0.5513, while the training AUC changes from 0.5412 to 0.5946. It suggests

that the test AUC around 54% may be a stable threshold of the model, which reflects the ability of our methods to dig out the market information contained in our dataset. It is noteworthy that, 54% is not a bad result in the Chinese stock market, according to the experience of the industry.

- In the training set, the performance is more sensitive to Max_Depth than to N_Steps. In detail, given Max_Depth = 2, the training AUC changes from 0.5412 to 0.5533 for N_Steps = 10, ..., 50 (the 1st-5th models); However, given N_Steps = 10, the training AUC changes from 0.5412 to 0.5818 for Max_Depth = 2, 4, 6.

We find that, as the depth of the trees and the number of iterations increases, the AUC for the test set increases stably without significant change. We can conclude that, in these cases, the more iteration steps, the better the classifier, and the more complex the base learner trees, the better the classifier.

Table 3: Model performance results.

Model No.	Hyperparameters		Training Set		Test Set	
	Max_Depth	N_Steps	Training AUC	Training Error	Test AUC	Test Error
1	2	10	0.5412	0.4701	0.5433	0.4713
2	2	20	0.5436	0.4700	0.5462	0.4741
3	2	30	0.5499	0.4665	0.5468	0.4728
4	2	40	0.5511	0.4656	0.5476	0.4716
5	2	50	0.5533	0.4636	0.5480	0.4714
6	4	10	0.5628	0.4545	0.5463	0.4671
7	4	20	0.5682	0.4515	0.5487	0.4697
8	4	30	0.5699	0.4500	0.5489	0.4681
9	4	40	0.5713	0.4505	0.5498	0.4669
10	4	50	0.5723	0.4498	0.5500	0.4669
11	6	10	0.5818	0.4418	0.5458	0.4715
12	6	20	0.5870	0.4392	0.5490	0.4683
13	6	30	0.5913	0.4353	0.5502	0.4675
14	6	40	0.5930	0.4346	0.5506	0.4676
15	6	50	0.5946	0.4338	0.5513	0.4670
16	8	100	0.6300	0.4108	0.5519	0.4663
17	8	500	0.6356	0.4071	0.5531	0.4659
18	8	1000	0.6358	0.4071	0.5532	0.4659
19	10	100	0.6731	0.3805	0.5486	0.4679
20	10	500	0.6769	0.3780	0.5502	0.4677
21	10	1000	0.6828	0.3740	0.5499	0.4681

4.3 Strategy results

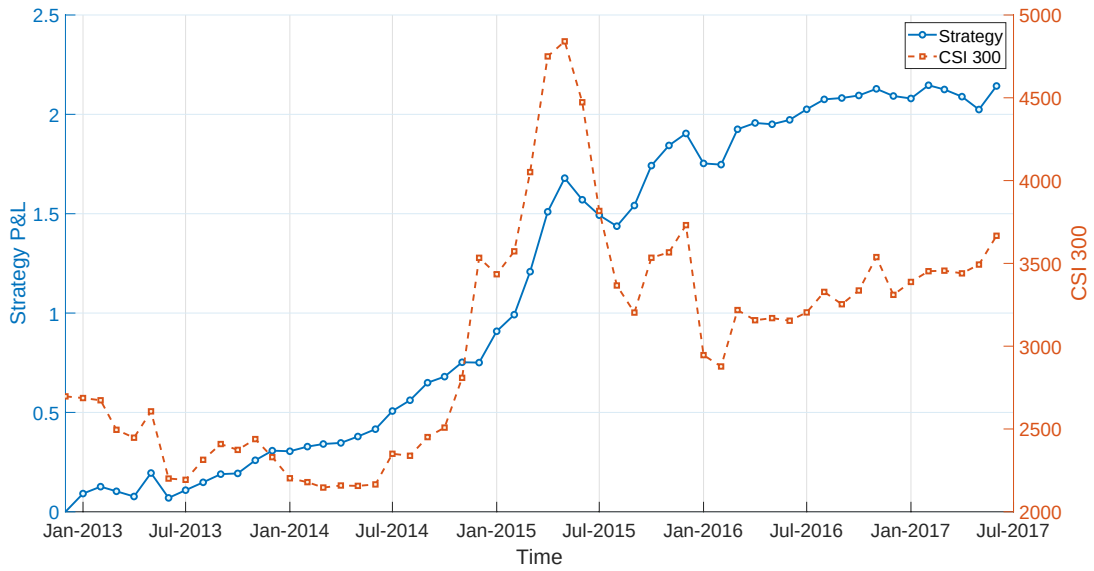
In this section, we use the classifier with good performance in the training set to establish equal-weighted strategies in the test set (January 2013–June 2017). We choose the 16th classifier in Table 3: Max_Depth=8, and N_steps=100.

Our strategy is: At the beginning of each month, we treat the last month’s exposures of the 60 factors for all stocks as the input of the classifier. Then, we use the output of the classifier to determine whether to take a long or a short position. For the long-short strategy, we long the top 50 stocks with the highest probability of going up, and short the bottom 50 stocks with the highest probability of going down; For the long-only strategy, we only long the top 50 stocks with the highest probability of going up.

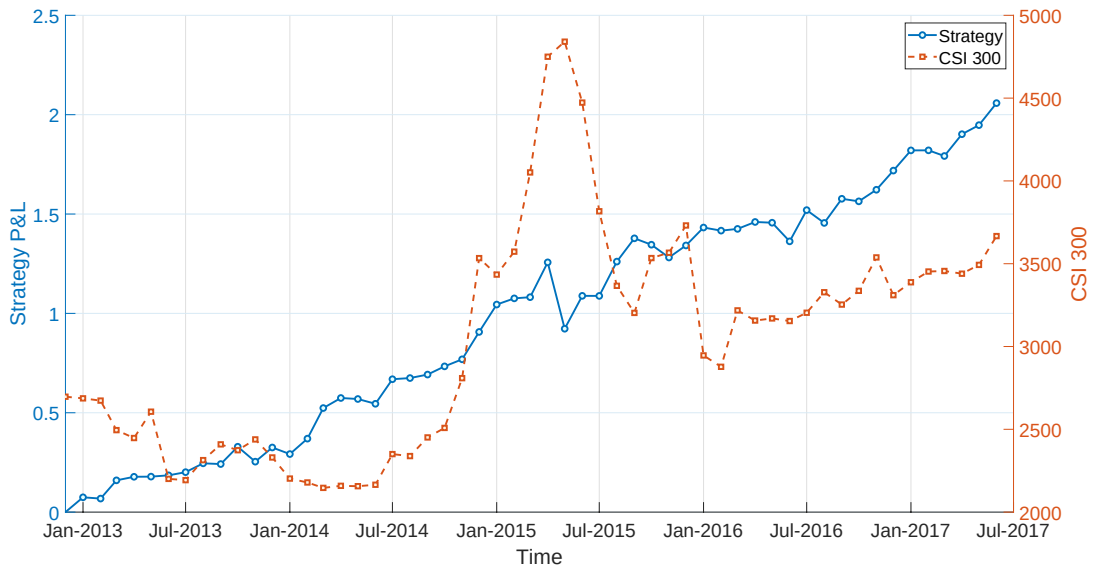
In Fig. 8, we use the CSI 300 to denote the benchmark of the buy-and-hold strategy which we want to compare with. Fig. 8(b) and 8(a) show the P&Ls of the long-only strategy and the long-short strategy, respectively. Both the transaction cost rates of taking a long position and a short position are set as 0.15%. Table 4 shows the performance summaries of our strategies, including the win rate, the Sharpe ratio, the average return, the standard deviation of return, and the maximum drawdown as different measures of the performance.

From the strategy performance, we have the following results:

- The performance of our strategies is remarkable, although the AdaBoost classifier is fixed for the whole test period (January 2013–June 2017). The average annualized return for both the long-only and the long-short strategy are over 45% during the test period. Particularly, the Chinese A-share market experienced a dramatic fluctuation in 2015, but our strategies (the blue solid line) still have a relatively robust return comparing with the CSI 300 benchmark (the orange dashed line).
- The performance of our strategies is relatively stable, and even the maximum drawdown of the long-only strategy is much lower than the maximum drawdown of the benchmark CSI 300 (72.80%). The maximum drawdown 24.18% of the long-only strategy occurred



(a) Long-only strategy



(b) Long-short strategy

Fig. 8. The P&L of two strategies.

in August 2015, and 33.41% of the long-short strategy occurred in May 2015.

Once again, we should note that the goal of these empirical studies is to find an optimal strategy by AdaBoost which is a complicated classifier in-sample, as well as to demonstrate that the performance of the complicated model is not bad out-of-sample.

Overall, we can conclude from the empirical studies:

- For the equal-weighted factor investing strategy, increasing the depth of the base learners (decision trees) and the number of iterations of AdaBoost do not significantly reduce the out-of-sample performance;
- For factor investing strategies, it is a feasible way for investors to use the complicated AdaBoost method to learn in the training set.

Table 4: Strategy performance results.

Strategy	Win Rate	Sharpe Ratio	Average Return	Std.	Maximum Drawdown
long-only	0.704	1.665	47.62%	28.60%	24.18%
long-short	0.704	1.568	45.74%	29.17%	33.41%
CSI 300	0.537	0.243	7.99%	32.83%	72.80%

5 Conclusion

In order to implement machine learning in constructing equal-weighted portfolios interpretatively, our paper explains the success of AdaBoost and applies it in portfolio management.

We prove that a “complex” method, even interpolating, does not necessarily result in poor out-of-sample performance if it can lower the influence of the noise points. AdaBoost based on large trees without early stopping could be one of such methods, and we explain its success with our newly defined ION.

We illustrate the shortcomings of AdaBoost based on shallow trees from the perspective of its ability to dig out non-linear information, and emphasize the benefits of AdaBoost based on large trees.

Our empirical studies not only corroborate the theoretical results, but also give an effective approach to construct equal-weighted portfolios via machine learning.

Conclusively, we show that AdaBoost can minimize the influence of the noise points while interpolating the training set, and thus have a good out-of-sample performance. We confirm the conjectures in [Wyner et al. \(2017\)](#) under a mathematical framework. The empirical studies verify the potential applications of AdaBoost in portfolio management.

References

- Asness, C., A. Frazzini, R. Israel, T. J. Moskowitz, and L. H. Pedersen (2018). Size matters, if you control your junk. *Journal of Financial Economics* 129(3), 479–509.
- Breiman, L. (1998). Arcing classifier (with discussion and a rejoinder by the author). *The Annals of Statistics* 26(3), 801–849.
- Breiman, L. (1999). Prediction games and arcing algorithms. *Neural Computation* 11(7), 1493–1517.
- Breiman, L. (2001). Random forests. *Machine Learning* 45(1), 5–32.
- Chen, T. and C. Guestrin (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16, New York, NY, USA, pp. 785–794. ACM.
- Cover, T. and P. Hart (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory* 13(1), 21–27.
- Creamer, G. (2012). Model calibration and automated trading agent for euro futures. *Quantitative Finance* 12(4), 531–545.
- Creamer, G. and Y. Freund (2010). Automated trading with boosting and expert weighting. *Quantitative Finance* 10(4), 401–420.

- Creamer, G. G. and Y. Freund (2005). Using adaboost for equity investment scorecards. *Howe School Research Paper*. NIPS Workshop Machine Learning in Finance, 2005, Whistler, British Columbia, Canada.
- DeMiguel, V., L. Garlappi, and R. Uppal (2007). Optimal versus naive diversification: How inefficient is the $1/N$ portfolio strategy? *The Review of Financial Studies* 22(5), 1915–1953.
- Devroye, L., L. Györfi, and G. Lugosi (1996). *A Probabilistic Theory of Pattern Recognition*, Volume 31 of *Stochastic Modelling and Applied Probability*. Springer-Verlag New York.
- D’Hondt, C., R. De Winne, E. Ghysels, and S. Raymond (2020). Artificial intelligence alter egos: Who might benefit from robo-investing? *Journal of Empirical Finance* 59, 278–299.
- Feng, G., S. Giglio, and D. Xiu (2020). Taming the factor zoo: a test of new factors. *The Journal of Finance* 75(3), 1327–1370.
- Fiévet, L. and D. Sornette (2018). Decision trees unearth return sign predictability in the s&p 500. *Quantitative Finance* 18(11), 1797–1814.
- Freund, Y. and R. E. Schapire (1996). Experiments with a new boosting algorithm. In *Proceedings of the Thirteenth International Conference on International Conference on Machine Learning*, ICML’96, San Francisco, CA, USA, pp. 148–156. Morgan Kaufmann Publishers Inc.
- Friedman, J., T. Hastie, and R. Tibshirani (2000). Additive logistic regression: a statistical view of boosting (with discussion and a rejoinder by the authors). *The Annals of Statistics* 28(2), 337–407.
- Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *The Annals of Statistics* 29(5), 1189–1232.

- Gu, S., B. Kelly, and D. Xiu (2020). Empirical asset pricing via machine learning. *The Review of Financial Studies* 33(5), 2223–2273.
- Harvey, C. R. (2017). Presidential address: the scientific outlook in financial economics. *The Journal of Finance* 72(4), 1399–1440.
- Harvey, C. R., Y. Liu, and H. Zhu (2016). ... and the cross-section of expected returns. *The Review of Financial Studies* 29(1), 5–68.
- Hastie, T., R. Tibshirani, and J. H. Friedman (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2 ed.). Springer Series in Statistics. Springer, New York.
- James, G. M. (2003). Variance and bias for general loss functions. *Machine Learning* 51(2), 115–135.
- Jobson, J. D. and B. M. Korkie (1981). Performance hypothesis testing with the sharpe and treynor measures. *The Journal of Finance* 36(4), 889–908.
- Ke, G., Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu (2017). Lightgbm: A highly efficient gradient boosting decision tree. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), *Advances in Neural Information Processing Systems*, Volume 30, pp. 3146–3154. Curran Associates, Inc.
- López de Prado, M. (2018). *Advances in Financial Machine Learning*. Wiley.
- Mease, D. and A. Wyner (2008). Evidence contrary to the statistical view of boosting. *Journal of Machine Learning Research* 9(Feb), 131–156.
- Michaud, R. O. (1989). The markowitz optimization enigma: Is ‘optimized’ optimal? *Financial Analysts Journal* 45(1), 31–42.

- Rasekhschaffe, K. C. and R. C. Jones (2019). Machine learning for stock selection. *Financial Analysts Journal* 75(3), 70–88.
- Schapire, R. E., Y. Freund, P. Bartlett, and W. S. Lee (1998). Boosting the margin: a new explanation for the effectiveness of voting methods. *The Annals of Statistics* 26(5), 1651–1686.
- Tu, J. and G. Zhou (2011). Markowitz meets talmud: A combination of sophisticated and naive diversification strategies. *Journal of Financial Economics* 99(1), 204–215.
- Wang, S., Y. Luo, R. Cahan, M.-A. Alvarez, J. Jussa, and Z. Chen (2012). Signal processing: the rise of the machines. *Deutsche Bank Quantitative Strategy*, 5 June 2012.
- Wyner, A. J. (2003). On boosting and the exponential loss. In *Proceedings of the Ninth International Workshop on Artificial Intelligence and Statistics, AISTATS 2003, Key West, Florida, USA, January 3-6, 2003*.
- Wyner, A. J., M. Olson, J. Bleich, and D. Mease (2017). Explaining the success of adaboost and random forests as interpolating classifiers. *Journal of Machine Learning Research* 18(48), 1–33.
- Zhang, H., B. Jacobsen, F. de Roon, and F. Jiang (2019). Boosting portfolio choice in the big data era. *SSRN 3495901*.

This is called **stacking**, which stands for “stacked generalization” [Wol92]. Note that the combination weights used by stacking need to be trained on a separate dataset, otherwise they would put all their mass on the best performing base model.

18.2.2 Ensembling is not Bayes model averaging

It is worth noting that an ensemble of models is not the same as using Bayes model averaging over models (Section 4.6), as pointed out in [Min00]. An ensemble considers a larger hypothesis class of the form

$$p(y|\mathbf{x}, \mathbf{w}, \boldsymbol{\theta}) = \sum_{m \in \mathcal{M}} w_m p(y|\mathbf{x}, \boldsymbol{\theta}_m) \quad (18.13)$$

whereas BMA uses

$$p(y|\mathbf{x}, \mathcal{D}) = \sum_{m \in \mathcal{M}} p(m|\mathcal{D}) p(y|\mathbf{x}, m, \mathcal{D}) \quad (18.14)$$

The key difference is that in the case of BMA, the weights $p(m|\mathcal{D})$ sum to one, and in the limit of infinite data, only a single model will be chosen (namely the MAP model). By contrast, the ensemble weights w_m are arbitrary, and don’t collapse in this way to a single model.

18.3 Bagging

In this section, we discuss **bagging** [Bre96], which stands for “bootstrap aggregating”. This is a simple form of ensemble learning in which we fit M different base models to different randomly sampled versions of the data; this encourages the different models to make diverse predictions. The datasets are sampled with replacement (a technique known as bootstrap sampling, Section 4.7.3), so a given example may appear multiple times, until we have a total of N examples per model (where N is the number of original data points).

The disadvantage of bootstrap is that each base model only sees, on average, 63% of the unique input examples. To see why, note that the chance that a single item will not be selected from a set of size N in any of N draws is $(1 - 1/N)^N$. In the limit of large N , this becomes $e^{-1} \approx 0.37$, which means only $1 - 0.37 = 0.63$ of the data points will be selected.

The 37% of the training instances that are not used by a given base model are called **out-of-bag instances** (oob). We can use the predicted performance of the base model on these oob instances as an estimate of test set performance. This provides a useful alternative to cross validation.

The main advantage of bootstrap is that it prevents the ensemble from relying too much on any individual training example, which enhances robustness and generalization [Gra04]. For example, comparing Figure 18.3b and Figure 18.3c, we see that omitting a single example from the training set can have a large impact on the decision tree that we learn (even though the tree growing algorithm is otherwise deterministic). By averaging the predictions from both of these models, we get the more reasonable prediction model in Figure 18.3d. This advantage generally increases with the size of the ensemble, as shown in Figure 18.4. (Of course, larger ensembles take more memory and more time.)

Bagging does not always improve performance. In particular, it relies on the base models being unstable estimators, so that omitting some of the data significantly changes the resulting model fit.

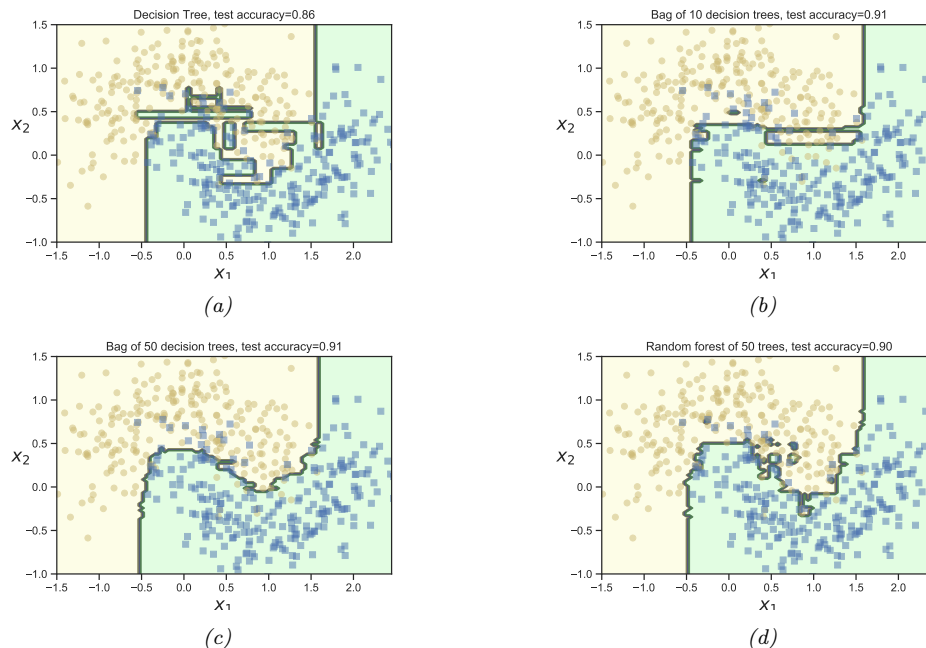


Figure 18.4: (a) A single decision tree. (b-c) Bagging ensemble of 10 and 50 trees. (d) Random forest of 50 trees. Adapted from Figure 7.5 of [Gér19]. Generated by [bagging_trees.ipynb](#) and [rf_demo_2d.ipynb](#).

This is the case for decision trees, but not for other models, such as nearest neighbor classifiers. For neural networks, the story is more mixed. They can be unstable wrt their training set. On the other hand, deep networks will underperform if they only see 63% of the data, so bagged DNNs do not usually work well [NTL20].

18.4 Random forests

Bagging relies on the assumption that re-running the same learning algorithm on different subsets of the data will result in sufficiently diverse base models. The technique known as **random forests** [Bre01] tries to decorrelate the base learners even further by learning trees based on a randomly chosen subset of input variables (at each node of the tree), as well as a randomly chosen subset of data cases. It does this by modifying Equation (18.5) so the the feature split dimension j is optimized over a random subset of the features, $S_i \subset \{1, \dots, D\}$.

For example, consider the email spam dataset [HTF09, p301]. This dataset contains 4601 email messages, each of which is classified as spam (1) or non-spam (0). The data was open sourced by George Forman from Hewlett-Packard (HP) Labs.

There are 57 quantitative (real-valued) features, as follows:

- 48 features corresponding to the percentage of words in the email that match a given word, such as “remove” or “labs”.

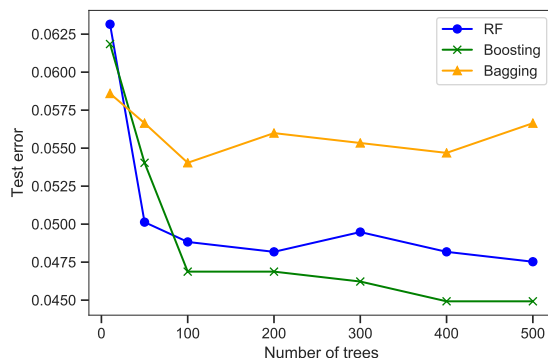


Figure 18.5: Predictive accuracy vs size of tree ensemble for bagging, random forests and gradient boosting with log loss. Adapted from Figure 15.1 of [HTF09]. Generated by `spam_tree_ensemble_compare.ipynb`.

- 6 features corresponding to the percentage of characters in the email that match a given character, namely ; . [! \$ #
- 3 features corresponding to the average length, max length, and sum of lengths of uninterrupted sequences of capital letters. (These features are called CAPAVE, CAPMAX and CAPTOT.)

Figure 18.5 shows that random forests work much better than bagged decision trees, because many input features are irrelevant. (We also see that a method called “boosting”, discussed in Section 18.5, works even better; however, this requires sequentially fitting trees, whereas random forests can be fit in parallel.)

18.5 Boosting

Ensembles of trees, whether fit by bagging or the random forest algorithm, corresponding to a model of the form

$$f(\mathbf{x}; \boldsymbol{\theta}) = \sum_{m=1}^M \beta_m F_m(\mathbf{x}; \boldsymbol{\theta}_m) \quad (18.15)$$

where F_m is the m 'th tree, and β_m is the corresponding weight, often set to $\beta_m = 1/M$. We can generalize this by allowing the F_m functions to be general function approximators, such as neural networks, not just trees. The result is called an **additive model** [HTF09]. We can think of this as a linear model with **adaptive basis functions**. The goal, as usual, is to minimize the empirical loss (with an optional regularizer):

$$\mathcal{L}(f) = \sum_{i=1}^N \ell(y_i, f(\mathbf{x}_i)) \quad (18.16)$$

Boosting [Sch90; FS96] is an algorithm for sequentially fitting additive models where each F_m is a binary classifier that returns $F_m \in \{-1, +1\}$. In particular, we first fit F_1 on the original data,

and then we weight the data samples by the errors made by F_1 , so misclassified examples get more weight. Next we fit F_2 to this weighted data set. We keep repeating this process until we have fit the desired number M of components. (M is a hyper-parameter that controls the complexity of the overall model, and can be chosen by monitoring performance on a validation set, and using early stopping.)

It can be shown that, as long as each F_m has an accuracy that is better than chance (even on the weighted dataset), then the final ensemble of classifiers will have higher accuracy than any given component. That is, if F_m is a **weak learner** (so its accuracy is only slightly better than 50%), then we can boost its performance using the above procedure so that the final f becomes a **strong learner**. (See e.g., [SF12] for more details on the learning theory approach to boosting.)

Note that boosting reduces the bias of the strong learner, by fitting trees that depend on each other, whereas bagging and RF reduce the variance by fitting independent trees. In many cases, boosting can work better. See Figure 18.5 for an example.

The original boosting algorithm focused on binary classification with a particular loss function that we will explain in Section 18.5.3, and was derived from the PAC learning theory framework (see Section 5.4.4). In the rest of this section, we focus on a more statistical version of boosting, due to [FHT00; Fri01], which works with arbitrary loss functions, making the method suitable for regression, multi-class classification, ranking, etc. Our presentation is based on [HTF09, ch10] and [BH07], which should be consulted for further details.

18.5.1 Forward stagewise additive modeling

In this section, we discuss **forward stagewise additive modeling**, in which we sequentially optimize the objective in Equation (18.16) for general (differentiable) loss functions, where f is an additive model as in Equation 18.15. That is, at iteration m , we compute

$$(\beta_m, \theta_m) = \underset{\beta, \theta}{\operatorname{argmin}} \sum_{i=1}^N \ell(y_i, f_{m-1}(\mathbf{x}_i) + \beta F(\mathbf{x}_i; \theta)) \quad (18.17)$$

We then set

$$f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \beta_m F(\mathbf{x}; \theta_m) = f_{m-1}(\mathbf{x}) + \beta_m F_m(\mathbf{x}) \quad (18.18)$$

(Note that we do not adjust the parameters of previously added models.) The details on how to perform this optimization step depend on the loss function that we choose, and (in some cases) on the form of the weak learner F , as we discuss below.

18.5.2 Quadratic loss and least squares boosting

Suppose we use squared error loss, $\ell(y, \hat{y}) = (y - \hat{y})^2$. In this case, the i 'th term in the objective at step m becomes

$$\ell(y_i, f_{m-1}(\mathbf{x}_i) + \beta F(\mathbf{x}_i; \theta)) = (y_i - f_{m-1}(\mathbf{x}_i) - \beta F(\mathbf{x}_i; \theta))^2 = (r_{im} - \beta F(\mathbf{x}_i; \theta))^2 \quad (18.19)$$

where $r_{im} = y_i - f_{m-1}(\mathbf{x}_i)$ is the residual of the current model on the i 'th observation. We can minimize the above objective by simply setting $\beta = 1$, and fitting F to the residual errors. This is called **least squares boosting** [BY03].

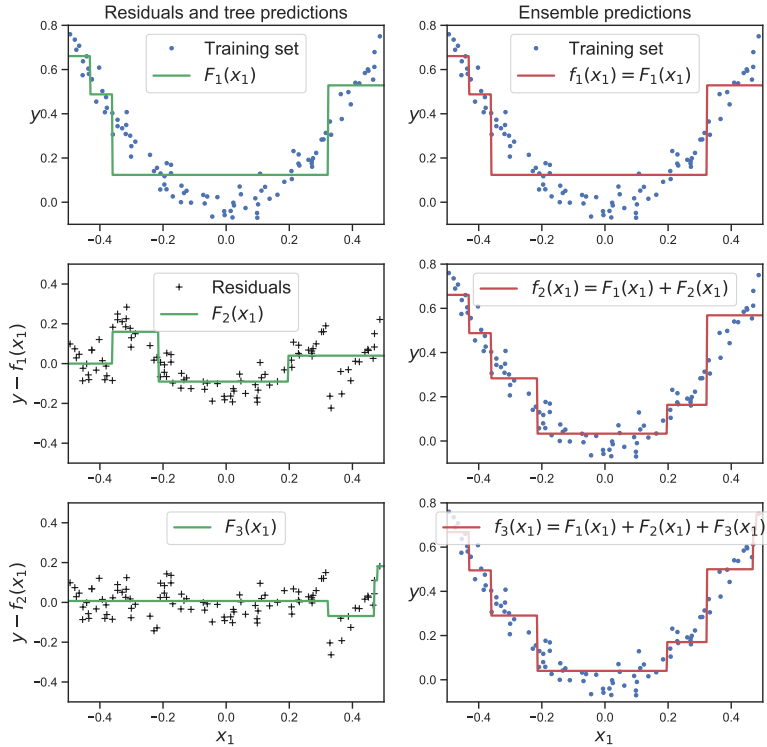


Figure 18.6: Illustration of boosting using a regression tree of depth 2 applied to a 1d dataset. Adapted from Figure 7.9 of [Gér19]. Generated by `boosted_regr_trees.ipynb`.

We give an example of this process in Figure 18.6, where we use a regression tree of depth 2 as the weak learner. On the left, we show the result of fitting the weak learner to the residuals, and on the right, we show the current strong learner. We see how each new weak learner that is added to the ensemble corrects the errors made by earlier versions of the model.

18.5.3 Exponential loss and AdaBoost

Suppose we are interested in binary classification, i.e., predicting $\tilde{y}_i \in \{-1, +1\}$. Let us assume the weak learner computes

$$p(y = 1|\mathbf{x}) = \frac{e^{F(\mathbf{x})}}{e^{-F(\mathbf{x})} + e^{F(\mathbf{x})}} = \frac{1}{1 + e^{-2F(\mathbf{x})}} \quad (18.20)$$

so $F(\mathbf{x})$ returns half the log odds. We know from Equation (10.13) that the negative log likelihood is given by

$$\ell(\tilde{y}, F(\mathbf{x})) = \log(1 + e^{-2\tilde{y}F(\mathbf{x})}) \quad (18.21)$$

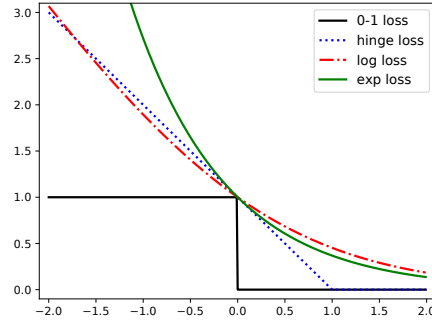


Figure 18.7: Illustration of various loss functions for binary classification. The horizontal axis is the margin $m(\mathbf{x}) = \tilde{y}F(\mathbf{x})$, the vertical axis is the loss. The log loss uses log base 2. Generated by [hinge_loss_plot.ipynb](#).

We can minimize this by ensuring that the **margin** $m(\mathbf{x}) = \tilde{y}F(\mathbf{x})$ is as large as possible. We see from Figure 18.7 that the log loss is a smooth upper bound on the 0-1 loss. We also see that it penalizes negative margins more heavily than positive ones, as desired (since positive margins are already correctly classified).

However, we can also use other loss functions. In this section, we consider the **exponential loss**

$$\ell(\tilde{y}, F(\mathbf{x})) = \exp(-\tilde{y}F(\mathbf{x})) \quad (18.22)$$

We see from Figure 18.7 that this is also a smooth upper bound on the 0-1 loss. In the population setting (with infinite sample size), the optimal solution to the exponential loss is the same as for log loss. To see this, we can just set the derivative of the expected loss (for each $\mathbf{x}$) to zero:

$$\frac{\partial}{\partial F(\mathbf{x})} \mathbb{E} \left[e^{-\tilde{y}f(\mathbf{x})} | \mathbf{x} \right] = \frac{\partial}{\partial F(\mathbf{x})} [p(\tilde{y} = 1 | \mathbf{x})e^{-F(\mathbf{x})} + p(\tilde{y} = -1 | \mathbf{x})e^{F(\mathbf{x})}] \quad (18.23)$$

$$= -p(\tilde{y} = 1 | \mathbf{x})e^{-F(\mathbf{x})} + p(\tilde{y} = -1 | \mathbf{x})e^{F(\mathbf{x})} \quad (18.24)$$

$$= 0 \Rightarrow \frac{p(\tilde{y} = 1 | \mathbf{x})}{p(\tilde{y} = -1 | \mathbf{x})} = e^{2F(\mathbf{x})} \quad (18.25)$$

However, it turns out that the exponential loss is easier to optimize in the boosting setting, as we show below. (We consider the log loss case in Section 18.5.4.)

We now discuss how to solve for the m 'th weak learner, F_m , when we use exponential loss. We will assume that the base classifier F_m returns a binary class label; the resulting algorithm is called **discrete AdaBoost** [FHT00]. If F_m returns a probability instead, a modified algorithm, known as **real AdaBoost**, can be used [FHT00].

At step m we have to minimize

$$L_m(F) = \sum_{i=1}^N \exp[-\tilde{y}_i(f_{m-1}(\mathbf{x}_i) + \beta F(\mathbf{x}_i))] = \sum_{i=1}^N \omega_{i,m} \exp(-\beta \tilde{y}_i F(\mathbf{x}_i)) \quad (18.26)$$

where $\omega_{i,m} \triangleq \exp(-\tilde{y}_i f_{m-1}(\mathbf{x}_i))$ is a weight applied to datacase i , and $\tilde{y}_i \in \{-1, +1\}$. We can rewrite this objective as follows:

$$L_m = e^{-\beta} \sum_{\tilde{y}_i = F(\mathbf{x}_i)} \omega_{i,m} + e^{\beta} \sum_{\tilde{y}_i \neq F(\mathbf{x}_i)} \omega_{i,m} \quad (18.27)$$

$$= (e^{\beta} - e^{-\beta}) \sum_{i=1}^N \omega_{i,m} \mathbb{I}(\tilde{y}_i \neq F(\mathbf{x}_i)) + e^{-\beta} \sum_{i=1}^N \omega_{i,m} \quad (18.28)$$

Consequently the optimal function to add is

$$F_m = \operatorname{argmin}_F \sum_{i=1}^N \omega_{i,m} \mathbb{I}(\tilde{y}_i \neq F(\mathbf{x}_i)) \quad (18.29)$$

This can be found by applying the weak learner to a weighted version of the dataset, with weights $\omega_{i,m}$.

All that remains is to solve for the size of the update, β . Substituting F_m into L_m and solving for β we find

$$\beta_m = \frac{1}{2} \log \frac{1 - \operatorname{err}_m}{\operatorname{err}_m} \quad (18.30)$$

where

$$\operatorname{err}_m = \frac{\sum_{i=1}^N \omega_{i,m} \mathbb{I}(\tilde{y}_i \neq F_m(\mathbf{x}_i))}{\sum_{i=1}^N \omega_{i,m}} \quad (18.31)$$

Therefore overall update becomes

$$f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \beta_m F_m(\mathbf{x}) \quad (18.32)$$

After updating the strong learner, we need to recompute the weights for the next iteration, as follows:

$$\omega_{i,m+1} = e^{-\tilde{y}_i f_m(\mathbf{x}_i)} = e^{-\tilde{y}_i f_{m-1}(\mathbf{x}_i) - \tilde{y}_i \beta_m F_m(\mathbf{x}_i)} = \omega_{i,m} e^{-\tilde{y}_i \beta_m F_m(\mathbf{x}_i)} \quad (18.33)$$

If $\tilde{y}_i = F_m(\mathbf{x}_i)$, then $\tilde{y}_i F_m(\mathbf{x}_i) = 1$, and if $\tilde{y}_i \neq F_m(\mathbf{x}_i)$, then $\tilde{y}_i F_m(\mathbf{x}_i) = -1$. Hence $-\tilde{y}_i F_m(\mathbf{x}_i) = 2\mathbb{I}(\tilde{y}_i \neq F_m(\mathbf{x}_i)) - 1$, so the update becomes

$$\omega_{i,m+1} = \omega_{i,m} e^{\beta_m (2\mathbb{I}(\tilde{y}_i \neq F_m(\mathbf{x}_i)) - 1)} = \omega_{i,m} e^{2\beta_m \mathbb{I}(\tilde{y}_i \neq F_m(\mathbf{x}_i))} e^{-\beta_m} \quad (18.34)$$

Since the $e^{-\beta_m}$ is constant across all examples, it can be dropped. If we then define $\alpha_m = 2\beta_m$, the update becomes

$$\omega_{i,m+1} = \begin{cases} \omega_{i,m} e^{\alpha_m} & \text{if } \tilde{y}_i \neq F_m(\mathbf{x}_i) \\ \omega_{i,m} & \text{otherwise} \end{cases} \quad (18.35)$$

Thus we see that we exponentially increase weights of misclassified examples. The resulting algorithm shown in Algorithm 18.1, and is known as **Adaboost.M1** [FS96].

A multiclass generalization of exponential loss, and an adaboost-like algorithm to minimize it, known as **SAMME** (stagewise additive modeling using a multiclass exponential loss function), is described in [Has+09]. This is implemented in scikit learn (the AdaBoostClassifier class).

Algorithm 18.1: Adaboost.M1, for binary classification with exponential loss

```

1  $\omega_i = 1/N$ 
2 for  $m = 1 : M$  do
3   Fit a classifier  $F_m(\mathbf{x})$  to the training set using weights  $\mathbf{w}$ 
4   Compute  $\text{err}_m = \frac{\sum_{i=1}^N \omega_{i,m} \mathbb{I}(\tilde{y}_i \neq F_m(\mathbf{x}_i))}{\sum_{i=1}^N \omega_{i,m}}$ 
5   Compute  $\alpha_m = \log[(1 - \text{err}_m)/\text{err}_m]$ 
6   Set  $\omega_i \leftarrow \omega_i \exp[\alpha_m \mathbb{I}(\tilde{y}_i \neq F_m(\mathbf{x}_i))]$ 
7 Return  $f(\mathbf{x}) = \text{sgn} \left[ \sum_{m=1}^M \alpha_m F_m(\mathbf{x}) \right]$ 

```

18.5.4 LogitBoost

The trouble with exponential loss is that it puts a lot of weight on misclassified examples, as is apparent from the exponential blowup on the left hand side of Figure 18.7. This makes the method very sensitive to outliers (misabeled examples). In addition, $e^{-\tilde{y}f}$ is not the logarithm of any pmf for binary variables $\tilde{y} \in \{-1, +1\}$; consequently we cannot recover probability estimates from $f(\mathbf{x})$.

A natural alternative is to use log loss, as we discussed in Section 18.5.3. This only punishes mistakes linearly, as is clear from Figure 18.7. Furthermore, it means that we will be able to extract probabilities from the final learned function, using

$$p(y = 1|\mathbf{x}) = \frac{e^{f(\mathbf{x})}}{e^{-f(\mathbf{x})} + e^{f(\mathbf{x})}} = \frac{1}{1 + e^{-2f(\mathbf{x})}} \quad (18.36)$$

The goal is to minimize the expected log-loss, given by

$$L_m(F) = \sum_{i=1}^N \log [1 + \exp (-2\tilde{y}_i(f_{m-1}(\mathbf{x}) + F(\mathbf{x}_i)))] \quad (18.37)$$

By performing a Newton update on this objective (similar to IRLS), one can derive the algorithm shown in Algorithm 18.2. This is known as **logitBoost** [FHT00]. The key subroutine is the ability of the weak learner F to solve a weighted least squares problem. This method can be generalized to the multi-class setting, as explained in [FHT00].

18.5.5 Gradient boosting

Rather than deriving new versions of boosting for every different loss function, it is possible to derive a generic version, known as **gradient boosting** [Fri01; Mas+00]. To explain this, imagine solving $\hat{\mathbf{f}} = \text{argmin}_{\mathbf{f}} \mathcal{L}(\mathbf{f})$ by performing gradient descent in the space of functions. Since functions are infinite dimensional objects, we will represent them by their values on the training set, $\mathbf{f} = (f(\mathbf{x}_1), \dots, f(\mathbf{x}_N))$. At step m , let $\mathbf{g}_m$ be the gradient of $\mathcal{L}(\mathbf{f})$ evaluated at $\mathbf{f} = \mathbf{f}_{m-1}$:

$$g_{im} = \left[\frac{\partial \ell(y_i, f(\mathbf{x}_i))}{\partial f(\mathbf{x}_i)} \right]_{f=f_{m-1}} \quad (18.38)$$

Algorithm 18.2: LogitBoost, for binary classification with log-loss

```

1  $\omega_i = 1/N, \pi_i = 1/2$ 
2 for  $m = 1 : M$  do
3   Compute the working response  $z_i = \frac{y_i^* - \pi_i}{\pi_i(1 - \pi_i)}$ 
4   Compute the weights  $\omega_i = \pi_i(1 - \pi_i)$ 
5    $F_m = \operatorname{argmin}_F \sum_{i=1}^N \omega_i (z_i - F(\mathbf{x}_i))^2$ 
6   Update  $f(\mathbf{x}) \leftarrow f(\mathbf{x}) + \frac{1}{2} F_m(\mathbf{x})$ 
7   Compute  $\pi_i = 1/(1 + \exp(-2f(\mathbf{x}_i)))$ ;
8 Return  $f(\mathbf{x}) = \operatorname{sgn} \left[ \sum_{m=1}^M F_m(\mathbf{x}) \right]$ 

```

Name	Loss	$-\partial \ell(y_i, f(\mathbf{x}_i)) / \partial f(\mathbf{x}_i)$
Squared error	$\frac{1}{2}(y_i - f(\mathbf{x}_i))^2$	$y_i - f(\mathbf{x}_i)$
Absolute error	$ y_i - f(\mathbf{x}_i) $	$\operatorname{sgn}(y_i - f(\mathbf{x}_i))$
Exponential loss	$\exp(-\tilde{y}_i f(\mathbf{x}_i))$	$-\tilde{y}_i \exp(-\tilde{y}_i f(\mathbf{x}_i))$
Binary Logloss	$\log(1 + e^{-\tilde{y}_i f_i})$	$y_i - \pi_i$
Multiclass logloss	$-\sum_c y_{ic} \log \pi_{ic}$	$y_{ic} - \pi_{ic}$

Table 18.1: Some commonly used loss functions and their gradients. For binary classification problems, we assume $\tilde{y}_i \in \{-1, +1\}$, and $\pi_i = \sigma(2f(\mathbf{x}_i))$. For regression problems, we assume $y_i \in \mathbb{R}$. Adapted from [HTF09, p360] and [BH07, p483].

Gradients of some common loss functions are given in Table 18.1. We then make the update

$$\mathbf{f}_m = \mathbf{f}_{m-1} - \beta_m \mathbf{g}_m \quad (18.39)$$

where β_m is the step length, chosen by

$$\beta_m = \operatorname{argmin}_{\beta} \mathcal{L}(\mathbf{f}_{m-1} - \beta \mathbf{g}_m) \quad (18.40)$$

In its current form, this is not much use, since it only optimizes f at a fixed set of N points, so we do not learn a function that can generalize. However, we can modify the algorithm by fitting a weak learner to approximate the negative gradient signal. That is, we use this update

$$F_m = \operatorname{argmin}_F \sum_{i=1}^N (-g_{im} - F(\mathbf{x}_i))^2 \quad (18.41)$$

The overall algorithm is summarized in Algorithm 18.3. We have omitted the line search step for β_m , which is not strictly necessary, as argued in [BH07]. However, we have introduced a learning rate or **shrinkage factor** $0 < \nu \leq 1$, to control the size of the updates, for regularization purposes.

If we apply this algorithm using squared loss, we recover L2Boosting, since $-g_{im} = y_i - f_{m-1}(\mathbf{x}_i)$ is just the residual error. We can also apply this algorithm to other loss functions, such as absolute loss or Huber loss (Section 5.1.5.3), which is useful for robust regression problems.

Algorithm 18.3: Gradient boosting

```

1 Initialize  $f_0(\mathbf{x}) = \operatorname{argmin}_F \sum_{i=1}^N L(y_i, F(\mathbf{x}_i))$ 
2 for  $m = 1 : M$  do
3   Compute the gradient residual using  $r_{im} = - \left[ \frac{\partial L(y_i, f(\mathbf{x}_i))}{\partial f(\mathbf{x}_i)} \right]_{f(\mathbf{x}_i)=f_{m-1}(\mathbf{x}_i)}$ 
4   Use the weak learner to compute  $F_m = \operatorname{argmin}_F \sum_{i=1}^N (r_{im} - F(\mathbf{x}_i))^2$ 
5   Update  $f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \nu F_m(\mathbf{x})$ 
6 Return  $f(\mathbf{x}) = f_M(\mathbf{x})$ 

```

For classification, we can use log-loss. In this case, we get an algorithm known as **BinomialBoost** [BH07]. The advantage of this over LogitBoost is that it does not need to be able to do weighted fitting: it just applies any black-box regression model to the gradient vector. To apply this to multi-class classification, we can fit C separate regression trees, using the pseudo residual of the form

$$-g_{icm} = \frac{\partial \ell(y_i, f_{1m}(\mathbf{x}_i), \dots, f_{Cm}(\mathbf{x}_i))}{\partial f_{cm}(\mathbf{x}_i)} = \mathbb{I}(y_i = c) - \pi_{ic} \quad (18.42)$$

Although the trees are fit separately, their predictions are combined via a softmax transform

$$p(y = c | \mathbf{x}) = \frac{e^{f_c(\mathbf{x})}}{\sum_{c'=1}^C e^{f_{c'}(\mathbf{x})}} \quad (18.43)$$

When we have large datasets, we can use a stochastic variant in which we subsample (without replacement) a random fraction of the data to pass to the regression tree at each iteration. This is called **stochastic gradient boosting** [Fri99]. Not only is it faster, but it can also generalize better, because subsampling the data is a form of regularization.

18.5.5.1 Gradient tree boosting

In practice, gradient boosting nearly always assumes that the weak learner is a regression tree, which is a model of the form

$$F_m(\mathbf{x}) = \sum_{j=1}^{J_m} w_{jm} \mathbb{I}(\mathbf{x} \in R_{jm}) \quad (18.44)$$

where w_{jm} is the predicted output for region R_{jm} . (In general, w_{jm} could be a vector.) This combination is called **gradient boosted regression trees**, or **gradient tree boosting**. (A related version is known as **MART**, which stands for “multivariate additive regression trees” [FM03].)

To use this in gradient boosting, we first find good regions R_{jm} for tree m using standard regression tree learning (see Section 18.1) on the residuals; we then (re)solve for the weights of each leaf by solving

$$\hat{w}_{jm} = \operatorname{argmin}_w \sum_{\mathbf{x}_i \in R_{jm}} \ell(y_i, f_{m-1}(\mathbf{x}_i) + w) \quad (18.45)$$

For squared error (as used by gradient boosting), the optimal weight $\hat{w}_{jm}$ is just the mean of the residuals in that leaf.

18.5.5.2 XGBoost

XGBoost (<https://github.com/dmlc/xgboost>), which stands for “extreme gradient boosting”, is a very efficient and widely used implementation of gradient boosted trees, that adds a few more improvements beyond the description in Section 18.5.5.1. The details can be found in [CG16], but in brief, the extensions are as follows: it adds a regularizer on the tree complexity, it uses a second order approximation of the loss (from [FHT00]) instead of just a linear approximation, it samples features at internal nodes (as in random forests), and it uses various computer science methods (such as handling out-of-core computation for large datasets) to ensure scalability.²

In more detail, XGBoost optimizes the following regularized objective

$$\mathcal{L}(f) = \sum_{i=1}^N \ell(y_i, f(\mathbf{x}_i)) + \Omega(f) \quad (18.46)$$

where

$$\Omega(f) = \gamma J + \frac{1}{2} \lambda \sum_{j=1}^J w_j^2 \quad (18.47)$$

is the regularizer, where J is the number of leaves, and $\gamma \geq 0$ and $\lambda \geq 0$ are regularization coefficients. At the m 'th step, the loss is given by

$$\mathcal{L}_m(F_m) = \sum_{i=1}^N \ell(y_i, f_{m-1}(\mathbf{x}_i) + F_m(\mathbf{x}_i)) + \Omega(F_m) + \text{const} \quad (18.48)$$

We can compute a second order Taylor expansion of this as follows:

$$\mathcal{L}_m(F_m) \approx \sum_{i=1}^N \left[\ell(y_i, f_{m-1}(\mathbf{x}_i)) + g_{im} F_m(\mathbf{x}_i) + \frac{1}{2} h_{im} F_m^2(\mathbf{x}_i) \right] + \Omega(F_m) + \text{const} \quad (18.49)$$

where h_{im} is the Hessian

$$h_{im} = \left[\frac{\partial^2 \ell(y_i, f(\mathbf{x}_i))}{\partial f(\mathbf{x}_i)^2} \right]_{f=f_{m-1}} \quad (18.50)$$

In the case of regression trees, we have $F(\mathbf{x}) = w_{q(\mathbf{x})}$, where $q : \mathbb{R}^D \rightarrow \{1, \dots, J\}$ specifies which leaf node $\mathbf{x}$ belongs to, and $\mathbf{w} \in \mathbb{R}^J$ are the leaf weights. Hence we can rewrite Equation (18.49) as

2. Some other popular gradient boosted trees packages are **CatBoost** (<https://catboost.ai/>) and **LightGBM** (<https://github.com/Microsoft/LightGBM>).

follows, dropping terms that are independent of F_m :

$$\mathcal{L}_m(q, \mathbf{w}) \approx \sum_{i=1}^N \left[g_{im} F_m(\mathbf{x}_i) + \frac{1}{2} h_{im} F_m^2(\mathbf{x}_i) \right] + \gamma J + \frac{1}{2} \lambda \sum_{j=1}^J w_j^2 \quad (18.51)$$

$$= \sum_{j=1}^J \left[\left(\sum_{i \in I_j} g_{im} \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} h_{im} + \lambda \right) w_j^2 \right] + \gamma J \quad (18.52)$$

where $I_j = \{i : q(\mathbf{x}_i) = j\}$ is the set of indices of data points assigned to the j 'th leaf.

Let us define $G_{jm} = \sum_{i \in I_j} g_{im}$ and $H_{jm} = \sum_{i \in I_j} h_{im}$. Then the above simplifies to

$$\mathcal{L}_m(q, \mathbf{w}) = \sum_{j=1}^J \left[G_{jm} w_j + \frac{1}{2} (H_{jm} + \lambda) w_j^2 \right] + \gamma J \quad (18.53)$$

This is a quadratic in each w_j , so the optimal weights are given by

$$w_j^* = -\frac{G_{jm}}{H_{jm} + \lambda} \quad (18.54)$$

The loss for evaluating different tree structures q then becomes

$$\mathcal{L}_m(q, \mathbf{w}^*) = -\frac{1}{2} \sum_{j=1}^J \frac{G_{jm}^2}{H_{jm} + \lambda} + \gamma J \quad (18.55)$$

We can greedily optimize this using a recursive node splitting procedure, as in Section 18.1. Specifically, for a given leaf j , we consider splitting it into a left and right half, $I = I_L \cup I_R$. We can compute the gain (reduction in loss) of such a split as follows:

$$\text{gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{(H_L + H_R) + \lambda} \right] - \gamma \quad (18.56)$$

where $G_L = \sum_{i \in I_L} g_{im}$, $G_R = \sum_{i \in I_R} g_{im}$, $H_L = \sum_{i \in I_L} h_{im}$, and $H_R = \sum_{i \in I_R} h_{im}$. Thus we see that it is not worth splitting a node if the gain is negative (i.e., the first term is less than γ).

A fast approximation for evaluating this objective, that does not require sorting the features (for choosing the optimal threshold to split on), is described in [CG16].

18.6 Interpreting tree ensembles

Trees are popular because they are interpretable. Unfortunately, ensembles of trees (whether in the form of bagging, random forests, or boosting) lose that property. Fortunately, there are some simple methods we can use to interpret what function has been learned.

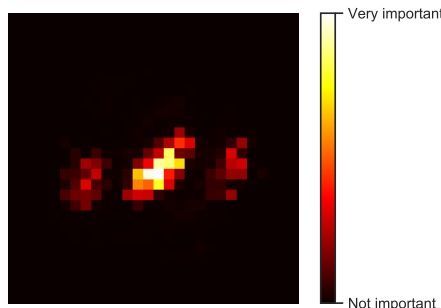


Figure 18.8: Feature importance of a random forest classifier trained to distinguish MNIST digits from classes 0 and 8. Adapted from Figure 7.6 of [Gér19]. Generated by `rf_feature_importance_mnist.ipynb`.

18.6.1 Feature importance

For a single decision tree T , [BFO84] proposed the following measure for **feature importance** of feature k :

$$R_k(T) = \sum_{j=1}^{J-1} G_j \mathbb{I}(v_j = k) \quad (18.57)$$

where the sum is over all non-leaf (internal) nodes, G_j is the gain in accuracy (reduction in cost) at node j , and $v_j = k$ if node j uses feature k . We can get a more reliable estimate by averaging over all trees in the ensemble:

$$R_k = \frac{1}{M} \sum_{m=1}^M R_k(T_m) \quad (18.58)$$

After computing these scores, we can normalize them so the largest value is 100%. We give some examples below.

Figure 18.8 gives an example of estimating feature importance for a classifier trained to distinguish MNIST digits from classes 0 and 8. We see that it focuses on the parts of the image that differ between these classes.

In Figure 18.9, we plot the relative importance of each of the features for the spam dataset (Section 18.4). Not surprisingly, we find that the most important features are the words “george” (the name of the recipient) and “hp” (the company he worked for), as well as the characters ! and \$. (Note it can be the presence or absence of these features that is informative.)

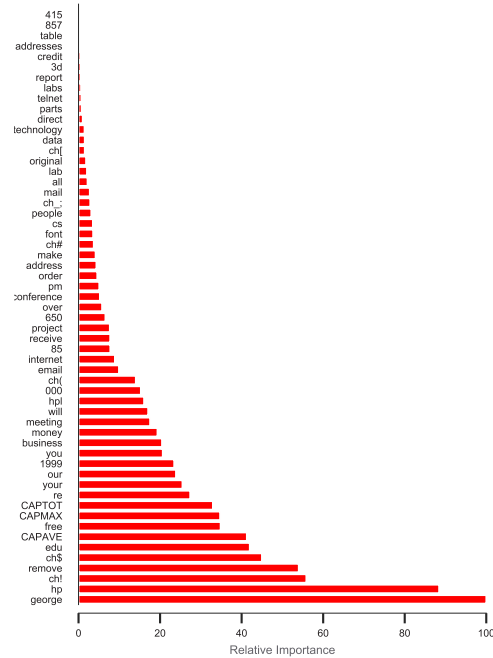


Figure 18.9: Feature importance of a gradient boosted classifier trained to distinguish spam from non-spam email. The dataset has X training examples with Y features, corresponding to token frequency. Adapted from Figure 10.6 of [HTF09]. Generated by `spam_tree_ensemble_interpret.ipynb`.

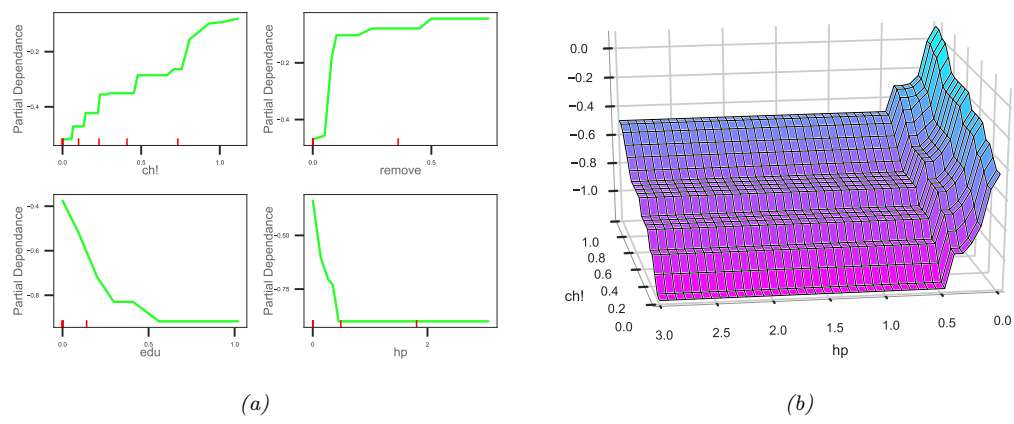


Figure 18.10: (a) Partial dependence of log-odds of the spam class for 4 important predictors. The red ticks at the base of the plot are deciles of the empirical distribution for this feature. (b) Joint partial dependence of log-odds on the features hp and $!$. Adapted from Figure 10.6–10.8 of [HTF09]. Generated by `spam_tree_ensemble_interpret.ipynb`.

18.6.2 Partial dependency plots

After we have identified the most relevant input features, we can try to assess the impact they have on the output. A **partial dependency plot** for feature k is a plot of

$$\bar{f}_k(x_k) = \frac{1}{N} \sum_{n=1}^N f(\mathbf{x}_{n,-k}, x_k) \quad (18.59)$$

vs x_k . Thus we marginalize out all features except k . In the case of a binary classifier, we can convert this to log odds, $\log p(y=1|x_k)/p(y=0|x_k)$, before plotting. We illustrate this for our spam example in Figure 18.10a for 4 different features. We see that as the frequency of “!” and “remove” increases, so does the probability of spam. Conversely, as the frequency of “edu” or “hp” increases, the probability of spam decreases.

We can also try to capture interaction effects between features j and k by computing

$$\bar{f}_{jk}(x_j, x_k) = \frac{1}{N} \sum_{n=1}^N f(\mathbf{x}_{n,-jk}, x_j, x_k) \quad (18.60)$$

We illustrate this for our spam example in Figure 18.10b for hp and !. We see that higher frequency of “!” makes it more likely to be spam, but much more so if the word “hp” is missing.

PART V

Beyond Supervised Learning

19 Learning with Fewer Labeled Examples

Many ML models, especially neural networks, often have many more parameters than we have labeled training examples. For example, a ResNet CNN (Section 14.3.4) with 50 layers has 23 million parameters. Transformer models (Section 15.5) can be even bigger. Of course these parameters are highly correlated, so they are not independent “degrees of freedom”. Nevertheless, such big models are slow to train and, more importantly, they may easily overfit. This is particularly a problem when you do not have a large labeled training set. In this chapter, we discuss some ways to tackle this issue, beyond the generic regularization techniques we discussed in Section 13.5 such as early stopping, weight decay and dropout.

19.1 Data augmentation

Suppose we just have a single small labeled dataset. In some cases, we may be able to create artificially modified versions of the input vectors, which capture the kinds of variations we expect to see at test time, while keeping the original labels unchanged. This is called **data augmentation**.¹ We give some examples below, and then discuss why this approach works.

19.1.1 Examples

For image classification tasks, standard data augmentation methods include random crops, zooms, and mirror image flips, as illustrated in Figure 19.1. [GVZ16] gives a more sophisticated example, where they render text characters onto an image in a realistic way, thereby creating a very large dataset of text “in the wild”. They used this to train a state of the art visual text localization and reading system. Other examples of data augmentation include artificially adding background noise to clean speech signals, and artificially replacing characters or words at random in text documents.

If we afford to train and test the model many times using different versions of the data, we can learn which augmentations work best, using blackbox optimization methods such as RL (see e.g., [Cub+19]) or Bayesian optimization (see e.g., [Lim+19]); this is called **AutoAugment**. We can also learn to combine multiple augmentations together; this is called **AutoAugment** [Cub+19].

For some examples of augmentation in NLP, see e.g., [Fen+21].

1. The term “data augmentation” is also used in statistics to mean the addition of auxiliary latent variables to a model in order to speed up convergence of posterior inference algorithms [DM01].



Figure 19.1: Illustration of random crops and zooms of the image on the left. Generated by `image_augmentation_jax.ipynb`.

19.1.2 Theoretical justification

Data augmentation often significantly improves performance (predictive accuracy, robustness, etc). At first this might seem like we are getting something for nothing, since we have not provided additional data. However, the data augmentation mechanism can be viewed as a way to algorithmically inject prior knowledge.

To see this, recall that in standard ERM training, we minimize the empirical risk

$$R(f) = \int \ell(f(\mathbf{x}), \mathbf{y}) p^*(\mathbf{x}, \mathbf{y}) d\mathbf{x} d\mathbf{y} \quad (19.1)$$

where we approximate $p^*(\mathbf{x}, \mathbf{y})$ by the empirical distribution

$$p_{\mathcal{D}}(\mathbf{x}, \mathbf{y}) = \frac{1}{N} \sum_{n=1}^N \delta(\mathbf{x} - \mathbf{x}_n) \delta(\mathbf{y} - \mathbf{y}_n) \quad (19.2)$$

We can think of data augmentation as replacing the empirical distribution with the following algorithmically smoothed distribution

$$p_{\mathcal{D}}(\mathbf{x}, \mathbf{y} | A) = \frac{1}{N} \sum_{n=1}^N p(\mathbf{x} | \mathbf{x}_n, A) \delta(\mathbf{y} - \mathbf{y}_n) \quad (19.3)$$

where A is the data augmentation algorithm, which generates a sample $\mathbf{x}$ from a training point $\mathbf{x}_n$, such that the label (“semantics”) is not changed. (A very simple example would be a Gaussian kernel, $p(\mathbf{x} | \mathbf{x}_n, A) = \mathcal{N}(\mathbf{x} | \mathbf{x}_n, \sigma^2 \mathbf{I})$.) This has been called **vicinal risk minimization** [Cha+01], since we are minimizing the risk in the vicinity of each training point $\mathbf{x}$. For more details on this perspective, see [Zha+17b; CDL19; Dao+19].

19.2 Transfer learning

This section is coauthored with Colin Raffel.

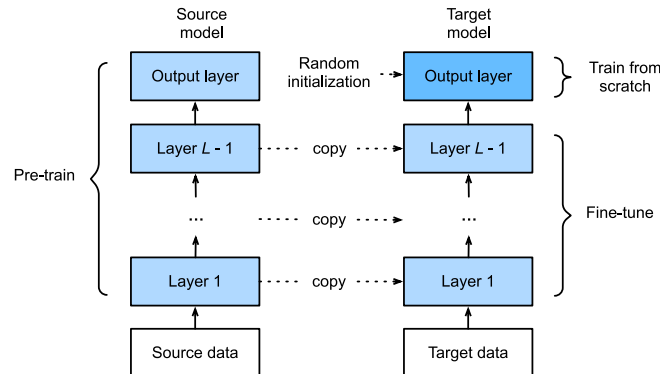


Figure 19.2: Illustration of fine-tuning a model on a new dataset. The final output layer is trained from scratch, since it might correspond to a different label set. The other layers are initialized at their previous parameters, and then optionally updated using a small learning rate. From Figure 13.2.1 of [Zha+20]. Used with kind permission of Aston Zhang.

Many data-poor tasks have some high-level structural similarity to other data-rich tasks. For example, consider the task of **fine-grained visual classification** of endangered bird species. Given that endangered birds are by definition rare, it is unlikely that a large quantity of diverse labeled images of these birds exist. However, birds bear many structural similarities across species - for example, most birds have wings, feathers, beaks, claws, etc. We therefore might expect that first training a model on a large dataset of non-endangered bird species and then continuing to train it on a small dataset of endangered species could produce better performance than training on the small dataset alone.

This is called **transfer learning**, since we are transferring information from one dataset to another, via a shared set of parameters. More precisely, we first perform a **pre-training phase**, in which we train a model with parameters θ on a large **source dataset** $\mathcal{D}_p$; this may be labeled or unlabeled. We then perform a second **fine-tuning phase** on the small labeled **target dataset** $\mathcal{D}_q$ of interest. We discuss these two phases in more detail below, but for more information, see e.g., [Tan+18; Zhu+21] for recent surveys.

19.2.1 Fine-tuning

Suppose, for now, that we already have a pretrained classifier, $p(y|\mathbf{x}, \theta_p)$, such as a CNN, that works well for inputs $\mathbf{x} \in \mathcal{X}_p$ (e.g. natural images) and outputs $y \in \mathcal{Y}_p$ (e.g., ImageNet labels), where the data comes from a distribution $p(\mathbf{x}, y)$ similar to the one used in training. Now we want to create a new model $q(y|\mathbf{x}, \theta_q)$ that works well for inputs $\mathbf{x} \in \mathcal{X}_q$ (e.g. bird images) and outputs $y \in \mathcal{Y}_q$ (e.g., fine-grained bird labels), where the data comes from a distribution $q(\mathbf{x}, y)$ which may be different from p .

We will assume that the set of possible inputs is the same, so $\mathcal{X}_q \approx \mathcal{X}_p$ (e.g., both are RGB images), or that we can easily transform inputs from domain p to domain q (e.g., we can convert an RGB image to grayscale by dropping the chrominance channels and just keeping luminance). (If this is not

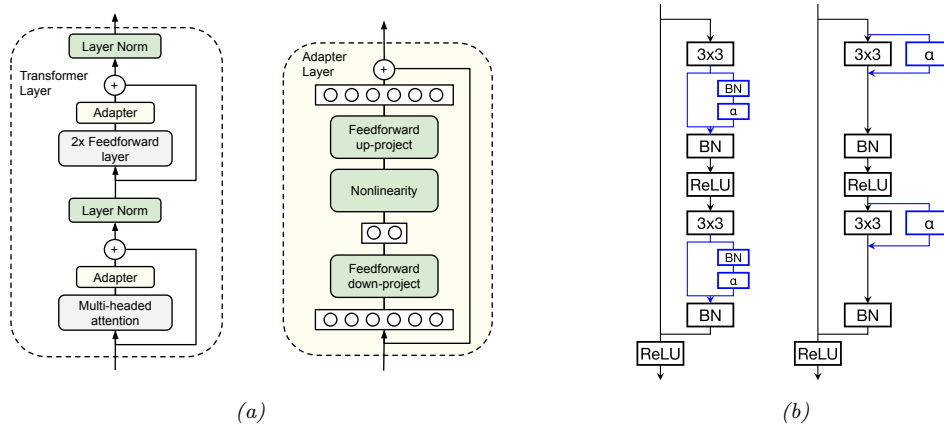


Figure 19.3: (a) Adding adapter layers to a transformer. From Figure 2 of [Hou+19]. Used with kind permission of Neil Houlsby. (b) Adding adapter layers to a resnet. From Figure 2 of [RBV18]. Used with kind permission of Sylvestre-Alvise Rebuffi.

the case, then we may need to use a method called domain adaptation, that modifies models to map between modalities, as discussed in Section 19.2.5.)

However, the output domains are usually different, i.e., $\mathcal{Y}_q \neq \mathcal{Y}_p$. For example, $\mathcal{Y}_p$ might be Imagenet labels and $\mathcal{Y}_q$ might be medical labels (e.g., types of diabetic retinopathy [Arc+19]). In this case, we need to “translate” the output of the pre-trained model to the new domain. This is easy to do with neural networks: we simply “chop off” the final layer of the original model, and add a new “head” to model the new class labels, as illustrated in Figure 19.2. For example, suppose $p(y|\mathbf{x}, \boldsymbol{\theta}_p) = \text{softmax}(y|\mathbf{W}_2\mathbf{h}(\mathbf{x}; \boldsymbol{\theta}_1) + \mathbf{b}_2)$, where $\boldsymbol{\theta}_p = (\mathbf{W}_2, \mathbf{b}_2, \boldsymbol{\theta}_1)$. Then we can construct $q(y|\boldsymbol{\theta}_q) = \text{softmax}(y|\mathbf{W}_3\mathbf{h}(\mathbf{x}; \boldsymbol{\theta}_1) + \mathbf{b}_3)$, where $\boldsymbol{\theta}_q = (\mathbf{W}_3, \mathbf{b}_3, \boldsymbol{\theta}_1)$ and $\mathbf{h}(\mathbf{x}; \boldsymbol{\theta}_1)$ is the shared nonlinear feature extractor.

After performing this “model surgery”, we can fine-tune the new model with parameters $\boldsymbol{\theta}_q = (\boldsymbol{\theta}_1, \boldsymbol{\theta}_3)$, where $\boldsymbol{\theta}_1$ parameterizes the feature extractor, and $\boldsymbol{\theta}_3$ parameterizes the final linear layer that maps features to the new set of labels. If we treat $\boldsymbol{\theta}_1$ as “frozen parameters”, then the resulting model $q(y|\mathbf{x}, \boldsymbol{\theta}_q)$ is linear in its parameters, so we have a convex optimization problem for which many simple and efficient fitting methods exist (see Part II). This is particularly helpful in the long-tail setting, where some classes are very rare [Kan+20]. However, a linear “decoder” may be too limiting, so we can also allow $\boldsymbol{\theta}_1$ to be fine-tuned as well, but using a lower learning rate, to prevent the values moving too far from the values estimated on $\mathcal{D}_p$.

19.2.2 Adapters

One disadvantage of fine-tuning all the model parameters of a pre-trained model is that it can be slow, since there are often many parameters, and we may need to use a small learning rate to prevent the low-level feature extractors from diverging too far from their prior values. In addition, every new task requires a new model to be trained, making task sharing hard. An alternative approach is to keep the pre-trained model untouched, but to add new parameters to modify its internal behavior to